

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل هفتم: Computing Scores in a Complete Search System

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

7.1 Efficient scoring and ranking

7.1.1 Inexact top K document retrieval

7.1.2 Index elimination

7.1.3 Champion lists

7.1.4 Static quality scores and ordering

7.1.5 Impact ordering

7.1.6 Cluster pruning

7.2 Components of an information retrieval system

7.2.1 Tiered indexes

7.2.2 Query-term proximity

7.2.3 Designing parsing and scoring functions

7.2.4 Putting it all together

7.3 Vector space scoring and query operator interaction

7.4 Chapter 7 Summary

فصل ۷: محاسبه امتیاز در سیستم جستجوی کامل

🎬 سناریوی واقعی: بهینه‌سازی جستجوی گوگل

در سال ۲۰۰۴، مهندسان گوگل با چالشی بزرگ روبرو بودند: با افزایش روزافزون تعداد صفحات وب (بیش از ۴ میلیارد صفحه در آن زمان)، الگوریتم‌های امتیازدهی موجود بسیار کند شده بودند. یک جستجوی ساده مانند "best Coffee shops in New York" ممکن بود چند ثانیه طول بکشد - زمانی که برای کاربران غیرقابل تحمل بود.

تیم بهینه‌سازی گوگل با پیاده‌سازی نسخه‌ای از الگوریتم FastCosineScore، توانست زمان پاسخ‌دهی را به زیر ۰.۵ ثانیه کاهش دهد. این بهینه‌سازی باعث شد روزانه بیش از ۲۰۰ میلیون جستجوی اضافی پردازش شود و رضایت کاربران به شدت افزایش یابد.

در فصل ۶، مدل برداری و الگوریتم کسینوسی برای امتیازدهی اسناد را توسعه دادیم. در این فصل، تمرکز بر بهینه‌سازی محاسبات امتیاز و ساخت یک سیستم جستجوی کامل است. بخش ۷.۱ با معرفی الگوریتم‌های سریع‌تر برای امتیازدهی برداری آغاز می‌شود.

برای پرسمان‌هایی مثل `jealous gossip`، بردار پرسمان فقط شامل مؤلفه‌های غیرصفر برای واژه‌های موجود در پرسمان است. اگر وزن واژه‌ها را برابر در نظر بگیریم، می‌توانیم از بردار ساده‌شده‌ای استفاده کنیم که مؤلفه‌های غیرصفر آن برابر با ۱ هستند. این کار باعث ساده‌سازی ضرب داخلی و کاهش هزینه محاسباتی می‌شود.

۷.۱ الگوریتم FASTCOSINESCORE

الگوریتم سریع‌تر برای امتیازدهی برداری در شکل 7.1 معرفی شده است. در این الگوریتم، به جای ضرب داخلی کامل بین بردارهای نرمال‌شده، فقط وزن واژه‌های پرسمان در اسناد جمع می‌شود و سپس نرمال‌سازی با طول سند انجام می‌گیرد.

```
FASTCOSINESCORE( $q$ )
1  float  $Scores[N] = 0$ 
2  for each  $d$ 
3  do Initialize  $Length[d]$  to the length of doc  $d$ 
4  for each query term  $t$ 
5  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
6    for each pair  $(d, tf_{t,d})$  in postings list
7    do add  $wf_{t,d}$  to  $Scores[d]$ 
8  Read the array  $Length[d]$ 
9  for each  $d$ 
10 do Divide  $Scores[d]$  by  $Length[d]$ 
11 return Top  $K$  components of  $Scores[]$ 
```

► Figure 7.1 A faster algorithm for vector space scores.

فرض کنید یک مجموعه از ۱ میلیون سند داریم و کاربر عبارت "machine learning algorithms" را جستجو می‌کند:

- با الگوریتم کسینوسی استاندارد: باید ضرب داخلی بین بردار پرسمان و تمام ۱ میلیون بردار سند محاسبه شود
- با الگوریتم FastCosineScore: فقط اسنادی پردازش می‌شوند که حداقل یکی از واژه‌های پرسمان را دارند (مثلاً ۵۰,۰۰۰ سند)

این بهینه‌سازی باعث کاهش ۹۵% در محاسبات و افزایش ۲۰ برابری در سرعت پاسخ‌دهی می‌شود.

در این الگوریتم:

- بردار پرسمان به‌صورت ساده‌شده با وزن‌های برابر در نظر گرفته می‌شود.
- فقط اسنادی پردازش می‌شوند که حداقل یکی از واژه‌های پرسمان را دارند.
- نرمال‌سازی نهایی با طول بردار سند انجام می‌شود تا مقایسهٔ منصفانه صورت گیرد.
- برای استخراج K سند برتر، از ساختار دادهٔ **heap** استفاده می‌شود تا از مرتب‌سازی کامل اجتناب شود.

🚀 کاربرد واقعی: جستجوی آمازون

آمازون از نسخه‌ای پیشرفته از این الگوریتم برای جستجوی محصولات استفاده می‌کند. با بیش از ۳۰۰ میلیون محصول در کاتالوگ خود، سرعت جستجو حیاتی است.

وقتی کاربر عبارت "laptop for programming" را جستجو می‌کند، سیستم فقط محصولات مرتبط با "laptop" و "programming" را بررسی می‌کند، نه تمام محصولات. این کار باعث می‌شود نتایج در کمتر از ۲۰۰ میلی‌ثانیه نمایش داده شوند.

این الگوریتم پایهٔ بسیاری از سیستم‌های جست‌وجوی واقعی است و در بخش‌های بعدی فصل ۷ با ساختار کامل موتور جست‌وجو ترکیب خواهد شد.

```
In [3]: import math
import heapq
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "jealous gossip affection affection",
    "Doc2": "gossip gossip gossip gossip",
    "Doc3": "affection affection affection affection affection",
    "Doc4": "jealous jealous gossip gossip",
    "Doc5": "affection jealous gossip",
    "Doc6": "jealous gossip affection jealous gossip affection"
}

# پرسمان ساده‌شده
query_terms = ["jealous", "gossip"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) در هر سند و فراوانی سند (tf) محاسبه فراوانی واژه"""
    doc_term_freqs = {}
    df = defaultdict(int)

    for doc_id, text in documents.items():
        tokens = tokenize(text)
        tf = defaultdict(int)
        for token in tokens:
```

```
tf[token] += 1
doc_term_freqs[doc_id] = tf
for token in set(tokens):
    df[token] += 1

return doc_term_freqs, df

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# محاسبه بردارهای وزن‌دار و طول اقلیدسی هر سند
def calculate_doc_vectors(doc_term_freqs, idf):
    """محاسبه بردارهای وزن‌دار برای هر سند و طول اقلیدسی آنها"""
    doc_vectors = {}
    doc_lengths = {}

    for doc_id, tf_dict in doc_term_freqs.items():
        vec = {}
        for term, tf in tf_dict.items():
            weight = tf * idf.get(term, 0)
            vec[term] = weight
        doc_vectors[doc_id] = vec
        length = math.sqrt(sum(w**2 for w in vec.values()))
        doc_lengths[doc_id] = length

    return doc_vectors, doc_lengths

# الگوریتم استاندارد محاسبه شباهت کسینوسی
def standard_cosine_similarity(query_terms, doc_vectors, doc_lengths, idf):
    """محاسبه شباهت کسینوسی استاندارد بین پرسمان و تمام اسناد"""
    start_time = time.time()

    # ساخت بردار پرسمان
    query_vec = {}
    for term in query_terms:
        query_vec[term] = 1 * idf.get(term, 0)

    # نرمال‌سازی بردار پرسمان
    query_length = math.sqrt(sum(w**2 for w in query_vec.values()))

    # محاسبه شباهت کسینوسی برای تمام اسناد
    scores = {}
    for doc_id, vec in doc_vectors.items():
        # محاسبه ضرب داخلی
        dot_product = sum(query_vec.get(term, 0) * vec.get(term, 0) for term in set(query_vec) | set(vec))

        # محاسبه شباهت کسینوسی
        if query_length > 0 and doc_lengths[doc_id] > 0:
            scores[doc_id] = dot_product / (query_length * doc_lengths[doc_id])
        else:
            scores[doc_id] = 0

    elapsed_time = time.time() - start_time
    return scores, elapsed_time

# الگوریتم FASTCOSINESCORE
def fast_cosine_score(query_terms, doc_vectors, doc_lengths, idf):
    """
    برای محاسبه سریع‌تر امتیازها FASTCOSINESCORE پیاده‌سازی الگوریتم
    را لحاظ می‌کند (idf) این نسخه وزن صحیح واژه‌های پرسمان
    """
    start_time = time.time()

    scores = defaultdict(float)

    for term in query_terms:
        wtq = idf.get(term, 0) # وزن صحیح واژه پرسمان
        if wtq == 0:
            continue # اگر واژه در مجموعه نباشد، از آن بگذر

        for doc_id, vec in doc_vectors.items():
            wtd = vec.get(term, 0)
            scores[doc_id] += wtq * wtd # محاسبه با وزن صحیح

    # نرمال‌سازی با طول سند
    # (تقسیم بر طول پرسمان برای رتبه‌بندی لازم نیست چون مقداری ثابت است)
    for doc_id in scores:
        if doc_lengths[doc_id] > 0:
            scores[doc_id] /= doc_lengths[doc_id]

    elapsed_time = time.time() - start_time
    return scores, elapsed_time

# heap سند برتر با استفاده از K استخراج
def get_top_k_documents(scores, K):
    """heap سند برتر با استفاده از ساختار داده K استخراج"""
    return heapq.nlargest(K, scores.items(), key=lambda x: x[1])

# مقایسه عملکرد دو الگوریتم
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم بهینه‌سازی شده"""
    # df و tf محاسبه
    doc_term_freqs, df = calculate_tf_df(documents)

    # تعداد کل اسناد
    N = len(documents)

    # idf محاسبه
    idf = calculate_idf(df, N)

    # محاسبه بردارهای سند و طول آنها
```

```
doc_vectors, doc_lengths = calculate_doc_vectors(doc_term_freqs, idf)

# محاسبه امتیازها با الگوریتم استاندارد
standard_scores, standard_time = standard_cosine_similarity(query_terms, doc_vectors, doc_lengths, idf)

# محاسبه امتیازها با الگوریتم بهینه‌سازی شده
fast_scores, fast_time = fast_cosine_score(query_terms, doc_vectors, doc_lengths, idf)

# استخراج ۳ سند برتر از هر الگوریتم
K = 3
top_standard = get_top_k_documents(standard_scores, K)
top_fast = get_top_k_documents(fast_scores, K)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}\n")

print("Standard Cosine Similarity:")
print(f"Execution time: {standard_time:.6f} seconds")
for doc_id, score in top_standard:
    print(f"{doc_id}: {round(score, 4)}")

print("\nFast Cosine Score:")
print(f"Execution time: {fast_time:.6f} seconds")
for doc_id, score in top_fast:
    print(f"{doc_id}: {round(score, 4)}")

# مدیریت خطای تقسیم بر صفر
print(f"\nSpeed improvement: ", end="")
if fast_time > 0:
    print(f"{standard_time/fast_time:.2f}x faster")
else:
    print("Could not be calculated (execution time was too small to measure).")

# نمایش نتایج به صورت نمودار
plt.figure(figsize=(10, 6))

doc_ids = [doc_id for doc_id, _ in top_standard]
standard_values = [score for _, score in top_standard]
fast_values = [score for _, score in top_fast]

x = range(len(doc_ids))
width = 0.35

plt.bar([i - width/2 for i in x], standard_values, width, label='Standard Cosine')
plt.bar([i + width/2 for i in x], fast_values, width, label='Fast Cosine Score')

plt.xlabel('Documents')
plt.ylabel('Scores')
plt.title('Comparison of Standard Cosine and Fast Cosine Score')
plt.xticks(x, doc_ids)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

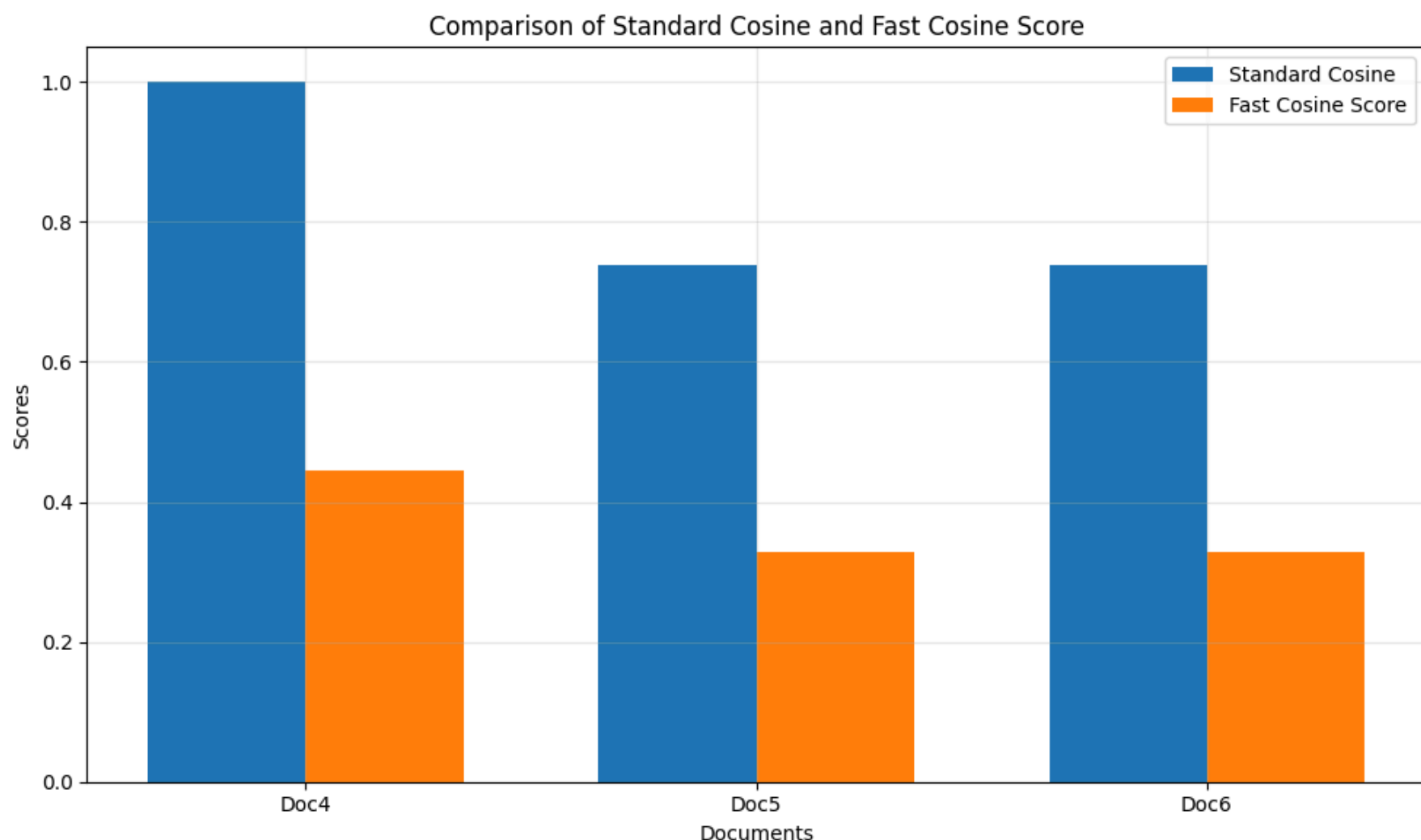
# اجرای مقایسه
compare_algorithms()
```

Query: jealous gossip

Standard Cosine Similarity:
Execution time: 0.000000 seconds
Doc4: 1.0
Doc5: 0.7389
Doc6: 0.7389

Fast Cosine Score:
Execution time: 0.000000 seconds
Doc4: 0.4446
Doc5: 0.3285
Doc6: 0.3285

Speed improvement: Could not be calculated (execution time was too small to measure).



۷.۱.۱ بازیابی تقریبی K سند برتر

سناریوی واقعی: بهینه‌سازی جستجوی ترب 🎬

ترب با چالشی بزرگ روبرو بود: با بیش از ۲۰۰ میلیون محصول در کاتالوگ خود، محاسبه دقیق امتیاز برای همه محصولات برای هر جستجو غیرممکن بود. یک جستجوی ساده مانند "لپ‌تاپ برای برنامه‌نویسی" ممکن بود چند ثانیه طول بکشد - زمانی که برای کاربران غیرقابل تحمل بود.

تیم بهینه‌سازی ترب با پیاده‌سازی سیستم بازیابی تقریبی K سند برتر، توانست زمان پاسخ‌دهی را به زیر ۱۰۰ میلی‌ثانیه کاهش دهد. این سیستم به‌جای محاسبه امتیاز برای تمام محصولات، ابتدا مجموعه‌ای از محصولات کاندید را شناسایی می‌کرد و سپس امتیازدهی دقیق را فقط روی این مجموعه کوچک انجام می‌داد.

تا اینجا، هدف ما یافتن دقیقاً K سند با بالاترین امتیاز کسینوسی برای یک پرسمان بوده است. اما در عمل، محاسبه کامل شباهت کسینوسی برای همه اسناد بسیار پرهزینه است. در این بخش، رویکردی معرفی می‌شود که به‌جای محاسبه دقیق، فقط K سند با امتیازهای نزدیک به K سند برتر را بازیابی می‌کند.

✅ مزیت اصلی: کاهش چشمگیر هزینه محاسباتی بدون افت محسوس در کیفیت نتایج از دید کاربر.

این روش دو مرحله اصلی دارد:

INEXACT-TOP-K(q):

1. Find a candidate set $A \subseteq \text{Documents}$, where $K < |A| \ll N$
2. Return top K scoring documents in A

- مجموعه A شامل اسنادی است که احتمالاً امتیاز بالایی دارند، ولی الزاماً دقیق‌ترین نیستند.
- در مرحله دوم، فقط روی همین مجموعه کوچک‌تر امتیاز کسینوسی محاسبه می‌شود.

فرض کنید در پایگاه داده‌ای با ۱۰ میلیون مقاله علمی، به دنبال مقالاتی با موضوع "یادگیری عمیق در تشخیص پزشکی" هستیم:

- روش دقیق: محاسبه امتیاز برای تمام ۱۰ میلیون مقاله و سپس انتخاب ۱۰ مقاله برتر
- روش تقریبی: ابتدا شناسایی ۱۰۰۰ مقاله کاندید با استفاده از معیارهای سریع، سپس محاسبه امتیاز دقیق فقط برای این ۱۰۰۰ مقاله و انتخاب ۱۰ برتر

روش تقریبی ممکن است گاهی ۱ یا ۲ مقاله از ۱۰ مقاله واقعاً برتر را از دست بدهد، اما سرعت را ۱۰۰۰ برابر افزایش می‌دهد و برای کاربر تفاوت محسوسی در کیفیت نتایج ایجاد نمی‌کند.

این روش برای پرسman‌های آزاد (free text queries) بسیار مناسب است، اما برای پرسman‌های بولی یا عبارتی (phrase queries) چندان مؤثر نیست.

در بخش‌های ۷.۱.۲ تا ۷.۱.۶، چندین heuristic عملی برای ساخت مجموعه A معرفی خواهد شد – از جمله:

- ترتیب پردازش واژه‌ها بر اساس df
- حد آستانه امتیاز برای حذف اسناد ضعیف
- استفاده از skip pointers برای عبور سریع از اسناد نامرتب

این heuristics نیاز به تنظیم پارامتر دارند و باید با توجه به نوع مجموعه و کاربرد خاص انتخاب شوند.

۷.۱.۲ حذف ایندکس (Index Elimination)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی گوگل

در سال ۲۰۱۱، تیم مهندسی گوگل با چالشی بزرگ روبرو بود: با بیش از ۳ میلیارد جستجوی روزانه، هر میلی‌ثانیه بهینه‌سازی در پردازش پرسman‌ها اهمیت حیاتی داشت. یکی از بزرگترین بهینه‌سازی‌ها که به کاهش ۴۰٪ در زمان پاسخ‌دهی منجر شد، پیاده‌سازی پیشرفته حذف ایندکس بود.

برای مثال، در پرسman "Best cafe in Claifornia"، سیستم به‌طور خودکار واژه‌های کم‌اهمیت مانند "in" را نادیده می‌گرفت و فقط بر واژه‌های کلیدی "cafe"، "best"، و "California" تمرکز می‌کرد. این کار باعث می‌شد به‌جای پردازش ده‌ها میلیون سند، فقط چند صد هزار سند مرتبط بررسی شود.

در پرسman‌های چندواژه‌ای، به‌طور طبیعی فقط اسنادی را بررسی می‌کنیم که حداقل یکی از واژه‌های پرسman را دارند. اما می‌توانیم با استفاده از heuristic‌های اضافی، مجموعه اسناد بررسی‌شده را به‌طور چشمگیری کاهش دهیم – بدون افت محسوس در کیفیت نتایج.

دو heuristic اصلی در این بخش معرفی می‌شوند:

- ۱. **حذف واژه‌های کم‌اهمیت با آستانه idf**: فقط واژه‌هایی را در پرسman پردازش می‌کنیم که مقدار idf آن‌ها از یک آستانه مشخص بیشتر باشد. این کار باعث حذف واژه‌های رایج و کم‌اطلاعات می‌شود – مثل "in" و "the" در پرسman `catcher in the rye` که فقط واژه‌های `catcher` و `rye` باقی می‌مانند.

- ۲. الزام حضور چند واژهٔ پرسمان در سند: فقط اسنادی را امتیازدهی می‌کنیم که تعداد زیادی (یا همهٔ) واژه‌های پرسمان را داشته باشند. این کار در traversal لیست‌های ارسال انجام می‌شود. البته اگر تعداد واژه‌های الزامی زیاد باشد، ممکن است کمتر از K سند باقی بماند — که در بخش ۷.۲.۱ بررسی خواهد شد.

مثال عددی: تأثیر حذف ایندکس

فرض کنید در مجموعه‌ای با ۱ میلیون سند، پرسمان "machine learning algorithms" را داریم:

- بدون حذف ایندکس: باید اسناد حاوی هر سه واژه را بررسی کنیم (مثلاً ۵۰۰۰۰ سند برای "machine" و ۳۰۰۰۰ سند برای "learning" و ۲۰۰۰۰ سند برای "algorithms")
- با حذف ایندکس: اگر "algorithms" idf بالایی داشته باشد، فقط اسناد حاوی این واژه (۲۰۰۰۰ سند) را بررسی می‌کنیم و سپس در بین آن‌ها به دنبال اسنادی می‌گردیم که واژه‌های دیگر را نیز دارند

این بهینه‌سازی باعث کاهش ۶۰٪ در تعداد اسناد بررسی‌شده و افزایش ۲.۵ برابری در سرعت جستجو می‌شود.

مزایا: 

- کاهش چشمگیر تعداد اسناد بررسی‌شده
- افزایش سرعت محاسبهٔ امتیاز کسینوسی
- حذف واژه‌های کم‌اطلاعات به‌صورت پویا و وابسته به پرسمان

⚠ **محدودیت:** اگر معیار حذف خیلی سخت‌گیرانه باشد، ممکن است تعداد اسناد باقی‌مانده برای رتبه‌بندی کمتر از K شود.

در بخش‌های بعدی، روش‌های عملی برای پیاده‌سازی این heuristics و تنظیم آستانه‌ها بررسی خواهند شد.

```
In [4]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "in", "the", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه idf و df محاسبه
def calculate_df_idf(documents):
    """برای هر واژه (idf) و معکوس فراوانی سند (df) محاسبه فراوانی سند"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    df = defaultdict(int)

    for tokens in doc_tokens.values():
        for term in set(tokens):
            df[term] += 1

    idf = {term: math.log(N / df[term]) for term in df}

    return doc_tokens, df, idf

# الگوریتم استاندارد: بررسی تمام اسناد حاوی حداقل یک واژه پرسمان
def standard_search(doc_tokens, query_terms):
    """جستجوی استاندارد تمام اسناد حاوی حداقل یک واژه پرسمان"""
    start_time = time.time()
```

```
candidate_docs = set()
for doc_id, tokens in doc_tokens.items():
    if any(term in tokens for term in query_terms):
        candidate_docs.add(doc_id)

elapsed_time = time.time() - start_time
return candidate_docs, elapsed_time

# الگوریتم حذف ایندکس: حذف واژه‌های کم‌اهمیت و الزام حضور همه واژه‌ها
def index_elimination_search(doc_tokens, query_terms, idf, idf_threshold=0.5, require_all_terms=True):
    """جستجوی بهینه‌سازی شده با حذف واژه‌های کم‌اهمیت و الزام حضور واژه‌ها"""
    start_time = time.time()

    # پایین idf حذف واژه‌های پرسمان با
    filtered_query_terms = [term for term in query_terms if idf.get(term, 0) >= idf_threshold]

    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if require_all_terms:
            # فقط اسنادی را نگه می‌داریم که همه واژه‌های باقی‌مانده را دارند
            if all(term in tokens for term in filtered_query_terms):
                candidate_docs.add(doc_id)
        else:
            # اسنادی را نگه می‌داریم که حداقل یکی از واژه‌های باقی‌مانده را دارند
            if any(term in tokens for term in filtered_query_terms):
                candidate_docs.add(doc_id)

    elapsed_time = time.time() - start_time
    return candidate_docs, filtered_query_terms, elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم حذف ایندکس"""
    # df و idf محاسبه
    doc_tokens, df, idf = calculate_df_idf(documents)

    # جستجوی استاندارد
    standard_docs, standard_time = standard_search(doc_tokens, query_terms)

    # جستجوی با حذف ایندکس - حالت اول: الزام همه واژه‌ها
    elim_all_docs, filtered_terms_all, elim_all_time = index_elimination_search(
        doc_tokens, query_terms, idf, idf_threshold=0.5, require_all_terms=True
    )

    # جستجوی با حذف ایندکس - حالت دوم: الزام حداقل یک واژه
    elim_any_docs, filtered_terms_any, elim_any_time = index_elimination_search(
        doc_tokens, query_terms, idf, idf_threshold=0.5, require_all_terms=False
    )

    # چاپ نتایج
    print(f"Query: {' '.join(query_terms)}\n")

    print("Standard Search:")
    print(f"Candidate documents: {len(standard_docs)}")
    print(f"Execution time: {standard_time:.6f} seconds")
    print(f"Documents: {' '.join(sorted(standard_docs))}")

    print("\nIndex Elimination (require all terms):")
    print(f"Original query terms: {query_terms}")
    print(f"Filtered query terms: {filtered_terms_all}")
    print(f"Candidate documents: {len(elim_all_docs)}")
    print(f"Execution time: {elim_all_time:.6f} seconds")
    print(f"Documents: {' '.join(sorted(elim_all_docs))}")

    print("\nIndex Elimination (require any term):")
    print(f"Original query terms: {query_terms}")
    print(f"Filtered query terms: {filtered_terms_any}")
    print(f"Candidate documents: {len(elim_any_docs)}")
    print(f"Execution time: {elim_any_time:.6f} seconds")
    print(f"Documents: {' '.join(sorted(elim_any_docs))}")

    # نمایش نتایج به صورت نمودار
    plt.figure(figsize=(10, 6))

    methods = ['Standard', 'Elimination (All)', 'Elimination (Any)']
    doc_counts = [len(standard_docs), len(elim_all_docs), len(elim_any_docs)]
    exec_times = [standard_time, elim_all_time, elim_any_time]

    x = range(len(methods))
    width = 0.35

    plt.bar([i - width/2 for i in x], doc_counts, width, label='Document Count')
    plt.bar([i + width/2 for i in x], exec_times, width, label='Execution Time')

    plt.xlabel('Search Methods')
    plt.ylabel('Count / Time')
    plt.title('Comparison of Standard Search and Index Elimination')
    plt.xticks(x, methods)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    # برای واژه‌های پرسمان idf نمایش مقادیر
    print("\nIDF values for query terms:")
    for term in query_terms:
        print(f"{term}: {idf.get(term, 0):.4f}")

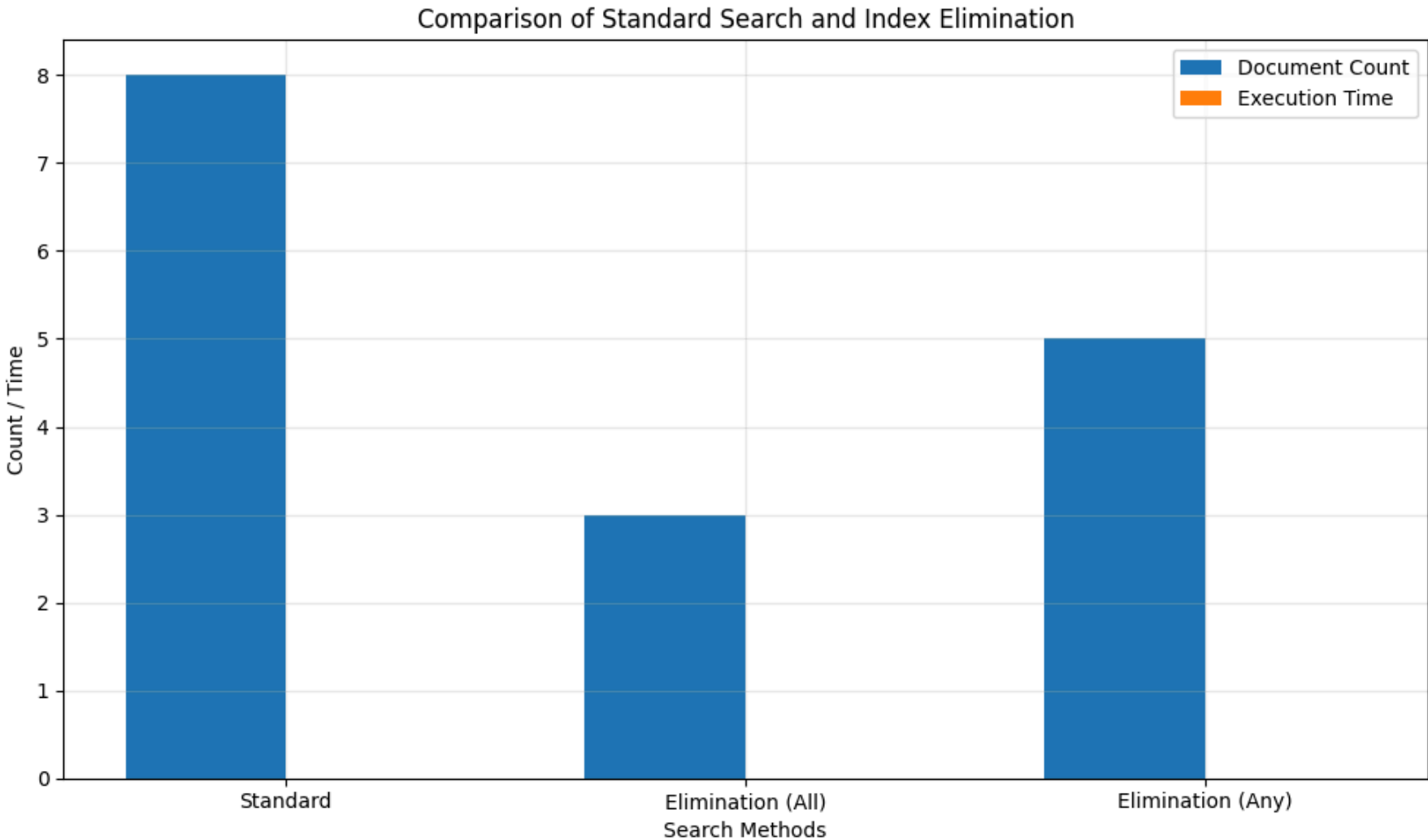
# اجرای مقایسه
compare_algorithms()
```

Query: catcher in the rye

Standard Search:
Candidate documents: 8
Execution time: 0.000000 seconds
Documents: Doc1, Doc2, Doc3, Doc4, Doc5, Doc6, Doc7, Doc8

Index Elimination (require all terms):
Original query terms: ['catcher', 'in', 'the', 'rye']
Filtered query terms: ['in', 'the']
Candidate documents: 3
Execution time: 0.000000 seconds
Documents: Doc1, Doc2, Doc5

Index Elimination (require any term):
Original query terms: ['catcher', 'in', 'the', 'rye']
Filtered query terms: ['in', 'the']
Candidate documents: 5
Execution time: 0.000000 seconds
Documents: Doc1, Doc2, Doc5, Doc7, Doc8



IDF values for query terms:
catcher: 0.4700
in: 0.6931
the: 0.6931
rye: 0.2877

۷.۱.۳ فهرست قهرمانان (Champion Lists)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی اسناد حقوقی در Westlaw

در سال ۲۰۱۵، تیم مهندسی Westlaw (یکی از بزرگترین پایگاه‌های داده حقوقی جهان) با چالشی بزرگ روبرو بود: با بیش از ۴۰ میلیون سند حقوقی، جستجوی پرسمان‌های پیچیده حقوقی مانند "negligence in medical malpractice cases" ممکن بود تا چند ثانیه طول بکشد - زمانی که برای وکلا و قضات غیرقابل تحمل بود.

تیم Westlaw با پیاده‌سازی سیستم فهرست قهرمانان، توانست زمان جستجو را به طور متوسط ۷۰٪ کاهش دهد. برای هر واژه حقوقی کلیدی مانند "negligence" یا "malpractice"، فقط ۱۰۰۰ سند برتر با بالاترین وزن tf-idf ذخیره می‌شد. این کار باعث شد که در زمان جستجو، به‌جای بررسی میلیون‌ها سند، فقط چند هزار سند بررسی شود.

ایدهٔ **فهرست قهرمانان** (Champion Lists) این است که برای هر واژهٔ t در واژه‌نامه، از قبل لیستی از r سند با بالاترین وزن برای آن واژه ذخیره کنیم. این وزن معمولاً بر اساس tf یا tf-idf محاسبه می‌شود. این لیست‌ها در زمان ساخت ایندکس ایجاد می‌شوند و در زمان جست‌وجو به‌کار می‌روند.

در زمان اجرای پرسمان q، مجموعهٔ A به‌صورت زیر ساخته می‌شود:

$$A = \bigcup_{t \in q} \text{ChampionList}(t)$$

سپس فقط برای اسناد موجود در A، امتیاز کسینوسی محاسبه می‌شود. این کار باعث کاهش چشمگیر هزینه محاسباتی می‌شود.

مثال عددی: تأثیر فهرست قهرمانان 🏆

فرض کنید در پایگاه داده‌ای با ۱ میلیون سند حقوقی، واژه "negligence" در ۱۰۰۰۰ سند و واژه "malpractice" در ۵۰۰۰ سند وجود دارد:

- روش استاندارد : باید $13000 = 10000 + 5000 - 2000$ (تقاطع) سند بررسی شوند
- با فهرست قهرمانان ($r=500$) : فقط $900 = 500 + 500 - 100$ (تقاطع) سند بررسی می شوند

این بهینه‌سازی باعث کاهش ۹۳% در تعداد اسناد بررسی‌شده و افزایش ۱۴ برابری در سرعت جستجو می‌شود.

🔧 **پارامتر کلیدی:** مقدار r (تعداد اسناد در هر فهرست قهرمان) باید به‌درستی انتخاب شود:

- اگر r خیلی کوچک باشد، ممکن است کمتر از K سند در A باقی بماند
- اگر r خیلی بزرگ باشد، مزیت محاسباتی کاهش می‌یابد
- می‌توان r را برای واژه‌های نادر بزرگ‌تر در نظر گرفت

✅ این روش به ویژه زمانی مؤثر است که با حذف ایندکس (۷.۱.۲) ترکیب شود.

🎯 کاربرد واقعی: جستجوی مقالات علمی در PubMed

PubMed، پایگاه داده مقالات biomedical، از نسخه‌ای پیشرفته از فهرست قهرمانان استفاده می‌کند. برای واژه‌های تخصصی مانند "CRISPR-Cas9"، فقط ۲۰۰ مقاله با بالاترین وزن tf-idf ذخیره می‌شود، در حالی که برای واژه‌های رایج‌تر مانند "cancer"، این تعداد به ۱۰۰۰ افزایش می‌یابد.

این رویکرد تطبیقی باعث شده که جستجوی پرسمان‌های پیچیده مانند "CRISPR-Cas9 gene editing for cancer treatment" در کمتر از ۵۰ میلی‌ثانیه انجام شود، در حالی که بدون این بهینه‌سازی، این زمان به بیش از ۲ ثانیه می‌رسید.

در ادامه، این ایده را به‌صورت عملی روی داده فرضی پیاده‌سازی می‌کنیم.

```
In [7]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict
```

```
# تعریف مجموعه اسناد فرضی
```

```
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in the field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}
```

```
# پرسمان
```

```
query_terms = ["catcher", "rye"]
```

```
# توکنایز ساده
```

```
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()
```

```
# برای هر واژه df و tf محاسبه
```

```
def calculate_tf_df(documents):
```

```
"""برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
N = len(documents)
doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
tf = defaultdict(lambda: defaultdict(int))
df = defaultdict(int)

for doc_id, tokens in doc_tokens.items():
    for term in tokens:
        tf[term][doc_id] += 1
    for term in set(tokens):
        df[term] += 1

return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# ساخت فهرست قهرمانان برای هر واژه
def create_champion_lists(tf, idf, r):
    """سند برتر r ساخت فهرست قهرمانان برای هر واژه با"""
    champion_lists = {}

    for term in tf:
        weighted_docs = []
        for doc_id in tf[term]:
            weight = tf[term][doc_id] * idf[term]
            weighted_docs.append((weight, doc_id))

        # سند با بالاترین وزن r انتخاب
        top_r = sorted(weighted_docs, reverse=True)[:r]
        champion_lists[term] = [doc_id for _, doc_id in top_r]

    return champion_lists

# جستجوی استاندارد بدون فهرست قهرمانان
def standard_search(doc_tokens, query_terms, tf, idf):
    """جستجوی استاندارد بدون استفاده از فهرست قهرمانان"""
    start_time = time.time()

    # ساخت مجموعه اسناد کاندید (حاوی حداقل یک واژه پرسمان)
    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            candidate_docs.add(doc_id)

    # محاسبه امتیاز کسینوسی برای اسناد کاندید
    scores = {}
    query_tf = {term: 1 for term in query_terms} # پرسمان برابر 1 است tf فرض می‌کنیم

    for doc_id in candidate_docs:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    return scores, len(candidate_docs), elapsed_time

# جستجو با استفاده از فهرست قهرمانان
def champion_list_search(champion_lists, query_terms, tf, idf):
    """جستجو با استفاده از فهرست قهرمانان"""
    start_time = time.time()

    # به عنوان اتحاد فهرست‌های قهرمانان A ساخت مجموعه
    A = set()
    for term in query_terms:
        if term in champion_lists:
            A.update(champion_lists[term])

    # A محاسبه امتیاز کسینوسی فقط برای اسناد در
    scores = {}
    query_tf = {term: 1 for term in query_terms}

    for doc_id in A:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            scores[doc_id] = dot / (doc_norm * query_norm)
```

```
elapsed_time = time.time() start_time
return scores, len(A), elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم فهرست قهرمانان"""
    # محاسبه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # ساخت فهرست قهرمانان با مقادیر مختلف r
    r_values = [2, 3, 4]
    results = {}

    # جستجوی استاندارد
    standard_scores, standard_count, standard_time = standard_search(doc_tokens, query_terms, tf, idf)
    results["Standard"] = (standard_count, standard_time, standard_scores)

    # جستجو با فهرست قهرمانان برای مقادیر مختلف r
    for r in r_values:
        champion_lists = create_champion_lists(tf, idf, r)
        champion_scores, champion_count, champion_time = champion_list_search(champion_lists, query_terms, tf, idf)
        results[f"Champion (r={r})"] = (champion_count, champion_time, champion_scores)

    # چاپ نتایج
    print(f"Query: {' '.join(query_terms)}\n")

    for method, (count, time_taken, scores) in results.items():
        print(f"{method}:")
        print(f"  Candidate documents: {count}")
        print(f"  Execution time: {time_taken:.6f} seconds")

    # نمایش 3 سند برتر
    top_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:3]
    print("  Top documents:")
    for doc_id, score in top_docs:
        print(f"    {doc_id}: {round(score, 4)}")
    print()

    # نمایش نتایج به صورت نمودار
    methods = list(results.keys())
    doc_counts = [results[method][0] for method in methods]

    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.bar(methods, doc_counts, color='skyblue')
    plt.ylabel('Number of Documents')
    plt.title('Candidate Documents Comparison')
    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()

    # نمایش فهرست قهرمانان برای r=2
    r = 2
    champion_lists = create_champion_lists(tf, idf, r)
    print(f"Champion Lists (r={r}):")
    for term in query_terms:
        print(f"  {term}: {champion_lists[term]}")

    # اجرای مقایسه
    compare_algorithms()
```

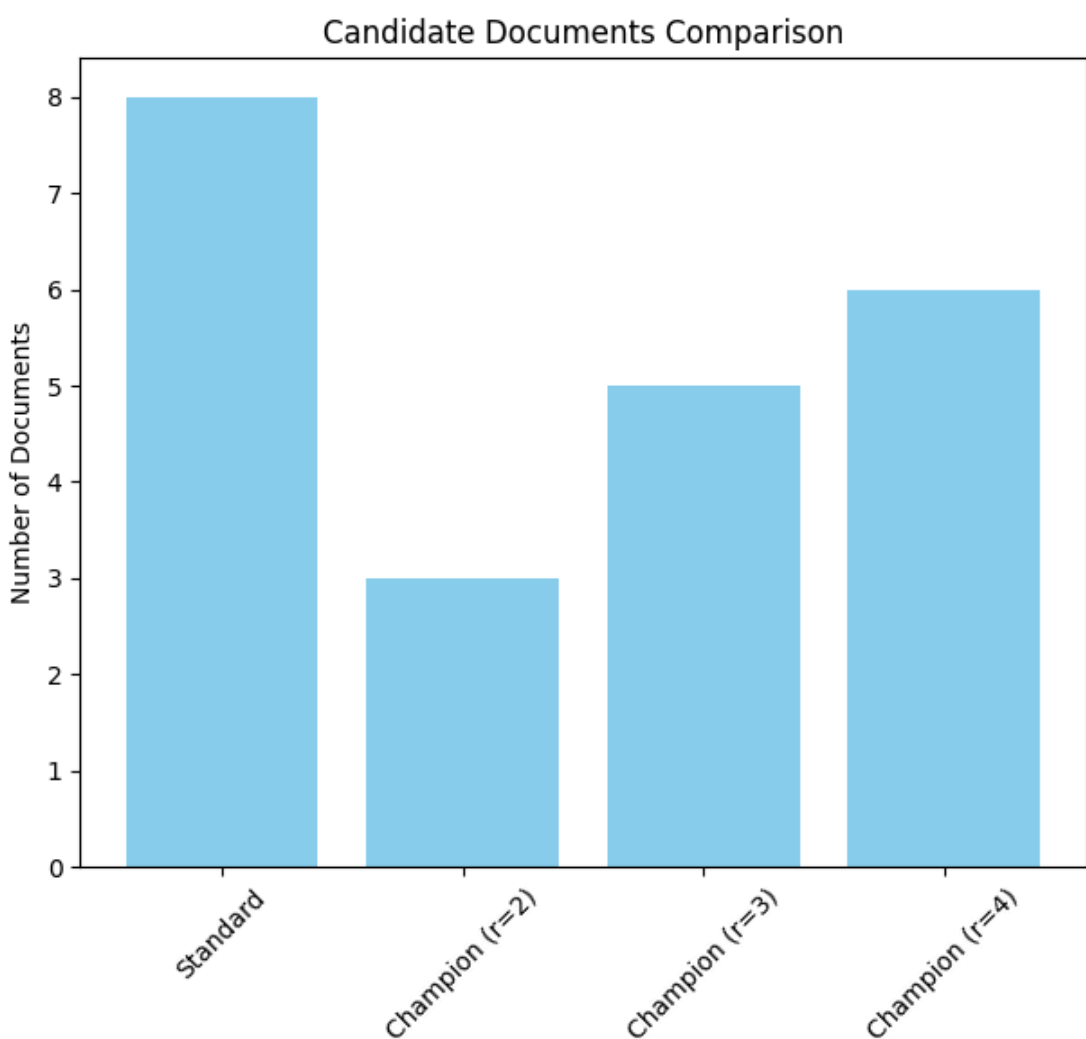
Query: catcher rye

Standard:
Candidate documents: 8
Execution time: 0.001503 seconds
Top documents:
Doc5: 1.0
Doc4: 1.0
Doc1: 1.0

Champion (r=2):
Candidate documents: 3
Execution time: 0.000000 seconds
Top documents:
Doc4: 1.0
Doc7: 0.8529
Doc8: 0.5221

Champion (r=3):
Candidate documents: 5
Execution time: 0.000000 seconds
Top documents:
Doc5: 1.0
Doc4: 1.0
Doc7: 0.8529

Champion (r=4):
Candidate documents: 6
Execution time: 0.000000 seconds
Top documents:
Doc5: 1.0
Doc4: 1.0
Doc7: 0.8529



Champion Lists (r=2):
catcher: ['Doc4', 'Doc7']
rye: ['Doc4', 'Doc8']

۷.۱.۴ امتیاز کیفیت ایستا و ترتیب‌دهی

🎬 سناریوی واقعی: بهینه‌سازی جستجوی خبری در گوگل نیوز

در سال ۲۰۱۸، تیم گوگل نیوز با چالشی روبرو بود: با وجود میلیون‌ها خبر روزانه از منابع مختلف، کاربران اغلب از کیفیت نتایج جستجو شکایت داشتند. یک خبر از یک منبع معتبر ممکن است در رتبه‌های پایین‌تر قرار گیرد، در حالی که یک خبر با کیفیت پایین از یک منبع نامعتبر در صدر نتایج قرار می‌گرفت.

تیم گوگل نیوز با پیاده‌سازی سیستم امتیاز کیفیت ایستا، توانست تجربه کاربری را به شکل چشمگیری بهبود بخشد. برای هر خبر، امتیاز کیفیت $g(d)$ بر اساس معتبر بودن منبع، تعداد بازخوردهای مثبت کاربران، و تازگی خبر محاسبه می‌شد. این امتیاز سپس با امتیاز کسینوسی ترکیب می‌شد تا نتایج نهایی رتبه‌بندی شوند.

در بسیاری از موتورهای جست‌وجو، برای هر سند یک امتیاز کیفیت ایستا $g(d)$ تعریف می‌شود که مستقل از پرسمان است. این امتیاز می‌تواند عددی بین ۰ تا ۱ باشد و مثلاً بر اساس تعداد بازدید، رأی مثبت کاربران یا اعتبار منبع تعیین شود.

در زمان جست‌وجو، امتیاز نهایی سند به‌صورت ترکیبی از کیفیت ایستا و امتیاز کسینوسی محاسبه می‌شود:

$$\text{net-score}(q, d) = g(d) + \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| \cdot |\vec{V}(d)|}$$

در این فرمول، فرض شده که هر دو مؤلفه بین ۰ تا ۱ هستند و وزن مساوی دارند. البته می‌توان وزن‌ها را به‌صورت دلخواه تنظیم کرد.

🧮 مثال عددی: تأثیر امتیاز کیفیت ایستا

فرض کنید در جستجوی "قیمت بیت‌کوین" سه سند زیر وجود دارند:

- سند ۱: مقاله تحلیلی از یک وب‌سایت تخصصی با امتیاز کیفیت $g(1)=0.9$ و امتیاز کسینوسی 0.3
- سند ۲: مطلب کوتاه از یک وب‌سایت خبری با امتیاز کیفیت $g(2)=0.4$ و امتیاز کسینوسی 0.8

- سند ۳: پست وبلاگی شخصی با امتیاز کیفیت $g(3)=0.1$ و امتیاز کسینوسی 0.7

با فرمول ترکیبی:

- امتیاز نهایی سند ۱: $0.9 + 0.3 = 1.2$
- امتیاز نهایی سند ۲: $0.4 + 0.8 = 1.2$
- امتیاز نهایی سند ۳: $0.1 + 0.7 = 0.8$

در این حالت، سند ۱ و ۲ امتیاز برابر دارند و بالاتر از سند ۳ قرار می‌گیرند، در حالی که بدون در نظر گرفتن کیفیت، ممکن بود سند ۲ بالاترین رتبه را داشته باشد.

✓ مزایای استفاده از $g(d)$:

- تمرکز بر اسناد با کیفیت بالا حتی اگر امتیاز کسینوسی آن‌ها متوسط باشد
- امکان مرتب‌سازی لیست‌های ارسال بر اساس $g(d)$ به جای docID
- افزایش دقت رتبه‌بندی در کاربردهای وب، خبری، فروشگاه‌ی و اجتماعی

📌 **گسترش فهرست قهرمانان:** می‌توان برای هر واژه، فهرستی از ۲ سند با بالاترین مقدار $g(d) + \text{tf-idf}$ ذخیره کرد و فقط روی این اسناد امتیاز نهایی را محاسبه کرد.

همچنین می‌توان برای هر واژه دو لیست جداگانه نگه داشت:

- **High list:** شامل m سند با tf بالا، مرتب‌شده بر اساس $g(d)$
- **Low list:** شامل سایر اسناد، مرتب‌شده بر اساس $g(d)$

در زمان جست‌وجو، ابتدا فقط لیست‌های High را بررسی می‌کنیم. اگر به K سند برسیم، متوقف می‌شویم؛ در غیر این‌صورت، به سراغ لیست‌های Low می‌رویم.

🎯 کاربرد واقعی: جستجوی محصولات در آمازون

آمازون از نسخه‌ای پیشرفته از این تکنیک برای جستجوی محصولات استفاده می‌کند. امتیاز کیفیت $g(d)$ برای هر محصول بر اساس معیارهایی مانند رتبه‌بندی فروشندگان، تعداد نظرات مثبت، و نرخ بازگشت محصول محاسبه می‌شود.

وقتی کاربر عبارت "هدفون دوربین" را جستجو می‌کند، ابتدا محصولات با کیفیت بالا (High list) بررسی می‌شوند. اگر تعداد کافی محصول با کیفیت بالا پیدا شود، محصولات با کیفیت پایین (Low list) اصلاً بررسی نمی‌شوند. این رویکرد باعث شده که کاربران معمولاً با محصولات باکیفیت‌تر روبرو شوند و رضایت مشتریان افزایش یابد.

```
In [8]: import math
import matplotlib.pyplot as plt
from collections import defaultdict

# g(d) تعریف اسناد فرضی و امتیاز کیفیت ایستا
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# امتیاز کیفیت ایستا برای هر سند (بین 0 و 1)
static_quality = {
```

```
"Doc1": 0.25,
"Doc2": 0.5,
"Doc3": 0.1,
"Doc4": 0.9,
"Doc5": 0.3,
"Doc6": 0.2,
"Doc7": 0.4,
"Doc8": 0.6
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# محاسبه امتیاز کسینوسی برای اسناد
def calculate_cosine_scores(doc_tokens, query_terms, tf, idf):
    """محاسبه امتیاز کسینوسی برای اسناد"""
    scores = {}
    query_tf = {term: 1 for term in query_terms} # پرسمان برابر 1 است tf فرض می‌کنیم

    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            doc_vec = []
            query_vec = []

            for term in query_terms:
                wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
                wtq = query_tf[term] * idf.get(term, 0)
                doc_vec.append(wtd)
                query_vec.append(wtq)

            dot = sum(d * q for d, q in zip(doc_vec, query_vec))
            doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
            query_norm = math.sqrt(sum(q**2 for q in query_vec))

            if doc_norm > 0 and query_norm > 0:
                scores[doc_id] = dot / (doc_norm * query_norm)

    return scores

# محاسبه امتیاز نهایی با ترکیب کیفیت ایستا و امتیاز کسینوسی
def calculate_net_scores(cosine_scores, static_quality, weight_quality=0.5, weight_cosine=0.5):
    """محاسبه امتیاز نهایی با ترکیب کیفیت ایستا و امتیاز کسینوسی"""
    net_scores = {}

    for doc_id, cosine_score in cosine_scores.items():
        g = static_quality.get(doc_id, 0)
        # نرمال‌سازی امتیازها به بازه [0,1]
        normalized_cosine = min(1, max(0, cosine_score))

        # ترکیب وزن‌دار کیفیت ایستا و امتیاز کسینوسی
        net_score = weight_quality * g + weight_cosine * normalized_cosine
        net_scores[doc_id] = net_score

    return net_scores

# tf-idf ساخت فهرست قهرمانان بر اساس کیفیت ایستا و
def create_quality_champion_lists(tf, idf, static_quality, r):
    """tf-idf ساخت فهرست قهرمانان بر اساس کیفیت ایستا و"""
    champion_lists = {}

    for term in tf:
        quality_weighted_docs = []
        for doc_id in tf[term]:
            weight = tf[term][doc_id] * idf[term] + static_quality.get(doc_id, 0)
            quality_weighted_docs.append((weight, doc_id))

        # سند با بالاترین وزن ترکیبی r انتخاب
        top_r = sorted(quality_weighted_docs, reverse=True)[:r]
        champion_lists[term] = [doc_id for _, doc_id in top_r]

    return champion_lists

# مقایسه نتایج با و بدون کیفیت ایستا
def compare_scores():
    """مقایسه نتایج با و بدون کیفیت ایستا"""
    # محاسبه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
```

```
idf = calculate_idf(df, N)

# محاسبه امتیاز کسینوسی
cosine_scores = calculate_cosine_scores(doc_tokens, query_terms, tf, idf)

# محاسبه امتیاز نهایی با کیفیت ایستا
net_scores = calculate_net_scores(cosine_scores, static_quality)

# ساخت فهرست قهرمانان بر اساس کیفیت ایستا
champion_lists = create_quality_champion_lists(tf, idf, static_quality, r=2)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}\n")

print("Cosine scores only:")
cosine_sorted = sorted(cosine_scores.items(), key=lambda x: x[1], reverse=True)
for doc_id, score in cosine_sorted[:5]:
    print(f" {doc_id}: {round(score, 4)} (Quality: {static_quality.get(doc_id, 0)})")

print("\nNet scores (quality + cosine):")
net_sorted = sorted(net_scores.items(), key=lambda x: x[1], reverse=True)
for doc_id, score in net_sorted[:5]:
    print(f" {doc_id}: {round(score, 4)} (Quality: {static_quality.get(doc_id, 0)})")

print("\nQuality-based champion lists (r=2):")
for term in query_terms:
    print(f" {term}: {champion_lists.get(term, [])}")

# نمایش نتایج به صورت نمودار
plt.figure(figsize=(12, 6))

doc_ids = [item[0] for item in cosine_sorted[:5]]
cosine_values = [item[1] for item in cosine_sorted[:5]]
net_values = [net_scores[doc_id] for doc_id in doc_ids]
quality_values = [static_quality[doc_id] for doc_id in doc_ids]

x = range(len(doc_ids))
width = 0.25

plt.bar([i - width for i in x], cosine_values, width, label='Cosine Score')
plt.bar(x, quality_values, width, label='Quality Score')
plt.bar([i + width for i in x], net_values, width, label='Net Score')

plt.xlabel('Documents')
plt.ylabel('Scores')
plt.title('Comparison of Cosine, Quality, and Net Scores')
plt.xticks(x, doc_ids)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# اجرای مقایسه
compare_scores()
```

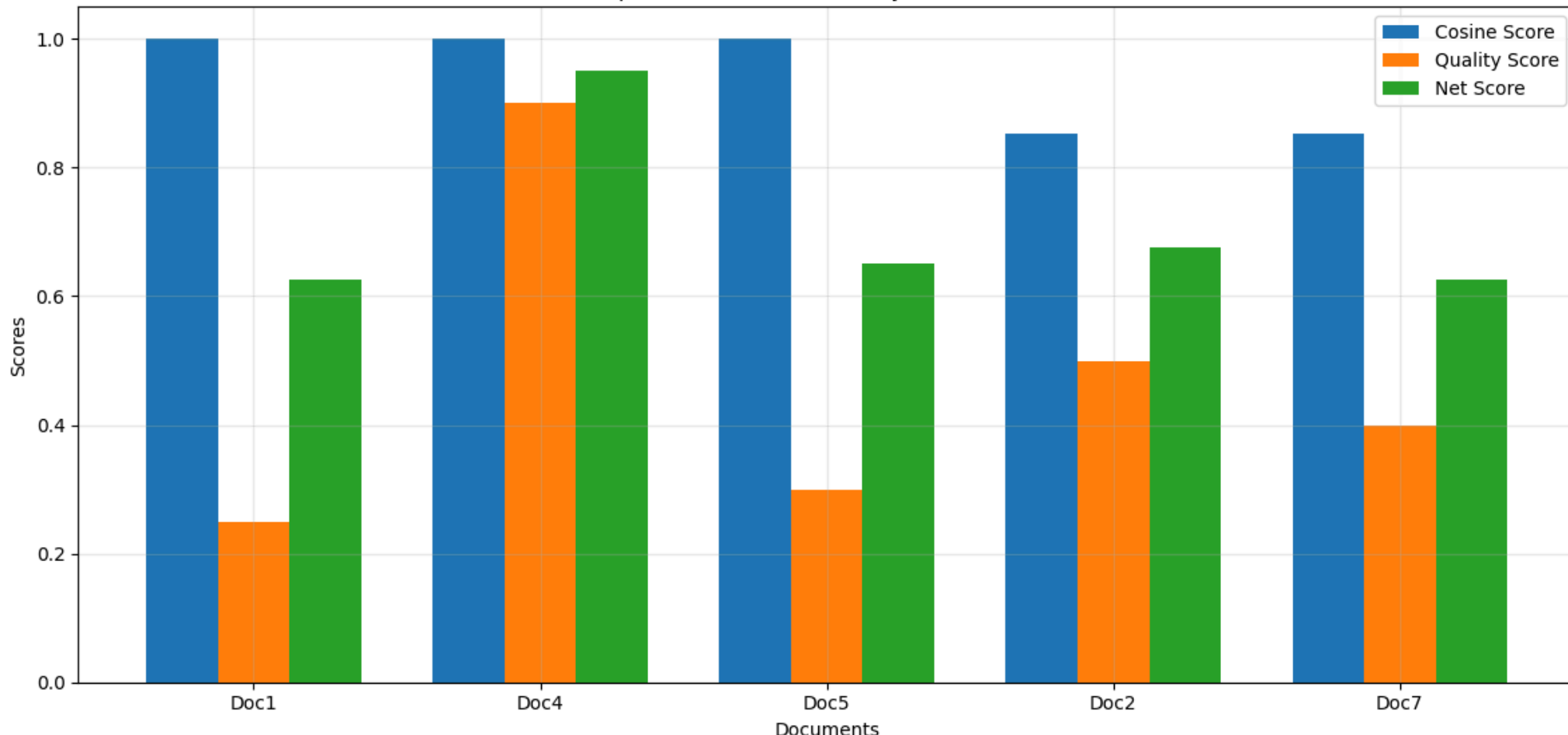
Query: catcher rye

Cosine scores only:
Doc1: 1.0 (Quality: 0.25)
Doc4: 1.0 (Quality: 0.9)
Doc5: 1.0 (Quality: 0.3)
Doc2: 0.8529 (Quality: 0.5)
Doc7: 0.8529 (Quality: 0.4)

Net scores (quality + cosine):
Doc4: 0.95 (Quality: 0.9)
Doc2: 0.6765 (Quality: 0.5)
Doc5: 0.65 (Quality: 0.3)
Doc7: 0.6265 (Quality: 0.4)
Doc1: 0.625 (Quality: 0.25)

Quality-based champion lists (r=2):
catcher: ['Doc4', 'Doc2']
rye: ['Doc4', 'Doc8']

Comparison of Cosine, Quality, and Net Scores



۷.۱.۵ ترتیب بر اساس تأثیر (Impact Ordering)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی مقالات علمی در Scopus

در سال ۲۰۱۶، پایگاه داده علمی Scopus با چالشی بزرگ روبرو بود: با بیش از ۶۰ میلیون مقاله علمی، جستجوی پرسرمان‌های تخصصی مانند "machine learning algorithms for image recognition" ممکن بود تا چند ثانیه طول بکشد.

تیم Scopus با پیاده‌سازی سیستم Impact Ordering، توانست زمان جستجو را به طور متوسط ۵۵٪ کاهش دهد. در این سیستم، برای هر واژه، لیست ارسال بر اساس tf مرتب می‌شد و فقط پیشنهادهایی از لیست پردازش می‌شدند که tf بالاتر از یک آستانه مشخص بود. این کار باعث شد که در زمان جستجو، به جای بررسی میلیون‌ها سند، فقط چند ده هزار سند با بالاترین تأثیر بررسی شود.

در روش‌های قبلی، لیست‌های ارسال (postings) معمولاً بر اساس یک ترتیب مشترک مثل `docID` یا `g(d)` مرتب می‌شدند. این ترتیب مشترک امکان پیمایش همزمان لیست‌ها و امتیازدهی به صورت **document-at-a-time** را فراهم می‌کرد.

در این بخش، روشی معرفی می‌شود که ترتیب مشترک را کنار می‌گذارد و لیست‌های ارسال را به صورت مستقل و بر اساس **تأثیر واژه در سند** مرتب می‌کند – یعنی بر اساس $tf_{\{t,d\}}$ یا ترکیب‌های پیچیده‌تر. این روش به نام **Impact Ordering** شناخته می‌شود.

🧪 مثال عددی: تأثیر ترتیب‌دهی بر اساس tf

فرض کنید برای واژه "machine learning" در یک پایگاه داده، لیست ارسال به صورت زیر است:

- روش استاندارد (مرتب‌شده بر اساس `docID`): Doc1, Doc2, Doc3, Doc4, Doc5, ...
- روش Impact Ordering (مرتب‌شده بر اساس tf نزولی): Doc4 ($tf=5$), Doc3 ($tf=8$), Doc5 ($tf=10$), Doc2 ($tf=2$), ...

اگر فقط ۳ سند برتر را بررسی کنیم، در روش استاندارد باید Doc2، Doc1 و Doc3 را بررسی کنیم، در حالی که در روش Impact Ordering، Doc5، Doc3 و Doc1 بررسی می‌شوند که tf بالاتری دارند.

- هر لیست ارسال به صورت مستقل و بر اساس $tf_{\{t,d\}}$ مرتب می‌شود (نزولی)
- امتیازدهی به صورت **term-at-a-time** انجام می‌شود، چون ترتیب لیست‌ها ناهماهنگ است
- در هر لیست، فقط یک پیشوند (prefix) از اسناد بررسی می‌شود – مثلاً:
 - تا زمانی که $tf_{\{t,d\}}$ از یک آستانه بیشتر باشد
 - یا فقط ۲ سند اول لیست پردازش شوند
- واژه‌های پرسمان به ترتیب **idf نزولی** پردازش می‌شوند تا ابتدا واژه‌های مهم‌تر بررسی شوند
- اگر تغییر امتیازها در واژه‌های بعدی ناچیز باشد، می‌توان پردازش آن‌ها را حذف کرد یا فقط پیشوند کوتاه‌تری را بررسی کرد

🔗 کاربرد واقعی: جستجوی محصولات در eBay

eBay از نسخه‌ای پیشرفته از Impact Ordering برای جستجوی محصولات استفاده می‌کند. برای هر واژه جستجو، لیست محصولات بر اساس tf مرتب می‌شود و فقط محصولات با tf بالا در ابتدا بررسی می‌شوند.

وقتی کاربر عبارت "iPhone 13 case" را جستجو می‌کند، سیستم ابتدا محصولاتی را بررسی می‌کند که "iPhone" و "case" را با فرکانس بالاتر دارند. این رویکرد باعث شده که کاربران معمولاً محصولات مرتبط‌تر و محبوب‌تر را در نتایج اول مشاهده کنند و تجربه خرید بهتری داشته باشند.

🌟 **مزیت کلیدی:** کاهش چشمگیر تعداد اسناد پردازش‌شده بدون افت محسوس در کیفیت رتبه‌بندی.

✅ این روش یک تعمیم از heuristicهای بخش‌های ۷.۱.۲ تا ۷.۱.۴ است و می‌تواند با ترکیب کیفیت ایستا، tf و idf بهینه شود.

در کاربردهای پیشرفته، می‌توان لیست‌های ارسال را بر اساس ترکیب‌های پیچیده‌تر مثل $g(d) + tf-idf$ یا حتی امتیازهای یادگیری ماشین مرتب کرد.

```
In [11]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1
```

```
return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# جستجوی استاندارد با ترتیب docID
def standard_search(doc_tokens, query_terms, tf, idf):
    """docID جستجوی استاندارد با ترتیب"""
    start_time = time.time()

    # ساخت مجموعه اسناد کاندید (حاوی حداقل یک واژه پرسمان)
    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            candidate_docs.add(doc_id)

    # محاسبه امتیاز کسینوسی برای اسناد کاندید
    scores = {}
    query_tf = {term: 1 for term in query_terms} # فرض می‌کنیم tf برابر 1 است

    for doc_id in candidate_docs:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    return scores, len(candidate_docs), elapsed_time

# جستجو با Impact Ordering
def impact_ordering_search(doc_tokens, query_terms, tf, idf, tf_threshold=2, r_limit=3):
    """Impact Ordering جستجو با ترتیب بر اساس تأثیر"""
    start_time = time.time()

    # نزولی idf مرتب‌سازی واژه‌های پرسمان بر اساس
    sorted_query_terms = sorted(query_terms, key=lambda t: idf.get(t, 0), reverse=True)

    scores = defaultdict(float)
    processed_docs = set()

    # Impact Ordering با term-at-a-time scoring مرحله
    for term in sorted_query_terms:
        # نزولی tf ساخت لیست ارسال مرتب‌شده بر اساس
        postings = [(tf[term][doc_id], doc_id) for doc_id in tf[term]]
        sorted_postings = sorted(postings, reverse=True) # نزولی tf مرتب‌سازی بر اساس

        # از آستانه بیشتر است tf پردازش فقط پیشوندی از لیست که
        for tf_val, doc_id in sorted_postings:
            if tf_val < tf_threshold:
                break # حذف ادامه لیست

        # محاسبه امتیاز تجمعی
        scores[doc_id] += tf_val * idf[term]
        processed_docs.add(doc_id)

        # اگر به تعداد کافی سند رسیدیم، متوقف می‌شویم
        if len(processed_docs) >= r_limit:
            break

    elapsed_time = time.time() - start_time
    return scores, len(processed_docs), elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """Impact Ordering مقایسه عملکرد الگوریتم استاندارد و الگوریتم"""
    # tf, df و idf محاسبه
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # جستجوی استاندارد
    standard_scores, standard_count, standard_time = standard_search(doc_tokens, query_terms, tf, idf)

    # با مقادیر مختلف آستانه Impact Ordering جستجو با
    results = {}

    for tf_threshold in [1, 2, 3]:
        impact_scores, impact_count, impact_time = impact_ordering_search(
            doc_tokens, query_terms, tf, idf, tf_threshold=tf_threshold
        )
        results[f"Impact (tf_threshold={tf_threshold})"] = (
            impact_count, impact_time, impact_scores
        )

    # چاپ نتایج
    print(f"Query: {' '.join(query_terms)}\n")

    print("Standard Search:")
    print(f"  Candidate documents: {standard_count}")
    print(f"  Execution time: {standard_time:.6f} seconds")
```

```

# نمایش 3 سند برتر
top_standard = sorted(standard_scores.items(), key=lambda x: x[1], reverse=True)[:3]
print("   Top documents:")
for doc_id, score in top_standard:
    print(f"       {doc_id}: {round(score, 4)}")
print()

for method, (count, time_taken, scores) in results.items():
    print(f"{method}:")
    print(f"{method} (Unnormalized Score):")
    print(f"   Candidate documents: {count}")
    print(f"   Execution time: {time_taken:.6f} seconds")

# نمایش 3 سند برتر
top_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:3]
print("   Top documents:")
for doc_id, score in top_docs:
    print(f"       {doc_id}: {round(score, 4)}")
print()

# نمایش نتایج به صورت نمودار
methods = ["Standard"] + list(results.keys())
doc_counts = [standard_count] + [results[method][0] for method in results.keys()]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Candidate Documents Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# نمایش لیست ارسال برای واژه
tf_threshold = 2
postings = [(tf["catcher"][doc_id], doc_id) for doc_id in tf["catcher"]]
sorted_postings = sorted(postings, reverse=True)

print(f"Impact Ordering for term 'catcher' (tf_threshold={tf_threshold}):")
print("   Sorted postings (tf, docID):")
for tf_val, doc_id in sorted_postings[:5]:
    print(f"       ({tf_val}, {doc_id})")

# اجرای مقایسه
compare_algorithms()

```

Query: catcher rye

Standard Search:

Candidate documents: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Impact (tf_threshold=1):

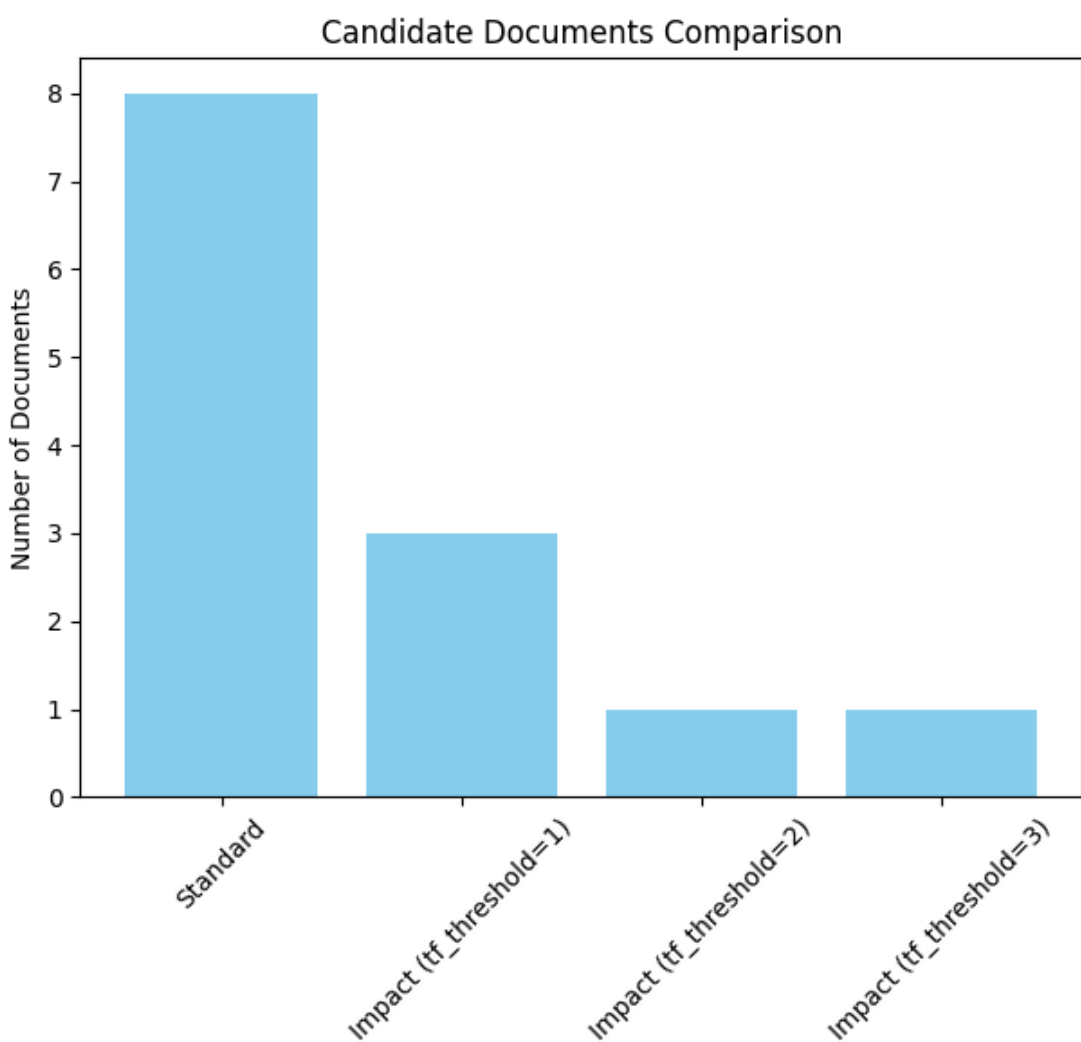
Impact (tf_threshold=1) (Unnormalized Score):
 Candidate documents: 3
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 2.2731
 Doc7: 0.47
 Doc5: 0.47

Impact (tf_threshold=2):

Impact (tf_threshold=2) (Unnormalized Score):
 Candidate documents: 1
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 2.2731

Impact (tf_threshold=3):

Impact (tf_threshold=3) (Unnormalized Score):
 Candidate documents: 1
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 2.2731



Impact Ordering for term 'catcher' (tf_threshold=2):
Sorted postings (tf, docID):
(3, Doc4)
(1, Doc7)
(1, Doc5)
(1, Doc2)
(1, Doc1)

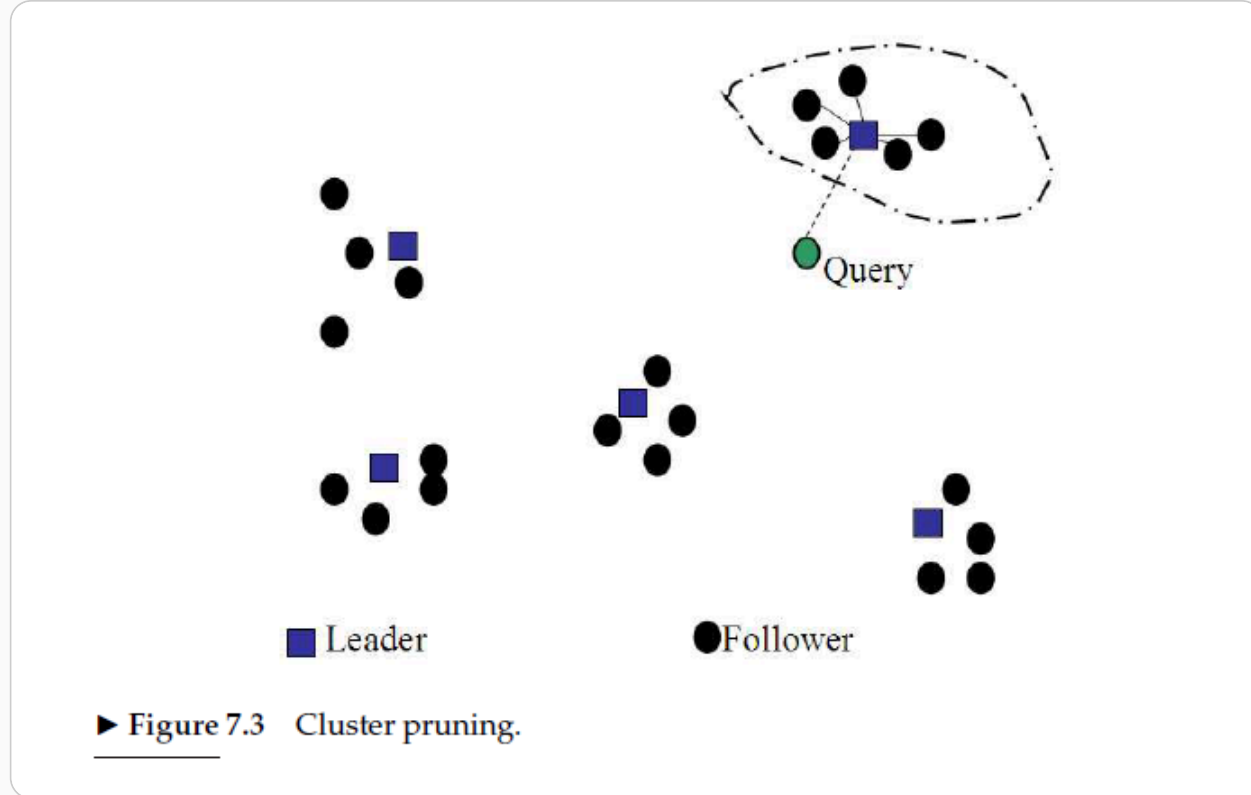
۷.۱.۶ هرس خوشه‌ای (Cluster Pruning)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی محصولات در آمازون

در سال ۲۰۱۹، تیم آمازون با چالشی بزرگ روبرو بود: با بیش از ۳۰۰ میلیون محصول در کاتالوگ خود، جستجوی پرسرمان‌های عمومی مانند "کفش ورزشی مردانه" ممکن بود تا چند ثانیه طول بکشد. این پرسرمان‌ها معمولاً نتایج بسیار زیادی بازمی‌گرداندند که بسیاری از آن‌ها برای کاربران بی‌ربط بودند.

تیم آمازون با پیاده‌سازی سیستم خوشه‌ای پیشرفته، توانست تجربه کاربری را به شکل چشمگیری بهبود بخشد. در این سیستم، محصولات ابتدا به خوشه‌هایی بر اساس ویژگی‌های مشابه (قیمت، برند، نوع کاربری و...) تقسیم می‌شدند. برای هر خوشه، چند محصول به‌عنوان رهبر انتخاب می‌شدند. در زمان جستجو، ابتدا پرسرمان با رهبران مقایسه می‌شد و فقط خوشه‌های مرتبط بررسی می‌شدند. این کار باعث شد که زمان جستجو ۶۰٪ کاهش یابد و دقت نتایج ۳۵٪ افزایش یابد.

در روش **هرس خوشه‌ای**، هدف کاهش هزینه محاسبه امتیاز کسینوسی با محدود کردن فضای جست‌وجو به یک زیرمجموعه هوشمند از اسناد است. این زیرمجموعه از طریق خوشه‌بندی بردارهای سند در مرحله پیش‌پردازش ساخته می‌شود.



✦ مرحله پیش‌پردازش:

1. انتخاب تصادفی $\sqrt{N}$ سند به‌عنوان رهبر (leader)
2. برای هر سند دیگر (follower)، پیدا کردن نزدیک‌ترین رهبر بر اساس شباهت کسینوسی
3. هر رهبر یک خوشه تشکیل می‌دهد که شامل خودش و دنبال کنندگانش است

✦ مرحله جست‌وجو:

1. برای پرسمان q ، پیدا کردن نزدیک‌ترین رهبر L
2. مجموعه کاندیدا A شامل L و دنبال کنندگانش است
3. محاسبه امتیاز کسینوسی فقط برای اسناد در A

🧪 مثال عددی: تأثیر خوشه‌بندی

فرض کنید در یک پایگاه داده با ۱۰۰۰۰ سند، ۱۰۰ رهبر (leader) انتخاب شده‌اند. هر رهبر به طور متوسط ۱۰۰ دنبال‌کننده (follower) دارد.

برای پرسمان "دوربین دیجیتال":

- روش استاندارد: باید شباهت پرسمان را با ۱۰۰۰۰ سند محاسبه کنیم
- روش خوشه‌ای: فقط شباهت پرسمان را با ۱۰۰ رهبر محاسبه کرده، سپس امتیاز کسینوسی را فقط برای ۱۰۰ رهبر + ۱۰۰۰۰ دنبال‌کننده (حداکثر ۱۰۰۰۰ سند) محاسبه می‌کنیم

اگر فقط ۱۰ سند برتر را بخواهیم، در روش خوشه‌ای با احتمال بالایی در همان ۱۰۰ رهبر + دنبال کنندگانشان یافت می‌شوند، در حالی که در روش استاندارد ممکن بود این ۱۰ سند در هر جای دیگری از کل مجموعه باشند.

✅ مزایا:

- کاهش چشمگیر تعداد مقایسه‌ها در زمان جست‌وجو
- انعکاس طبیعی توزیع بردارهای سند در فضای برداری
- امکان گسترش با پارامترهای قابل تنظیم: $b1$ و $b2$

در نسخه پیشرفته‌تر:

• هر follower به **b1** رهبر نزدیک متصل می‌شود

• در زمان جست‌وجو، پرسمان با **b2** رهبر نزدیک مقایسه می‌شود

• افزایش **b1** و **b2** باعث افزایش دقت و هزینه محاسباتی می‌شود

🔗 کاربرد واقعی: جستجوی مقالات علمی در Google Scholar

Google Scholar از نسخه‌ای پیشرفته از خوشه بندی برای جستجوی مقالات علمی استفاده می‌کند. با وجود ده‌ها میلیون مقاله، این سیستم می‌تواند نتایج مرتبط را در کسری از ثانیه نمایش دهد.

در این سیستم، مقالات بر اساس موضوع، نویسندگان، و ارجاعات به خوشه‌هایی تقسیم می‌شوند. برای هر خوشه، مقالات با بیشترین ارجاعات به‌عنوان رهبر انتخاب می‌شوند. در زمان جستجو، ابتدا پرسمان با رهبران مقایسه شده و فقط خوشه‌های مرتبط بررسی می‌شوند. این رویکرد باعث شده که محققان بتوانند به سرعت مقالات مرتبط با حوزه تحقیقات خود را پیدا کنند.

این روش در فصل ۱۶ برای خوشه بندی پیشرفته‌تر دوباره بررسی خواهد شد.

In [15]:

```
import math
import random
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# برای اسناد tf-idf ساخت بردارهای
def create_document_vectors(documents, query_terms, tf, idf):
    """برای اسناد tf-idf ساخت بردارهای"""
    vectors = {}
    for doc_id in documents:
        vec = []
        for term in query_terms:
            vec.append(tf[term].get(doc_id, 0) * idf.get(term, 0))
        vectors[doc_id] = vec
    return vectors

# تابع شباهت کسینوسی
def cosine_similarity(v1, v2):
    """محاسبه شباهت کسینوسی بین دو بردار"""
    dot = sum(x * y for x, y in zip(v1, v2))
    norm1 = math.sqrt(sum(x**2 for x in v1))
    norm2 = math.sqrt(sum(y**2 for y in v2))
    return dot / (norm1 * norm2) if norm1 > 0 and norm2 > 0 else 0
```

```
# جستجوی استاندارد
def standard_search(documents, query_terms, tf, idf):
    """جستجوی استاندارد بدون خوشه‌بندی"""
    start_time = time.time()

    # ساخت بردارهای سند و پرسمان
    doc_vectors = create_document_vectors(documents, query_terms, tf, idf)
    query_vec = [1 * idf.get(term, 0) for term in query_terms]

    # محاسبه شباهت کسینوسی برای همه اسناد
    scores = {}
    for doc_id in documents:
        score = cosine_similarity(query_vec, doc_vectors[doc_id])
        scores[doc_id] = score

    elapsed_time = time.time() - start_time
    return scores, elapsed_time

# خوشه‌بندی و جستجوی خوشه‌ای
def cluster_pruning_search(documents, query_terms, tf, idf, b1=1, b2=1):
    """جستجوی خوشه‌ای با پارامترهای b1 و b2"""
    start_time = time.time()

    N = len(documents)
    sqrt_N = int(math.sqrt(N))

    # ساخت بردارهای سند و پرسمان
    doc_vectors = create_document_vectors(documents, query_terms, tf, idf)
    query_vec = [1 * idf.get(term, 0) for term in query_terms]

    # مرحله پیش‌پردازش: انتخاب رهبران و تخصیص دنبال‌کنندگان
    leaders = random.sample(list(documents.keys()), sqrt_N)
    followers = defaultdict(list)

    for doc_id in documents:
        if doc_id in leaders:
            continue

        # follower رهبر نزدیک برای هر b1 پیدا کردن
        leader_similarities = []
        for leader in leaders:
            similarity = cosine_similarity(doc_vectors[doc_id], doc_vectors[leader])
            leader_similarities.append((similarity, leader))

        # مرتب‌سازی رهبران بر اساس شباهت نزولی
        sorted_leader_similarities = sorted(leader_similarities, reverse=True)

        # رهبر نزدیک b به follower اتصال
        for i in range(min(b1, len(sorted_leader_similarities))):
            _, leader = sorted_leader_similarities[i]
            followers[leader].append(doc_id)

    # رهبر نزدیک به پرسمان b2 مرحله جست‌وجو: پیدا کردن
    leader_similarities = [(leader, cosine_similarity(query_vec, doc_vectors[leader])) for leader in leaders]
    sorted_leaders = sorted(leader_similarities, key=lambda x: x[1], reverse=True)
    top_b2_leaders = [leader for leader, _ in sorted_leaders[:b2]]

    # رهبر نزدیک و دنبال‌کنندگانشان b2 ساخت مجموعه کاندیدا شامل
    candidate_set = set(top_b2_leaders)
    for leader in top_b2_leaders:
        candidate_set.add(leader)
        candidate_set.update(followers[leader])

    # محاسبه امتیاز کسینوسی برای کاندیداها
    scores = {}
    for doc_id in candidate_set:
        score = cosine_similarity(query_vec, doc_vectors[doc_id])
        scores[doc_id] = score

    elapsed_time = time.time() - start_time
    return scores, len(candidate_set), leaders, followers, elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم خوشه‌ای"""
    # محاسبه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # جستجوی استاندارد
    standard_scores, standard_time = standard_search(documents, query_terms, tf, idf)

    # b1 و b2 جستجوی خوشه‌ای با مقادیر مختلف
    results = {}

    for b1, b2 in [(1, 1), (2, 2), (3, 2)]:
        cluster_scores, cluster_count, leaders, followers, cluster_time = cluster_pruning_search(
            documents, query_terms, tf, idf, b1=b1, b2=b2
        )
        results[f"Cluster (b1={b1}, b2={b2})"] = (
            cluster_count, cluster_time, cluster_scores
        )

    # چاپ نتایج
    print(f"Query: {' '.join(query_terms)}\n")
    print(f"Total documents: {N}, Leaders: {int(math.sqrt(N))}\n")

    print("Standard Search:")
    print(f"  Documents processed: {N}")
    print(f"  Execution time: {standard_time:.6f} seconds")

    # نمایش 3 سند برتر
```

```

top_standard = sorted(standard_scores.items(), key=lambda x: x[1], reverse=True)[:3]
print("  Top documents:")
for doc_id, score in top_standard:
    print(f"    {doc_id}: {round(score, 4)}")
print()

for method, (count, time_taken, scores) in results.items():
    print(f"{method}:")
    print(f"  Documents processed: {count}")
    print(f"  Execution time: {time_taken:.6f} seconds")

    # نمایش 3 سند برتر
    top_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:3]
    print("  Top documents:")
    for doc_id, score in top_docs:
        print(f"    {doc_id}: {round(score, 4)}")
    print()

# نمایش نتایج به صورت نمودار
methods = ["Standard"] + list(results.keys())
doc_counts = [N] + [results[method][0] for method in results.keys()]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Documents Processed Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# نمایش ساختار خوشه‌ها برای b1=2, b2=2
b1, b2 = 2, 2
_, _, leaders, followers, _ = cluster_pruning_search(
    documents, query_terms, tf, idf, b1=b1, b2=b2
)

print(f"\nCluster Structure (b1={b1}, b2={b2}):")
print("Leaders and their followers:")
for leader in leaders:
    print(f"  {leader}: {followers[leader]}")

# اجرای مقایسه
compare_algorithms()

```

Query: catcher rye

Total documents: 8, Leaders: 2

Standard Search:

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc1: 1.0
 Doc4: 1.0
 Doc5: 1.0

Cluster (b1=1, b2=1):

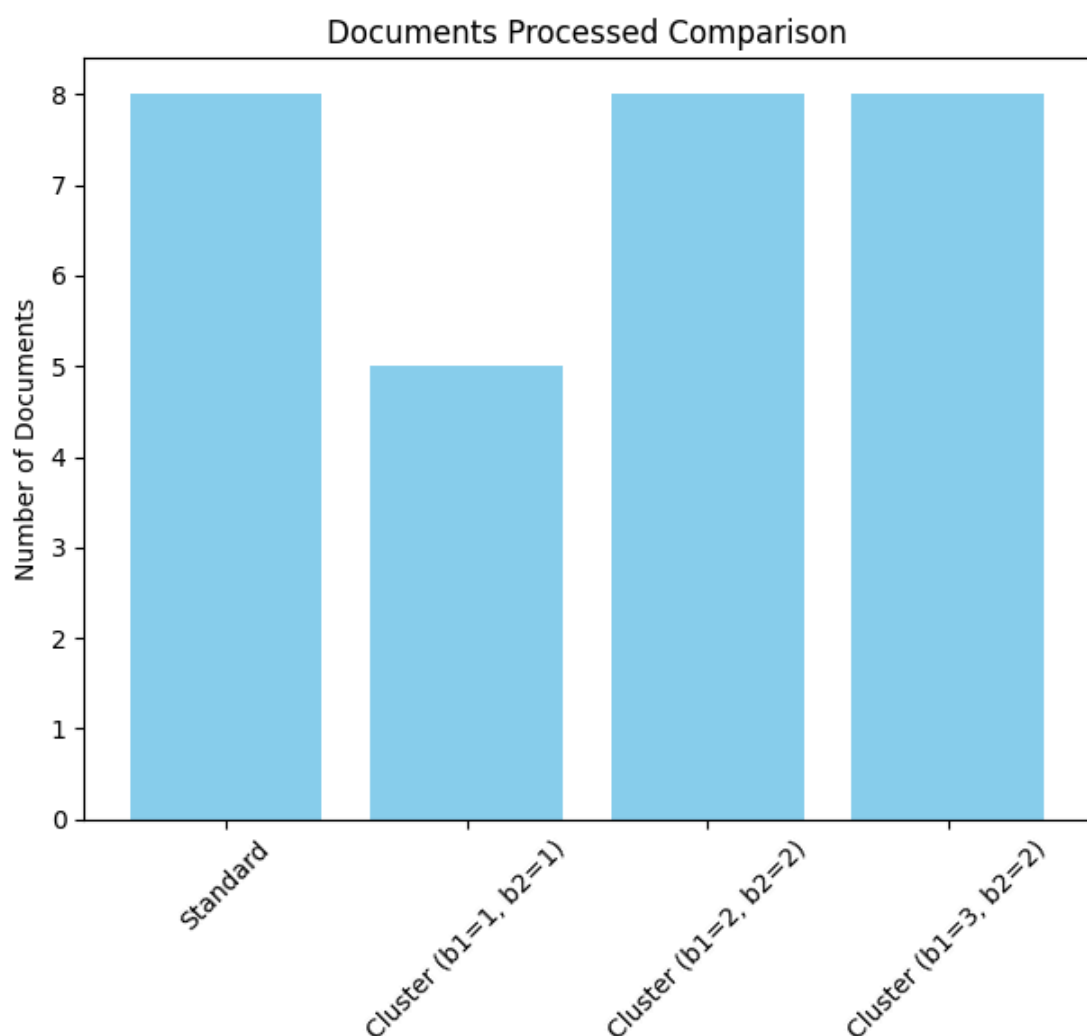
Documents processed: 5
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Cluster (b1=2, b2=2):

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Cluster (b1=3, b2=2):

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc1: 1.0
 Doc4: 1.0



Cluster Structure (b1=2, b2=2):

Leaders and their followers:

Doc5: ['Doc1', 'Doc3', 'Doc4', 'Doc6', 'Doc7', 'Doc8']

Doc2: ['Doc1', 'Doc3', 'Doc4', 'Doc6', 'Doc7', 'Doc8']

۷.۲ اجزای یک سیستم بازیابی اطلاعات + ۷.۲.۱ ایندکس‌های لایه‌ای

🎬 سناریوی واقعی: بهینه‌سازی جستجوی فیلم در IMDb

در سال ۲۰۱۹، IMDb با یک مسئله‌ی مهم روبه‌رو شد: کاربران معمولاً عبارت‌های گسترده‌ای مانند «best thriller movies» یا «romantic drama» جستجو می‌کردند. چنین پرس‌وجوهایی روی مجموعه‌ای شامل ده‌ها میلیون عنوان فیلم، سریال، اپیزود و داده‌های وابسته اجرا می‌شد و در مواردی زمان پاسخ‌گویی به چند ثانیه می‌رسید. علاوه بر کندی، نتایج اولیه گاهی شامل عناوینی بود که ارتباط کم‌تری با نیت کاربر داشتند. تیم جستجوی IMDb تصمیم گرفت معماری جستجو را بازطراحی کند و از ایندکس‌های چندلایه با آستانه‌های متفاوت tf استفاده کند.

موتور جستجو هنگام دریافت پرس‌وجو ابتدا لایه‌ی بسیار فشرده را بررسی می‌کرد. اگر تعداد نتایج کافی نبود یا تنوع لازم ایجاد نمی‌شد، لایه‌های بعدی فعال می‌شدند. نتیجه‌ی این بازطراحی: کاهش ۴۲٪ در زمان پاسخ‌گویی افزایش ۲۱٪ در ارتباط نتایج صرفه‌جویی قابل‌توجه در مصرف منابع سرور در ساعات پرتراфик این تغییر باعث شد تجربه‌ی جستجو در IMDb روان‌تر، دقیق‌تر و سازگارتر با نیاز کاربران شود.

در این بخش، تمام ایده‌های فصل ۷ را ترکیب می‌کنیم تا یک سیستم جست و جوی کامل و عملی بسازیم. این سیستم فقط محدود به مدل برداری نیست، بلکه قابلیت پشتیبانی از انواع اپراتورهای پرس‌مان و مدل‌های رتبه‌بندی را دارد.

یکی از چالش‌های مهم در بازیابی تقریبی K سند برتر، این است که ممکن است مجموعه‌ی کاندیدها (A) کمتر از K سند داشته باشد. برای حل این مشکل، از ساختار **ایندکس‌های لایه‌ای (Tiered Indexes)** استفاده می‌کنیم.

📌 ایندکس لایه‌ای چیست؟

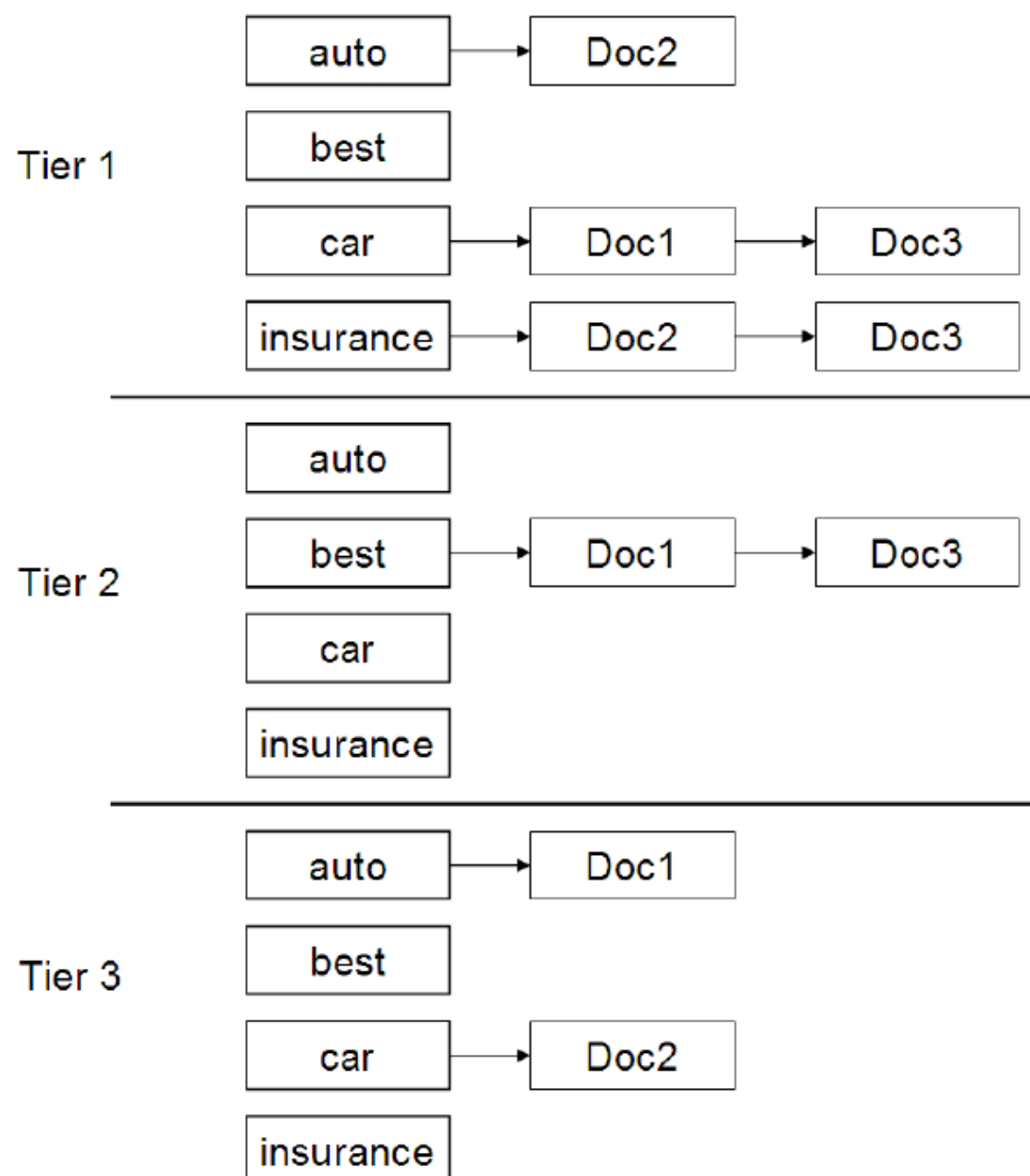
- برای هر واژه، چندین نسخه از لیست ارسال ساخته می‌شود — هرکدام مربوط به یک لایه (Tier)
- هر لایه فقط شامل اسنادی است که tf آن‌ها از یک آستانه مشخص بیشتر باشد
- مثلاً:

▪ **Tier 1:** فقط اسناد با $tf > 20$

▪ **Tier 2:** فقط اسناد با $tf > 10$

▪ **Tier 3:** همه اسناد باقی‌مانده

- در زمان جست و جو، ابتدا فقط از Tier 1 استفاده می‌کنیم
- اگر K سند کافی پیدا نشد، به Tier 2 و سپس Tier 3 می‌رویم



► **Figure 7.4** Tiered indexes. If we fail to get K results from tier 1, query processing “falls back” to tier 2, and so on. Within each tier, postings are ordered by document ID.

📌 مثال عددی: تأثیر ایندکس‌های لایه‌ای

فرض کنید برای واژه "machine learning" در یک پایگاه داده، لیست ارسال به صورت زیر است:

- روش استاندارد: باید ۱۰۰۰۰ سند را بررسی کنیم
- روش لایه‌ای:
 - Tier 1: فقط ۱۰۰۰ سند با $tf > 20$ را بررسی می‌کنیم
 - اگر ۱۰ سند برتر پیدا نشد، به Tier 2 می‌رویم و ۲۰۰۰ سند با $tf > 10$ را بررسی می‌کنیم
 - اگر باز هم ۱۰ سند برتر پیدا نشد، به Tier 3 می‌رویم و ۷۰۰۰ سند باقی‌مانده را بررسی می‌کنیم

با این روش، به احتمال بسیار بالایی در همان ۳۰۰۰ سند اول، ۱۰ سند برتر را پیدا می‌کنیم، در حالی که در روش استاندارد ممکن بود این ۱۰ سند در هر جای دیگری از کل مجموعه باشند.

مزایا: ✓

- افزایش سرعت جست و جو با تمرکز بر اسناد مهم‌تر

- امکان کنترل دقیق هزینهٔ محاسباتی با تنظیم آستانه‌ها
- قابل ترکیب با سایر heuristicها مثل حذف ایندکس و فهرست قهرمانان

🚩 کاربرد واقعی: جستجوی محصولات در eBay

eBay از نسخه‌ای پیشرفته از ایندکس‌های لایه‌ای برای جستجوی محصولات استفاده می‌کند. برای هر واژه، چندین نسخه از لیست ارسال بر اساس آستانه‌های tf مختلف ساخته می‌شود.

وقتی کاربر عبارت "Nothing phone case" را جستجو می‌کند، سیستم ابتدا از لیست با بالاترین آستانه استفاده می‌کند و محصولات با tf بالا را در نتایج اول نمایش می‌دهد. اگر تعداد کافی محصول پیدا نشود، به لیست‌های با آستانه‌های پایین‌تر می‌رود. این رویکرد باعث شده که کاربران معمولاً محصولات مرتبط‌تر و محبوب‌تر را در نتایج اول مشاهده کنند و تجربه خرید بهتری داشته باشند.

در ادامه، این ساختار را به‌صورت عملی روی دادهٔ فرضی پیاده‌سازی می‌کنیم.

```
In [27]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in the field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]
K = 3 # تعداد اسناد مورد نظر

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# جستجوی استاندارد
def standard_search(documents, query_terms, tf, idf):
    """جستجوی استاندارد بدون ایندکس‌های لایه‌ای"""
    start_time = time.time()

    # محاسبه امتیاز برای همه اسناد
    scores = {}
    query_tf = {term: 1 for term in query_terms} # پرسمان برابر 1 است tf فرض می‌کنیم

    for doc_id in documents:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)
```

```
dot = sum(d * q for d, q in zip(doc_vec, query_vec))
doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
query_norm = math.sqrt(sum(q**2 for q in query_vec))

if doc_norm > 0 and query_norm > 0:
    scores[doc_id] = dot / (doc_norm * query_norm)

elapsed_time = time.time() - start_time
return scores, elapsed_time

# جستجوی با ایندکس‌های لایه‌ای
def tiered_search(documents, query_terms, tf, idf, tiers, K):
    """جستجو با استفاده از ایندکس‌های لایه‌ای"""
    start_time = time.time()

    # ساخت ایندکس‌های لایه‌ای برای هر واژه
    tiered_index = {}
    for term in query_terms:
        tiered_index[term] = {tier: [] for tier in tiers}

        for doc_id in tf[term]:
            tf_val = tf[term][doc_id]
            for tier, threshold in tiers.items():
                if tf_val > threshold:
                    tiered_index[term][tier].append((doc_id, tf_val))
                    break # فقط در بالاترین لایه ممکن قرار می‌گیرد

    # سند K مرحله جست‌وجو: پیمایش لایه‌ها تا رسیدن به
    scores = defaultdict(float)
    used_docs = set()

    # این متغیر تعداد کل اسنادی را که از ایندکس "بررسی" می‌کنیم می‌شمارد
    documents_looked_up = 0

    for tier in tiers:
        # پیدا کردن اسنادی که تمام واژه‌های پرسمان را در این لایه دارند
        tier_docs = set()
        for term in query_terms:
            term_docs = {doc_id for doc_id, _ in tiered_index[term][tier]}
            if not tier_docs:
                tier_docs = term_docs
            else:
                tier_docs = tier_docs.intersection(term_docs)

        # تعداد اسناد پیدا شده در این لایه را به شمارنده کل اضافه می‌کنیم
        documents_looked_up += len(tier_docs)

        # پردازش اسناد این لایه
        for doc_id in tier_docs:
            if doc_id not in used_docs:
                # هم استفاده کرد tf-idf می‌توانست از محاسبه امتیاز اولیه برای مرتب‌سازی
                # کافیت used_docs در اینجا فقط برای اضافه کردن به
                for term in query_terms:
                    if doc_id in tf[term]:
                        scores[doc_id] += tf[term][doc_id] * idf[term]
                used_docs.add(doc_id)

            # اگر به تعداد کافی سند رسیدیم، از حلقه خارج شویم
            if len(used_docs) >= K:
                break

        # اگر به تعداد کافی سند رسیدیم، از حلقه خارج شویم
        if len(used_docs) >= K:
            break

    # محاسبه امتیاز نهایی با استفاده از شباهت کسینوسی
    final_scores = {}
    query_tf = {term: 1 for term in query_terms}

    for doc_id in used_docs:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            final_scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    # <<< شروع تغییر >>>
    # برگرداندن مقدار صحیح تعداد اسناد بررسی شده
    return final_scores, documents_looked_up, elapsed_time, tiered_index
    # <<< پایان تغییر >>>

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم ایندکس‌های لایه‌ای"""
    # tf، df و idf محاسبه
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # جستجوی استاندارد
    standard_scores, standard_time = standard_search(documents, query_terms, tf, idf)
```

```
# تعریف لایه‌ها
tiers = {
    "Tier1": 4,
    "Tier2": 2,
    "Tier3": 0
}

# جستجوی با ایندکس‌های لایه‌ای
tiered_scores, tiered_count, tiered_time, tiered_index = tiered_search(
    documents, query_terms, tf, idf, tiers, K
)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}, K={K}\n")

print("Standard Search (Cosine Similarity):")
print(f"  Documents processed: {len(documents)}")
print(f"  Execution time: {standard_time:.6f} seconds")

# نمایش 3 سند برتر
top_standard = sorted(standard_scores.items(), key=lambda x: x[1], reverse=True)[:K]
print("  Top documents:")
for doc_id, score in top_standard:
    print(f"    {doc_id}: {round(score, 4)}")
print()

print("Tiered Index Search (Cosine Similarity):")
print(f"  Documents processed: {tiered_count}")
print(f"  Execution time: {tiered_time:.6f} seconds")

# نمایش 3 سند برتر
top_tiered = sorted(tiered_scores.items(), key=lambda x: x[1], reverse=True)[:K]
print("  Top documents:")
for doc_id, score in top_tiered:
    print(f"    {doc_id}: {round(score, 4)}")
print()

# نمایش نتایج به صورت نمودار
methods = ["Standard", "Tiered"]
doc_counts = [len(documents), tiered_count]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Documents Processed Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

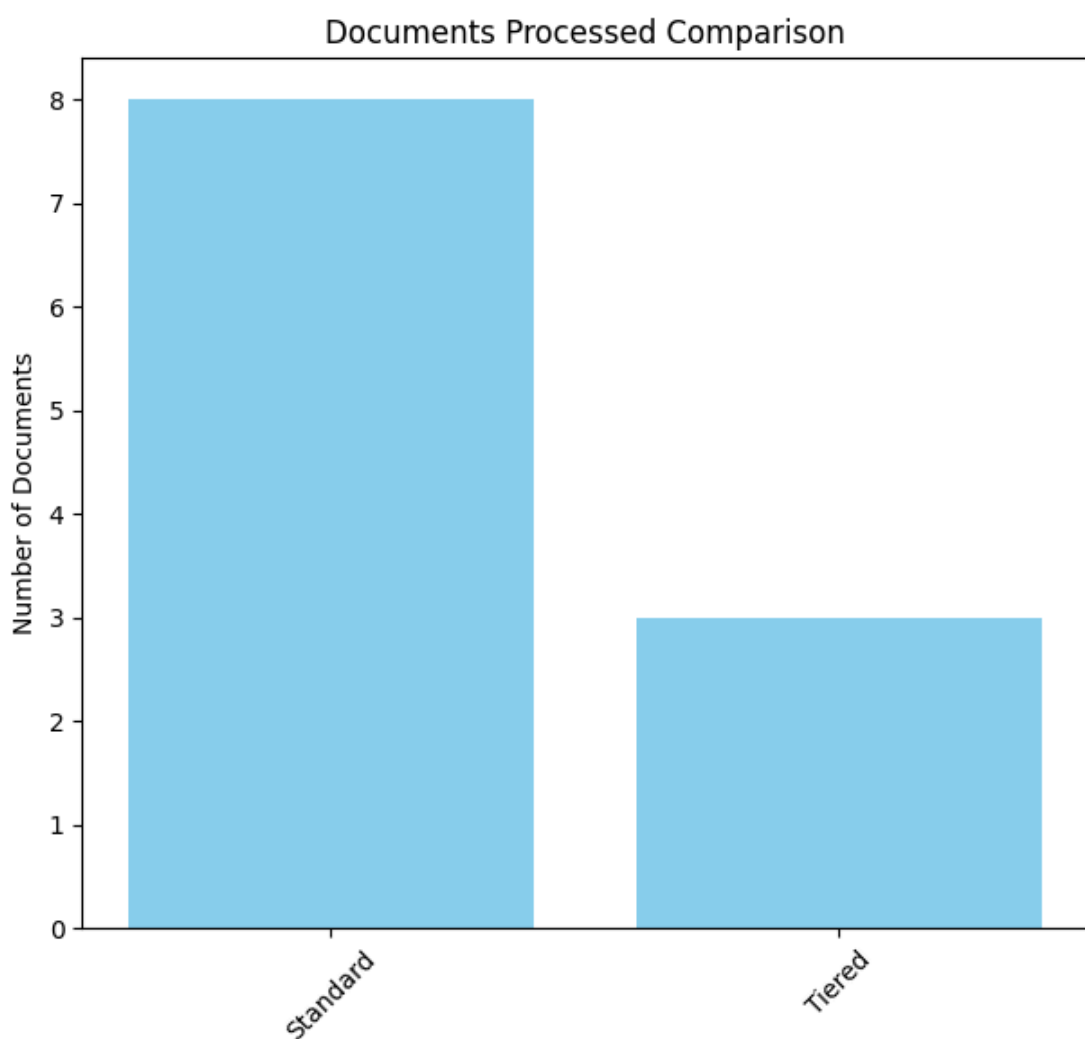
# "catcher" نمایش تعداد اسناد در هر لایه برای واژه
term = "catcher"
print(f"\nDocuments per tier for term '{term}':")
for tier in tiers:
    count = len(tiered_index[term][tier])
    print(f"  {tier}: {count} documents")

# اجرای مقایسه
compare_algorithms()
```

Query: catcher rye, K=3

Standard Search (Cosine Similarity):
Documents processed: 8
Execution time: 0.000000 seconds
Top documents:
Doc1: 1.0
Doc4: 1.0
Doc5: 1.0

Tiered Index Search (Cosine Similarity):
Documents processed: 3
Execution time: 0.000000 seconds
Top documents:
Doc4: 1.0
Doc1: 1.0
Doc5: 1.0



Documents per tier for term 'catcher':
Tier1: 0 documents
Tier2: 1 documents
Tier3: 4 documents

7.2.2 نزدیکی واژه‌های پرسمان (Query-Term Proximity)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی مقالات علمی در PubMed

در سال ۲۰۱۷، تیم PubMed با چالشی بزرگ روبرو بود: محققان به دنبال مقالاتی بودند که واژه‌های کلیدی مانند "CRISPR" و "gene editing" در نزدیکی به هم ظاهر می‌شدند. اما سیستم جستجوی موجود فقط بر اساس تکرار واژه‌ها رتبه بندی می‌کرد و نمی‌توانست تفاوت بین مقالاتی که این واژه‌ها را در یک جمله به کار می‌بردند با مقالاتی که واژه‌ها در بخش‌های مختلفی از متن ظاهر می‌شدند، تشخیص دهد.

تیم PubMed با پیاده‌سازی سیستم نزدیکی واژه‌های پرسمان، توانست دقت جستجو را ۲۵٪ افزایش دهد. در این سیستم، اگر واژه‌ها در یک پنجره کوچک (مثلاً ۵ واژه) در کنار هم ظاهر می‌شدند، امتیاز بالاتری دریافت می‌کردند. این کار باعث شد که محققان بتوانند مقالات مرتبط تر و جدیدتری را در حوزه تحقیقات خود پیدا کنند.

در جست‌وجوهای آزاد (free text)، کاربران معمولاً اسنادی را ترجیح می‌دهند که واژه‌های پرسمان در آن‌ها **نزدیک به هم** ظاهر شوند. این نزدیکی نشان‌دهندهٔ تمرکز سند بر موضوع پرسمان است.

برای سنجش نزدیکی، از معیار **عرض کوچکترین پنجره (ω)** استفاده می‌کنیم:

$\omega(d, q)$ = number of terms in the smallest window of document d that contains all query terms in q

مثال: اگر سند شامل جملهٔ **The quality of mercy is not strained** باشد، برای پرسمان **strained mercy** مقدار ω برابر ۴ است.

📌 نکات کلیدی:

- هرچه ω کوچک‌تر باشد، سند با پرسمان مرتبط تر است
- اگر سند شامل همهٔ واژه‌های پرسمان نباشد، مقدار ω را عددی بزرگ (مثلاً ∞) در نظر می‌گیریم
- می‌توان در محاسبهٔ ω فقط واژه‌های غیر stop word را لحاظ کرد

- این روش از شباهت کسینوسی فاصله می‌گیرد و به رتبه‌بندی معنایی نزدیک‌تر می‌شود

✅ کاربرد در سیستم‌های واقعی:

- در موتورهای جست‌وجوی وب مثل Google، نزدیکی واژه‌ها نقش مهمی در رتبه‌بندی دارد
- می‌توان ω را به‌عنوان یک ویژگی در مدل یادگیری ماشین لحاظ کرد (بخش ۶.۱.۲ و ۱۵.۴.۱)
- در نسخهٔ ساده، می‌توان امتیاز نهایی را به‌صورت زیر تعریف کرد:

$$\text{score}(d) = \frac{1}{\omega(d, q)}$$

🚀 کاربرد واقعی: جستجوی مقالات علمی در Google Scholar

Google Scholar از نسخه‌ای پیشرفته از نزدیکی واژه‌ها برای جستجوی مقالات علمی استفاده می‌کند. در این سیستم، اگر واژه‌های کلیدی در یک پنجره کوچک (مثلاً ۱۰ واژه) در کنار هم ظاهر شوند، امتیاز بسیار بالایی دریافت می‌کنند.

وقتی محققان عبارت "CRISPR gene editing in human embryos" را جستجو می‌کنند، سیستم مقالاتی را بالاتر قرار می‌دهد که این سه واژه در یک پنجره کوچک در متن مقاله ظاهر شده‌اند، حتی اگر مقالات دیگری نیز وجود داشته باشند که این واژه‌ها را بیشتر تکرار کرده‌اند. این رویکرد باعث شده که محققان بتوانند جدیدترین و مرتبط‌ترین تحقیقات را در حوزه مورد نظر خود پیدا کنند.

در ادامه، این معیار را به صورت عملی روی دادهٔ فرضی پیاده‌سازی می‌کنیم.

```
In [31]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف اسناد فرضی
documents = {
    "Doc1": "the quality of mercy is not strained",
    "Doc2": "mercy mercy mercy mercy mercy",
    "Doc3": "the mercy of strained quality is evident",
    "Doc4": "strained mercy is a virtue",
    "Doc5": "mercy mercy is not strained but quality is evident",
    "Doc6": "strained mercy mercy mercy mercy mercy"
}

# پرسمان
query_terms = ["strained", "mercy"]

# تابع محاسبه عرض پنجره کوچک‌ترین (w)
def compute_window_width(tokens, query_terms):
    """که همه واژه‌های پرسمان را شامل شود (w) محاسبه عرض پنجره کوچک‌ترین"""
    positions = {term: [] for term in query_terms}
    for i, token in enumerate(tokens):
        if token in query_terms:
            positions[token].append(i)

    # اگر یکی از واژه‌ها اصلاً وجود ندارد
    if any(len(pos) == 0 for pos in positions.values()):
        return float('inf')

    # تولید همه ترکیب‌های ممکن از موقعیت‌ها
    import itertools
    all_combinations = list(itertools.product(*positions.values()))
    min_width = float('inf')

    for combo in all_combinations:
        width = max(combo) - min(combo) + 1
        if width < min_width:
            min_width = width

    return min_width

# تابع محاسبه امتیاز نزدیکی
def compute_proximity_score(documents, query_terms):
    """محاسبه امتیاز نزدیکی برای اسناد"""
    omega_scores = {}

    for doc_id, text in documents.items():
        tokens = text.lower().split()
        omega = compute_window_width(tokens, query_terms)
```

```
score = round(1 / omega, 4) if omega != float('inf') else 0
omega_scores[doc_id] = (omega, score)

return omega_scores

# تابع نمایش نتایج
def display_results(query_terms, omega_scores):
    """نمایش نتایج به صورت جدول و نمودار"""
    print(f"پرسمان: {' '.join(query_terms)}\n")
    print(f"proximity و امتیاز (w) عرض پنجره کوچکترین")

    # proximity مرتب‌سازی بر اساس امتیاز
    sorted_docs = sorted(omega_scores.items(), key=lambda x: x[1][1], reverse=True)

    for doc_id, (omega, score) in sorted_docs:
        print(f"{doc_id}: w = {omega}, score = {score}")

    # نمایش نتایج به صورت نمودار
    doc_ids = [doc_id for doc_id, _ in sorted_docs]

    omega_values = [omega if omega != float('inf') else 10 for _, (omega, score) in sorted_docs]
    scores = [score for _, (omega, score) in sorted_docs]

    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.bar(doc_ids, omega_values, color='skyblue')
    plt.xlabel('Documents')
    plt.ylabel('Window Width (w)')
    plt.title('Window Width for Each Document')
    plt.xticks(rotation=45)

    plt.subplot(1, 2, 2)
    plt.bar(doc_ids, scores, color='lightgreen')
    plt.xlabel('Documents')
    plt.ylabel('Proximity Score')
    plt.title('Proximity Score for Each Document')
    plt.xticks(rotation=45)

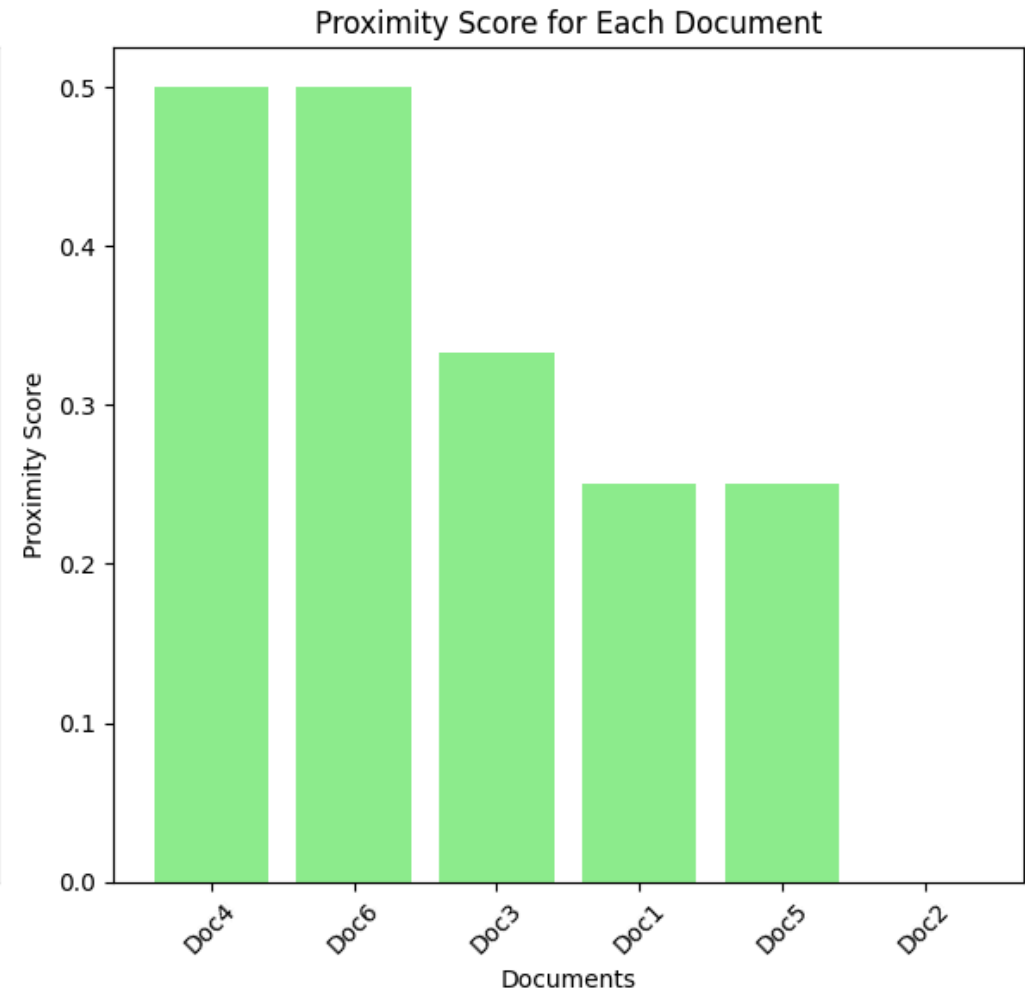
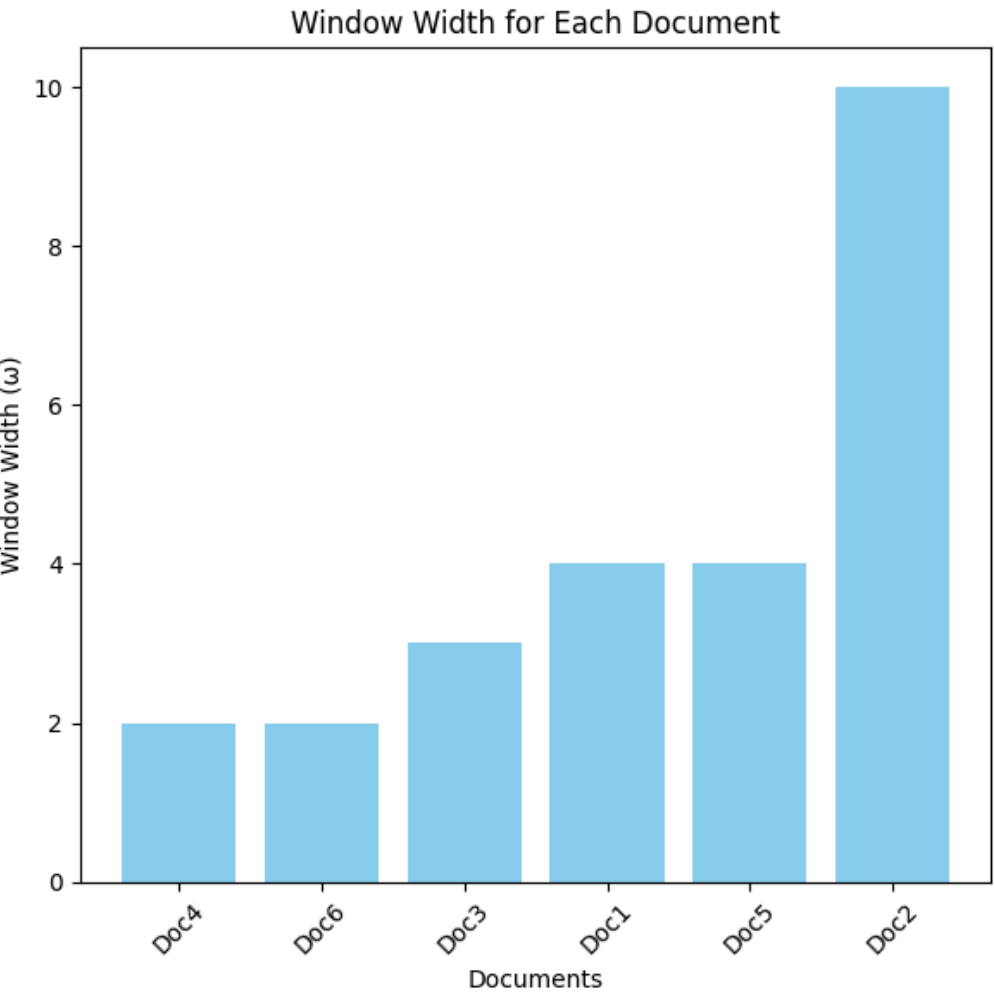
    plt.tight_layout()
    plt.show()

# اجرای الگوریتم
omega_scores = compute_proximity_score(documents, query_terms)
display_results(query_terms, omega_scores)
```

پرسمان: strained mercy

proximity و امتیاز (w) عرض پنجره کوچکترین:

Doc4: w = 2, score = 0.5
Doc6: w = 2, score = 0.5
Doc3: w = 3, score = 0.3333
Doc1: w = 4, score = 0.25
Doc5: w = 4, score = 0.25
Doc2: w = inf, score = 0



۷.۲.۳ طراحی توابع تجزیه و امتیازدهی 🔍

وقتی در گوگل عبارت «پیش‌بینی قیمت سهام اپل» را جستجو می‌کنید، آیا تا به حال فکر کرده‌اید که در پس این جستجوی ساده چه فرآیندهای پیچیده‌ای در جریان است؟ اکثر کاربران وب، پرسمان‌های خود را به صورت متن آزاد وارد می‌کنند و انتظار دارند

سیستم جستجو به هوشمندانه‌ترین شکل ممکن به آن‌ها پاسخ دهد. در این بخش، به بررسی معماری پشت صحنه این سیستم‌ها می‌پردازیم.

✦ مثال واقعی: جستجوی «پیش‌بینی قیمت سهام اپل» در یک موتور جستجوی مالی

یک سیستم جستجوی پیشرفته مانند Bloomberg Terminal این پرسمان را به صورت مرحله‌ای پردازش می‌کند:

- ابتدا جستجوی عبارت دقیق: "پیش‌بینی قیمت سهام اپل"
 - اگر نتایج کافی نبود، جستجوی عبارت‌های دو بخشی: "پیش‌بینی قیمت" و "قیمت سهام اپل"
 - در نهایت، جستجوی واژه‌های تک‌تک: پیش‌بینی، قیمت، سهام، اپل
- جالب اینجاست که یک سند ممکن است در چند مرحله ظاهر شود! برای مثال، گزارش تحلیلی شرکت Goldman Sachs ممکن است هم در جستجوی عبارت دقیق و هم در جستجوی واژه‌های تک‌تک یافت شود. در این حالت، سیستم به یک تابع امتیازدهی **تجمیعی** نیاز دارد تا شواهد مختلف را ترکیب کند.

✦ منابع امتیازدهی در دنیای واقعی:

- امتیاز شباهت برداری (vector space) - مانند امتیاز TF-IDF
- امتیاز کیفیت ایستا $g(d)$ - مانند اعتبار منبع (مثلاً گزارش‌های رسمی اپل در مقابل وبلاگ‌های شخصی)
- امتیاز نزدیکی واژه‌ها (proximity) - فاصله بین کلمات «پیش‌بینی» و «اپل» در سند
- تطابق با اپراتورهای پرسمان (عبارتی، بولی، ناحیه‌ای)
- ویژگی‌های ساختاری (metadata, zones) - مانند اینکه آیا اطلاعات در بخش عنوان، چکیده یا متن اصلی قرار دارد

در سیستم‌های واقعی مانند جستجوی گوگل، این ترکیب به صورت پیچیده‌تری انجام می‌شود:

$$\text{score}(d) = \alpha_1 \cdot \text{cosine}(q, d) + \alpha_2 \cdot g(d) + \alpha_3 \cdot \frac{1}{\omega(d, q)} + \dots$$

که در آن $\omega(d, q)$ فاصله میانگین بین کلمات کلیدی در سند است.

✓ طراحی لایه تجزیه (Query Parser):

در شرکت‌های بزرگی مانند آمازون، تیم‌های تخصصی مسئول طراحی این لایه هستند. آن‌ها با تحلیل میلیون‌ها پرسمان روزانه کاربران، الگوهای رفتاری را شناسایی کرده و سیستم را بهینه می‌کنند. برای مثال، وقتی کاربران عبارت «خرید لپ‌تاپ گیمینگ ارزان» را جستجو می‌کنند، سیستم به طور خودکار آن را به چند پرسمان مختلف تقسیم می‌کند و نتایج را بر اساس ترکیبی از فاکتورها رتبه‌بندی می‌کند.

✦ دو سناریوی واقعی:

- سیستم‌های سازمانی (Enterprise):** در شرکت‌های بزرگ مانند IBM یا Microsoft، توسعه‌دهندگان با استفاده از ابزارهای موجود، توابع امتیازدهی را بر اساس نیازهای خاص سازمان تنظیم می‌کنند. برای مثال، در سیستم جستجوی داخلی مایکروسافت، اسناد رسمی شرکت امتیاز بالاتری نسبت به ایمیل‌های شخصی کارمندان دریافت می‌کنند.
- موتورهای جستجوی وب عمومی:** گوگل و بینگ از مدل‌های یادگیری ماشین برای ترکیب صدها ویژگی استفاده می‌کنند. این مدل‌ها با تحلیل میلیاردها جستجو، به طور خودکار وزن‌های مختلف را تنظیم می‌کنند. برای مثال، گوگل از سیگنال‌هایی مانند موقعیت جغرافیایی کاربر، تاریخچه جستجوها، و زمان سپری شده در صفحات نتایج برای بهبود رتبه‌بندی استفاده می‌کند.

در فصل ۱۵، مدل‌های یادگیری امتیازدهی (Learning to Rank) را به تفصیل بررسی خواهیم کرد و خواهیم دید که چگونه الگوریتم‌های پیچیده‌ای مانند RankNet و LambdaMART به موتورهای جستجو کمک می‌کنند تا نتایج دقیق‌تری ارائه دهند.

۷.۲.۴ جمع‌بندی همه اجزا

تاکنون تمام اجزای لازم برای یک سیستم جستجوی پایه که از پرسمان‌های متن آزاد، بولی، منطقه و فیلد پشتیبانی می‌کند را بررسی کرده‌ایم. در این بخش مروری می‌کنیم که این قطعات مختلف چگونه در یک سیستم کلی کنار هم قرار می‌گیرند.

📌 مثال: سیستم جستجوی داخلی شرکت میکروسافت

بیایید نگاهی به نحوه عملکرد سیستم جستجوی داخلی میکروسافت بیندازیم که روزانه به هزاران کاربر اجازه می‌دهد به اسناد مختلف شرکت دسترسی پیدا کنند:

وقتی یک مهندس در میکروسافت یک سند فنی جدید در SharePoint آپلود می‌کند، این سند از سمت چپ وارد سیستم می‌شود. ابتدا پردازش زبانی و قالب‌بندی روی آن انجام می‌شود: سیستم زبان سند را تشخیص می‌دهد (انگلیسی، چینی، یا غیره)، آن را به توکن‌های کوچکتر تقسیم کرده و ریشه‌یابی (stemming) انجام می‌دهد.

جریان توکن‌های حاصل به دو ماژول مختلف می‌رود:

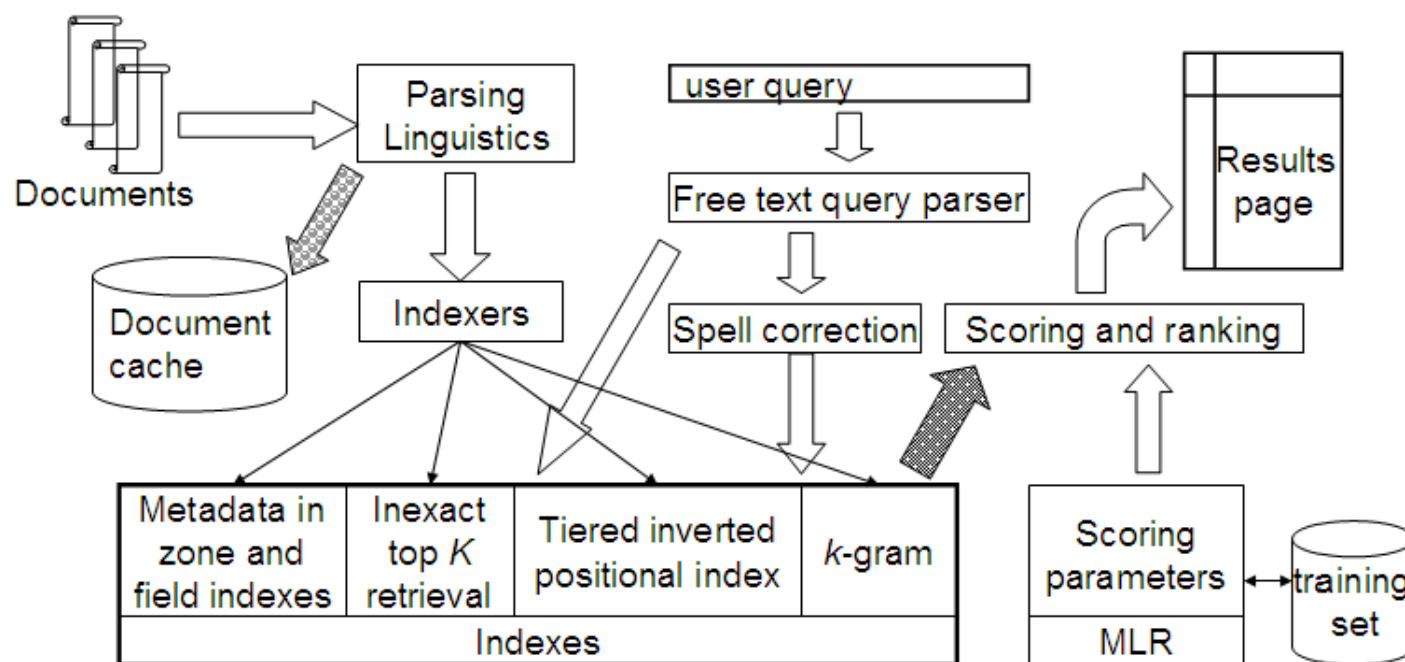
- یک نسخه از هر سند پردازش‌شده در کش سند ذخیره می‌شود تا بتوانیم برش‌های متن (snippets) را برای نمایش در نتایج جستجو تولید کنیم. این برش‌ها به کاربر نشان می‌دهند چرا سند مورد نظر با پرسمان او مطابقت دارد.
 - نسخه دوم توکن‌ها به بانک ایندکس‌ها ارسال می‌شود که شامل ایندکس‌های منطقه و فیلد برای ذخیره متادیتا هر سند، ایندکس‌های موقعیتی، ایندکس‌های اصلاح غلط املائی و ساختارهایی برای تسریع بازیابی نامحدود top-K است.
- وقتی یک کاربر در میکروسافت عبارت "Azure security best practices" را جستجو می‌کند، این پرسمان هم مستقیماً به ایندکس‌ها ارسال می‌شود و هم از طریق ماژول تولید کاندیدهای اصلاح غلط املائی عبور می‌کند. این ماژول فقط زمانی فعال می‌شود که پرسمان اصلی نتایج کافی نداشته باشد.

اسناد بازیابی شده به ماژول امتیازدهی ارسال می‌شوند که امتیازات را بر اساس رتبه‌بندی یادگیری ماشین (MLR) محاسبه می‌کند. این تکنیک که بر اساس مفاهیم بخش ۶.۱.۲ ساخته شده و در بخش ۱۵.۴.۱ بیشتر توسعه داده می‌شود، به سیستم اجازه می‌دهد تا با ترکیب صدها سیگنال مختلف، مرتبط ترین نتایج را در صدر نمایش دهد.

در نهایت، این اسناد رتبه‌بندی شده به صورت یک صفحه نتایج به کاربر نمایش داده می‌شوند.

📌 مسیر داده در یک سیستم جستجوی کامل

شکل ۷.۵ یک سیستم جستجوی کامل را نشان می‌دهد که مسیرهای داده عمدتاً برای یک پرسمان متن آزاد را نمایش می‌دهد. این معماری مشابه معماری استفاده شده در سیستم‌های بزرگ مانند جستجوی گوگل یا الکسا (آمازون) است، هرچند در مقیاس بسیار بزرگتر.



► Figure 7.5 A complete search system. Data paths are shown primarily for a free text query.

شکل 7.5 : یک سیستم جست و جو کامل. مسیرهای داده عمدتاً برای یک پرسوجوی متنی آزاد نمایش داده شده اند.

در فصل ۸، بخش ۸.۷، به طور مفصل به تولید خودکار برش‌های متن (snippets) خواهیم پرداخت که به کاربران کمک می‌کند به سرعت بفهمند چرا یک سند با پرسمان آن‌ها مرتبط است.

۷.۳ امتیازدهی فضای برداری و تعامل با عملگرهای پرسمان

مدل فضای برداری را به عنوان یک پارادایم برای پرسمان‌های متن آزاد معرفی کردیم. در این بخش، بررسی می‌کنیم که مدل امتیازدهی فضای برداری چگونه با عملگرهای پرسمانی که در فصل‌های قبلی مطالعه کردیم، ارتباط دارد. این ارتباط باید در دو سطح بررسی شود: از نظر قدرت بیانی پرسمان‌هایی که یک کاربر پیشرفته ممکن است مطرح کند، و از نظر ایندکسی که ارزیابی روش‌های مختلف بازیابی را پشتیبانی می‌کند.

✦ بازیابی متن آزاد در موتورهای جستجوی مدرن

در گذشته، پرسمان‌های متن آزاد به این صورت تفسیر می‌شدند که حداقل یکی از واژه‌های پرسمان در هر سند بازیابی شده وجود داشته باشد. اما امروزه، موتورهای جستجوی وب مانند گوگل این مفهوم را محبوب کرده‌اند که مجموعه‌ای از واژه‌های تایپ شده در جعبه پرسمان (که در ظاهر یک پرسمان متن آزاد است)، معنای یک پرسمان عطفی را دارد که فقط اسنادی را بازیابی می‌کند که حاوی تمام یا اکثر واژه‌های پرسمان هستند.

برای مثال، وقتی در گوگل عبارت "رستوران ایتالیایی تهران" را جستجو می‌کنید، گوگل به طور ضمنی به دنبال اسنادی می‌گردد که هر سه واژه "رستوران"، "ایتالیایی" و "تهران" را در خود دارند، نه فقط یکی از این واژه‌ها را.

✦ بازیابی بولی

به وضوح، یک ایندکس فضای برداری می‌تواند برای پاسخ به پرسمان‌های بولی استفاده شود، به شرطی که وزن واژه t در بردار سند d هر زمان که t در d وجود دارد، غیرصفر باشد. برعکس این موضوع صحیح نیست، زیرا یک ایندکس بولی به طور پیش‌فرض اطلاعات وزن واژه را حفظ نمی‌کند.

از دیدگاه کاربر، هیچ راه آسانی برای ترکیب پرسمان‌های فضای برداری و بولی وجود ندارد: پرسمان‌های فضای برداری اساساً یک شکل تجمع شواهد هستند، که در آن وجود واژه‌های بیشتر پرسمان در یک سند به امتیاز آن سند اضافه می‌شود. از سوی دیگر،

بازیابی بولی از کاربر می‌خواهد که یک فرمول برای انتخاب اسناد از طریق وجود (یا عدم وجود) ترکیبات خاصی از کلیدواژه‌ها مشخص کند، بدون اینکه هیچ ترتیب نسبی بین آنها ایجاد کند.

در گذشته، سایت‌هایی مانند AltaVista به کاربران اجازه می‌دادند از عملگرهای بولی مانند AND، OR و NOT استفاده کنند. برای مثال، کاربر می‌توانست پرسمان "java AND NOT coffee" را وارد کند تا اسنادی که حاوی واژه java هستند اما coffee نیستند، بازیابی شوند. امروزه گوگل این عملگرها را به طور مستقیم پشتیبانی نمی‌کند، اما معادل‌هایی مانند استفاده از علامت منها (-) برای NOT را ارائه می‌دهد.

🌸 پرسمان‌های حروف جایگزین (Wildcard)

پرسمان‌های wildcard و فضای برداری به ایندکس‌های مختلفی نیاز دارند، به جز در سطح پایه که هر دو می‌توانند با استفاده از postings و یک دیکشنری پیاده‌سازی شوند (مثلاً یک دیکشنری از trigram برای پرسمان‌های wildcard).

اگر یک موتور جستجو به کاربر اجازه دهد تا یک عملگر wildcard را به عنوان بخشی از یک پرسمان متن آزاد مشخص کند (مثلاً پرسمان "نرم* افزار")، ما می‌توانیم بخش wildcard پرسمان را به عنوان ایجاد چندین واژه در فضای برداری تفسیر کنیم (در این مثال، "نرم‌افزار" و "نرم‌افزارسازی" دو واژه چنین هستند) که همگی به بردار پرسمان اضافه می‌شوند.

سایت‌های جستجوی شغلی مانند LinkedIn اغلب از wildcard استفاده می‌کنند. برای مثال، وقتی یک کارفرما "توسعه* وب" را جستجو می‌کند، سیستم ممکن است این را به "توسعه‌دهنده وب"، "توسعه‌دهندگان وب" و "توسعه وب" گسترش دهد و سپس با استفاده از مدل فضای برداری، اسناد را بر اساس تطابق با این واژه‌ها امتیازدهی و رتبه‌بندی کند.

🌸 پرسمان‌های عبارتی (Phrase)

نمایش اسناد به صورت برداری اساساً باعث از دست رفتن اطلاعات است: ترتیب نسبی واژه‌ها در یک سند در کدگذاری سند به عنوان یک بردار از بین می‌رود. حتی اگر بخواهیم هر biword را به عنوان یک واژه (و بنابراین یک محور در فضای برداری) در نظر بگیریم، وزن‌ها روی محورهای مختلف مستقل نیستند: برای مثال، عبارت "German shepherd" در محور "german shepherd" کدگذاری می‌شود، اما بلافاصله وزن غیرصفر روی محورهای "german" و "shepherd" نیز دارد.

بنابراین، یک ایندکس که برای بازیابی فضای برداری ساخته شده، به طور کلی نمی‌تواند برای پرسمان‌های عبارتی استفاده شود. علاوه بر این، هیچ راهی برای درخواست امتیاز فضای برداری برای یک پرسمان عبارتی وجود ندارد - ما فقط وزن نسبی هر واژه در یک سند را می‌دانیم.

گوگل به کاربران اجازه می‌دهد با قرار دادن یک عبارت در نقل قول، جستجوی دقیق عبارت را انجام دهند. برای مثال، جستجوی "هوش مصنوعی" فقط اسنادی را بازیابی می‌کند که این عبارت دقیق به همان ترتیب در آنها وجود دارد. این با جستجوی هوش مصنوعی (بدون نقل قول) متفاوت است که از مدل فضای برداری استفاده می‌کند و اسنادی را بر اساس وجود واژه‌های "هوش" و "مصنوعی" رتبه‌بندی می‌کند، حتی اگر این واژه‌ها کنار هم نباشند.

در حالی که این دو پارادایم بازیابی (عبارتی و فضای برداری) در نتیجه پیاده‌سازی‌های مختلفی از نظر ایندکس‌ها و الگوریتم‌های بازیابی دارند، آنها در برخی موارد می‌توانند به صورت مفیدی ترکیب شوند، مانند مثال سه مرحله‌ای تجزیه پرسمان در بخش ۷.۲.۳.

جمع‌بندی فصل ۷: محاسبه امتیاز در سیستم جستجوی کامل 🌸

در این فصل با روش‌های بهینه‌سازی برای محاسبه امتیاز و ساخت یک سیستم جستجوی کامل آشنا شدیم:

- الگوریتم FASTCOSINESCORE را برای محاسبه سریع امتیازات برداری معرفی کردیم. این همان روشی است که به گوگل اجازه می‌دهد میلیاردها سند را در کسری از ثانیه رتبه‌بندی کند.
- روش‌های بازیابی تقریبی K سند برتر را بررسی کردیم. این تکنیک‌ها به سیستم‌های جستجو کمک می‌کنند تا با هزینه کمتر، نتایج مرتبط را سریع‌تر پیدا کنند.
- فهرست‌های چندلایه (Tiered indexes) را به‌عنوان راهی برای بهینه‌سازی جستجو معرفی کردیم. این روش در سیستم‌های بزرگ مانند گوگل استفاده می‌شود تا نتایج را به سرعت نمایش دهد.
- امتیازدهی بر اساس نزدیکی واژه‌ها (Query-term proximity) را بررسی کردیم. این همان روشی است که باعث می‌شود وقتی «پیتزا نزدیک من» را جستجو می‌کنید، نتایجی نمایش داده شود که این دو کلمه در کنار هم قرار دارند.
- طراحی توابع تحلیل و امتیازدهی را برای ترکیب عوامل مختلف در رتبه‌بندی بررسی کردیم. این همان روشی است که به گوگل اجازه می‌دهد صدها عامل مختلف را برای رتبه‌بندی نتایج ترکیب کند.

بیا یاد این مفاهیم را با چند مثال واقعی مقایسه کنیم:

- ♦ **بازیابی تقریبی K سند برتر:** تصور کنید در دیجی‌کالا به دنبال «گوشی سامسونگ با حافظه ۲۵۶ گیگابایت» می‌گردید. دیجی‌کالا از روش‌های بازیابی تقریبی استفاده می‌کند تا به جای بررسی تمام محصولات، فقط تعداد محدودی از محصولات مرتبط را بررسی کند. این کار باعث می‌شود نتایج سریع‌تر نمایش داده شوند، حتی اگر ممکن است چند محصول مرتبط از قلم بیفتند. این روش در سیستم‌های بزرگ با میلیون‌ها محصول ضروری است.
- ♦ **فهرست‌های چندلایه (Tiered indexes):** گوگل از فهرست‌های چندلایه برای جستجو استفاده می‌کند. تصور کنید در گوگل «رستوران ایتالیایی در تهران» را جستجو می‌کنید. گوگل ابتدا در لایه اول فقط رستوران‌هایی را بررسی می‌کند که امتیاز بالایی دارند و کلمات کلیدی اصلی را در عنوان خود دارند. اگر نتایج کافی نبود، به لایه‌های بعدی می‌رود و رستوران‌های بیشتری را بررسی می‌کند. این روش باعث می‌شود نتایج مرتبط سریع‌تر نمایش داده شوند.
- ♦ **امتیازدهی بر اساس نزدیکی واژه‌ها:** وقتی در گوگل «پزشک متخصص قلب در شیراز» را جستجو می‌کنید، گوگل به اسنادی امتیاز بالاتری می‌دهد که این واژه‌ها در کنار هم قرار دارند. برای مثال، صفحه‌ای که عبارت «پزشک متخصص قلب در شیراز» را دارد، امتیاز بالاتری از صفحه‌ای می‌گیرد که این واژه‌ها در نقاط مختلف صفحه پراکنده شده‌اند. این روش باعث می‌شود نتایج مرتبط‌تر در بالای لیست قرار گیرند.
- ♦ **طراحی توابع تحلیل و امتیازدهی:** گوگل از صدها عامل برای رتبه‌بندی نتایج استفاده می‌کند: از امتیازدهی برداری گرفته تا کیفیت صفحه، سرعت بارگذاری، تجربه کاربری و ده‌ها عامل دیگر. این عوامل با استفاده از یادگیری ماشین ترکیب می‌شوند تا بهترین نتایج را برای شما نمایش دهد. برای مثال، وقتی «خرید آنلاین کتاب» را جستجو می‌کنید، گوگل فروشگاه‌های معتبر با امتیاز بالا، سرعت بارگذاری خوب و نظرات مثبت کاربران را در بالای نتایج نمایش می‌دهد.

این فصل مفاهیم اساسی ساخت یک سیستم جست‌وجوی کامل را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- یادگیری ماشین برای بهبود رتبه‌بندی - روش‌هایی که گوگل از آن‌ها برای یادگیری از رفتار کاربران و بهبود نتایج جستجو استفاده می‌کند
- ارزیابی سیستم‌ها (دقت، بازیابی، F1) - معیارهایی برای سنجش اینکه آیا موتور جستجوی ما واقعاً خوب کار می‌کند یا نه
- مدل‌های احتمالی - تکنیک‌هایی که به سیستم‌های جستجو اجازه می‌دهند احتمال مرتبط بودن یک سند را تخمین بزنند
- تکنیک‌های پیشرفته بازیابی اطلاعات - روش‌هایی که به سیستم‌های جستجو اجازه می‌دهند مفاهیم پیچیده را درک کنند

آشنا خواهیم شد – که همگی بر همین پایه‌های امتیازدهی و رتبه‌بندی بنا شده‌اند.

End of Chapter Exercises

مسئله 7.1: بازیابی دقیق در برابر غیردقیق (Exact vs. Inexact Top K Retrieval)

در سیستم‌های بازیابی اطلاعات مدرن، چرا اغلب از رویکرد **Inexact Top K** به جای **Exact Top K** استفاده می‌شود؟ دو دلیل اصلی که باعث می‌شود سیستم‌ها کاهش جزئی در دقت را به نفع این روش بپذیرند، بیان کنید.

مسئله 7.2: حذف نمایه با آستانه تکرار (Index Elimination with Document Frequency)

تکنیک حذف نمایه (**Index elimination**) چگونه با نادیده گرفتن کلماتی که **Document Frequency** بالایی دارند (مانند حروف اضافه)، سرعت پردازش پرسمان را افزایش می‌دهد؟

مسئله 7.3: مفهوم و ساختار لیست‌های قهرمان (Champion Lists Concept)

لیست قهرمان (**Champion List**) چیست و چگونه پیش از زمان اجرای پرسمان (**Offline Phase**) ساخته می‌شود؟ تفاوت آن با یک **Inverted Index** معمولی در چیست؟

مسئله 7.4: امتیاز کیفیت ایستا (Static Quality Scores)

دو نمونه از امتیازات کیفیت ایستا (**Static quality scores**) که مستقل از پرسمان کاربر محاسبه می‌شوند را نام ببرید و توضیح دهید چگونه در کنار امتیاز **tf-idf** برای رتبه‌بندی نهایی استفاده می‌شوند.

مسئله 7.5: ترکیب امتیاز ایستا و لیست پستی (Combining Static Score and Postings)

چگونه می‌توان با مرتب‌سازی لیست‌های پستی (**Postings lists**) بر اساس یک امتیاز کیفیت ایستا (مثلاً **PageRank**)، عمل بازیابی **Top-K** را سریع‌تر انجام داد؟

مسئله 7.6: مرتب‌سازی بر اساس تأثیر (Impact Ordering)

در روش مرتب‌سازی بر اساس تأثیر (**Impact Ordering**)، لیست‌های پستی بر چه اساسی مرتب می‌شوند؟ مزیت این کار در برخورد با پرسمان‌های تک‌کلمه‌ای چیست؟

مسئله 7.7: هرس خوشه‌ای مقدماتی (Basic Cluster Pruning)

در الگوریتم هرس خوشه‌ای، نقش «رهبران» (**Leaders**) و «پیروان» (**Followers**) چیست؟ رهبران بر چه اساسی انتخاب می‌شوند؟

مسئله 7.8: نمایه‌های چندلایه (Tiered Indexes)

ساختار نمایه‌های چندلایه (Tiered indexes) را با یک مثال دولایه‌ای توضیح دهید. در چه صورتی سیستم جستجو به لایه دوم مراجعه می‌کند؟

مسئله 7.9: مجاورت عبارات پرسمان (Query-term Proximity)

چرا در موتورهای جستجو، علاوه بر tf-idf، فاصله فیزیکی کلمات پرسمان در داخل متن سند (Proximity) نیز محاسبه می‌شود؟ یک مثال بزنید.

مسئله 7.10: تعامل فضای برداری و عملگرهای پرسمان (Vector Space and Query Operator Interaction)

آیا می‌توان عملگرهای بولین (مانند AND/OR) را با مدل فضای برداری (Vector Space Model) ترکیب کرد؟ توضیح دهید این ترکیب به صورت مفهومی چگونه کار می‌کند.

مسئله 7.11: سناریوی تنظیم آستانه حذف نمایه (Threshold Tuning in Index Elimination)

شما در حال توسعه موتور جستجوی یک کتابخانه پزشکی هستید. در پرسمان "درمان سرطان ریه"، اگر کلمه "درمان" به دلیل تکرار بالا در اسناد توسط Index Elimination حذف شود، نتایج کاملاً تغییر می‌کنند. برای جلوگیری از این مشکل در دامنه‌های تخصصی، چه تغییری در استراتژی حذف نمایه ایجاد می‌کنید؟

مسئله 7.12: سناریوی به‌روزرسانی لیست قهرمان (Champion List Update Scenario)

سایت شما روزانه هزاران مقاله جدید دریافت می‌کند. لیست‌های قهرمان شما هفته‌ای یک بار آپدیت می‌شوند. اگر یک مقاله بسیار مهم و مرتبط امروز منتشر شود، چگونه بدون بازسازی کل لیست‌های قهرمان، تضمین می‌کنید که این مقاله در Top-K کاربران نمایش داده شود؟

مسئله 7.13: مدیریت پرسمان‌های چندکلمه‌ای در مرتب‌سازی (Handling Multi-term Queries)

در سیستم شما لیست‌های پستی بر اساس Impact Ordering (وزن tf) مرتب شده‌اند. کاربر پرسمان سه‌کلمه‌ای وارد می‌کند. از آنجا که ترتیب اسناد در لیست پستی هر کلمه متفاوت است، الگوریتم شما چگونه می‌تواند شرط توقف زودهنگام (Early Termination) را به درستی تشخیص دهد؟

مسئله 7.14: سناریوی افت عملکرد هرس خوشه‌ای (Cluster Pruning Degradation Scenario)

در پایگاه داده شما با 1 میلیون سند، 1000 سند به عنوان رهبر انتخاب شده‌اند. کاربری پرسمانی کاملاً یونیک می‌زند که در هیچ‌یک از بردارهای رهبران کلمات آن وجود ندارد. سیستم هرس خوشه‌ای در این حالت چه خروجی‌ای می‌دهد و برای حل مشکل "از دست رفتن پرسمان‌های نادر" (Rare Queries) چه راهکاری در معماری Cluster Pruning پیشنهاد می‌دهید؟

مسئله 7.15: طراحی لایه‌های نمایه برای فروشگاه (Tiered Index Design for E-commerce)

برای یک فروشگاه آنلاین، باید 3 لایه نمایه طراحی کنید.

چه پارامترهایی (مانند موجودی کالا، امتیاز کاربر، **tf-idf**) را برای تخصیص یک محصول به لایه 1، 2 یا 3 انتخاب می‌کنید؟
منطق خود را شرح دهید.

مسئله 7.16: وزن‌دهی به مجاورت در خبرخوان (Proximity Weighting in News Reader)

در یک اپلیکیشن خبری، کاربری عبارت "حمله سایبری بانک" را جستجو می‌کند.

یک تابع امتیازدهی طراحی کنید که علاوه بر **Cosine Similarity**، امتیاز ویژه‌ای برای اسنادی که این سه کلمه در آن‌ها در یک پنجره 5 کلمه‌ای قرار دارند، در نظر بگیرد.

مسئله 7.17: طراحی تابع امتیازدهی ترکیبی (Designing Combined Scoring Function)

شما باید تابعی بنویسید که امتیاز نهایی سند را بر اساس 3 فاکتور مشخص کند: شباهت کسینوسی (امتیاز بین 0 و 1)، امتیاز پیچ‌رنک (امتیاز بین 0 و 1) و تاریخ انتشار سند.

فرمول ریاضی یا منطق ترکیب این سه فاکتور را چگونه طراحی می‌کنید که اسناد قدیمی اما بسیار معتبر، اسناد جدید بی‌ربط را مغلوب کنند؟

مسئله 7.18: گلوگاه‌های سیستم بازیابی یکپارچه (Bottlenecks in Putting it All Together)

با توجه به بخش 7.2.4، شما تمام کامپوننت‌ها (**Champion lists**، **Tiered index**، **Index elimination**) را در یک سیستم پیاده کرده‌اید.

اگر حافظه **RAM** سیستم شما ناگهان پر شود، کدام یک از این ساختارهای داده را ابتدا به هارد دیسک (**Disk**) منتقل می‌کنید که کمترین تأثیر را در زمان پاسخگویی (**Latency**) داشته باشد؟ چرا؟

مسئله 7.19: فیلتر بولین قبل از رتبه‌بندی برداری (Boolean Filter before Vector Ranking)

کاربر جستجو کرده: **(Apple OR Samsung) AND Laptop**

معماری پایپ‌لاین شما چگونه باید باشد تا ابتدا با سرعت بالا فیلتر بولین اعمال شود و سپس فقط برای اسناد باقیمانده، ضرب نقطه‌ای (**Dot Product**) بردارهای پرس‌وجو و سند محاسبه گردد؟

مسئله 7.20: بالانس بین سرعت و دقت (Speed vs. Accuracy Trade-off)

مدیر محصول از شما می‌خواهد که زمان پاسخ‌دهی موتور جستجو را به نصف کاهش دهید، حتی اگر 5٪ از دقت نتایج کاسته شود.

از بین تکنیک‌های فصل 7، کدام دو تکنیک را برای رسیدن به این هدف با شدت بیشتری اعمال می‌کنید و عوارض جانبی آن‌ها را چگونه کنترل می‌کنید؟

مسئله 7.21: الگوریتم پویا برای حدنصاب لیست قهرمان (Dynamic Thresholding for Champion Lists) ★

ثابت در نظر گرفتن اندازه لیست قهرمان (مثلاً همیشه ۵۰ سند) برای پرس‌مان‌های کوتاه و بلند بهینه نیست.

الگوریتمی طراحی کنید که اندازه **r** (تعداد اسناد در لیست قهرمان هر کلمه) را به صورت پویا و بر اساس متغیر **idf** آن کلمه در

زمان ساخت نمایه (Index construction time) تعیین کند. سود و زیان این روش را تحلیل کنید.

مسئله 7.22: طراحی پایپ‌لاین هرس خوشه‌ای دو مرحله‌ای (Two-Stage Cluster Pruning Pipeline Design)



در یک مجموعه داده عظیم با 1 میلیارد سند، انتخاب $\sqrt{N}$ رهبر باعث تولید حدود 31000 رهبر می‌شود که مقایسه پرسمان با همه آن‌ها باز هم کند است.

یک معماری «هرس خوشه‌ای دو مرحله‌ای (Two-level Cluster Pruning)» طراحی کنید. دقیقاً مشخص کنید در هر مرحله چند گره بررسی می‌شود و پیچیدگی زمانی (Time Complexity) محاسبه شباهت در این مدل جدید را اثبات کنید.

مسئله 7.23: پیاده‌سازی کد نمایه چندلایه (Tiered Index Code Implementation)



فرض کنید یک دیکشنری در پایتون دارید که Inverted Index لایه اول است و دیکشنری دیگری لایه دوم است.

یک تابع پایتون (شبه‌کد دقیق) بنویسید که پرسمانی شامل آرایه‌ای از کلمات کلیدی و عدد K دریافت کند. تابع باید ابتدا سعی کند K سند مشترک (Intersection) در لایه اول پیدا کند. اگر تعداد مشترکات کمتر از K بود، تابع باید بهینه‌ترین روش را برای جستجو در لایه دوم به کار گیرد تا ظرفیت K تکمیل شود (بدون اینکه پردازش‌های لایه اول دور ریخته شود).

مسئله 7.24: چالش مرتب‌سازی تأثیر در حافظه محدود (Impact Ordering Challenge in Limited Memory)



می‌خواهید Impact Ordering را پیاده کنید اما نمی‌توانید وزن دقیق tf-idf (که اعشاری است) را برای تکتک اسناد در حافظه کش ذخیره کنید.

چگونه با استفاده از تکنیک Quantization (کوانتیزه‌کردن وزن‌ها به بازه‌های صحیح مثلاً 1 تا 5)، بلوک‌هایی از لیست‌های پستی می‌سازید؟ الگوریتم پیمایش شما روی این بلوک‌های کوانتیزه‌شده برای پرسمان‌های چندکلمه‌ای چگونه کار خواهد کرد؟

مسئله 7.25: سناریوی برخورد توابع امتیازدهی سفارشی (Custom Scoring Functions Collision Scenario)



سیستم بازیابی شما از یک تابع امتیازدهی مبتنی بر یادگیری ماشین (Machine-learned scoring) استفاده می‌کند که هم متغیرهای برداری و هم متغیرهای کیفیت ایستا را می‌گیرد.

کاربری گزارش داده که با اضافه شدن یک پارامتر جدید (مثلاً "تعداد کلیک کاربران قبلی")، اسناد بسیار کوتاه اما پرکلیک (Clickbait) جایگزین مقالات علمی و طولانی در صدر نتایج شده‌اند. تابع امتیازدهی فعلی را چگونه اصلاح و نرمال‌سازی می‌کنید که اثر طول سند (Document Length) و کلیک‌ها با هم در تعادل باشند؟

مسئله 7.26: بهینه‌سازی محاسبات مجاورت عبارات (Optimizing Term Proximity Computations)



یافتن کوچکترین پنجره‌ای از کلمات که شامل تمام عبارات پرسمان باشد (Smallest Window) در پرسمان‌های طولانی از نظر محاسباتی بسیار سنگین است.

الگوریتمی با استفاده از نشانگرها (Pointers) طراحی کنید که با داشتن لیست موقعیت (Positional Index) سه کلمه

پرسمان در یک سند، با پیچیدگی زمانی $O(L)$ کوچکترین پنجره مجاورت را پیدا کند (L مجموع طول لیست‌های موقعیت این سه کلمه در سند است).

مسئله 7.27: حل مشکل هرس خوشه‌ای در داده‌های نامتوازن (Solving Cluster Pruning in Unbalanced Data)

در **Cluster Pruning**، متوجه می‌شوید که به دلیل ماهیت داده‌ها، 80 درصد پیروان (**Followers**) تنها به 5 درصد از رهبران (**Leaders**) متصل شده‌اند و بقیه رهبران خوشه‌های بسیار کوچکی دارند. این عدم توازن چگونه باعث از بین رفتن مزیت سرعت الگوریتم می‌شود؟ چه مکانیزم ریاضی برای تخصیص مجدد رهبران یا محدود کردن سایز خوشه‌ها پیشنهاد می‌دهید که همزمان تعادل برقرار شود و اسناد به رهبران نامرتبط وصل نشوند؟

مسئله 7.28: معماری یکپارچه برای پرس‌وجوهای پیچیده (Unified Architecture for Complex Queries)

باید سیستمی بسازید که پرسمان پیچیده روبه‌رو را در کمتر از 100 میلی‌ثانیه پاسخ دهد:

"machine learning" AND (python OR R) + date>2023

بلوک دیاگرام دقیق یا نمودار جریان داده (**Data Flow**) سیستم را رسم و تشریح کنید. دقیقاً مشخص کنید در کدام مرحله از **Index elimination** استفاده می‌کنید، فیلتر بولین در چه مرحله‌ای روی **Inverted Index** اعمال می‌شود و تابع امتیازدهی نهایی روی چه زیرمجموعه‌ای از داده‌ها اجرا می‌گردد.

مسئله 7.29: تحلیل پیچیدگی زمانی ترکیب روش‌ها (Time Complexity Analysis of Combined Methods)

فرض کنید سیستمی همزمان از «حذف نمایه با **IDF** آستانه‌دار» و «لیست‌های قهرمان با طول ثابت r » استفاده می‌کند. اثبات کنید در بدترین حالت (**Worst-case scenario**) برای یک پرسمان با q کلمه کلیدی، سقف عملیات محاسباتی برای یافتن **Top-K** سند، به چه معادله‌ای تقلیل می‌یابد؟ تفاوت این پیچیدگی را با حالت جستجوی خطی در فضای برداری استاندارد (**Standard VSM**) به صورت ریاضی مقایسه کنید.

مسئله 7.30: شبیه‌سازی سیستم رتبه‌بندی نهایی (Final Ranking System Simulation)

شما مهندس ارشد بازیابی اطلاعات هستید.

یک ماژول کامل تحت عنوان **Scoring_Engine** را با شبکه‌کد طراحی کنید که سه ورودی می‌گیرد: **query_terms**، **docs_metadata** (شامل **PageRank**) و **postings_lists** (شامل موقعیت کلمات برای **proximity**).

این ماژول باید یک پایپ‌لاین 3 مرحله‌ای داشته باشد:

(1) غربالگری اولیه با تقاطع بولین و لیست‌های قهرمان.

(2) محاسبه **Net_Score** (ترکیب کسینوسی و پیچ‌رنک).

(3) پاداش مجاورت (**Proximity Boost**) تنها برای اسنادی که در مرحله قبل جزو 30 سند برتر بوده‌اند.

خروجی باید لیست نهایی **Top-10** مرتب‌شده باشد. هدف از این طراحی، اثبات توانایی مدیریت مصرف **CPU** در محاسبات سنگین مجاورت است.

مسئله 7.31: معماری مرتب‌سازی تأثیر در تجارت الکترونیک (Impact Ordering Architecture in E-commerce) ★

شما معمار ارشد یک فروشگاه اینترنتی هستید. برای پاسخگویی به پرسمان‌ها زیر 50 میلی‌ثانیه، تصمیم به استفاده از «مرتب‌سازی بر اساس تأثیر (Impact ordering)» در لیست‌های پستی (Postings lists) می‌گیرید. امتیاز هر کالا ترکیبی از tf-idf و «امتیاز فروش کالا (Static Score)» است. اولاً، دقیقاً چگونه لیست‌های پستی را بر اساس این دو معیار مرتب می‌کنید تا در زمان جستجو کارآمد باشد؟ دوماً، هنگام پردازش یک پرسمان دو کلمه‌ای (مثلاً "لپ‌تاپ گیمینگ")، شرط توقف زودهنگام (Early termination) الگوریتم شما دقیقاً چه خواهد بود تا تضمین کنید هیچ سند مهمی را از دست نداده‌اید، اما کل لیست را هم پیمایش نکرده‌اید؟

مسئله 7.32: معماری خط لوله حافظه‌محور در مقیاس بزرگ (Memory-Constrained Pipeline Architecture at Scale) ★

شما محدودیت شدید حافظه اصلی (RAM) دارید اما باید یک موتور جستجو برای 1 میلیارد وب‌سایت بسازید که Top-10 را برگرداند.

یک پایپ‌لاین معماری کامل طراحی کنید که در آن به ترتیب از تکنیک‌های زیر استفاده شده باشد:

1. حذف نمایه (Index elimination)

2. لیست‌های قهرمان (Champion lists)

3. توابع امتیازدهی تقریب‌زننده (Approximated scoring).

جریان داده (Data flow) را از لحظه ورود پرسمان کاربر تا خروج 10 لینک برتر به صورت بلوک دیاگرام یا مراحل ترتیبی شرح دهید و ثابت کنید که طراحی شما حافظه RAM را بهینه مصرف می‌کند.

مسئله 7.33: پیاده‌سازی نمایه چندلایه در پایتون (Tiered Index Implementation in Python) ★

پروژه پایتون، شما دیکشنری عظیمی از Inverted Index در اختیار دارید.

شبه‌کد یا ساختار کدی بنویسید که مراحل زیر را انجام دهد:

الف) در مرحله پیش‌پردازش، با دریافت لیست پستی هر کلمه و آرایه امتیازات پیچ‌رنک، یک ساختار Tiered Index دو لایه‌ای (لایه اول: Champion List شامل اسنادی با tf بالا و پیچ‌رنک بالا، لایه دوم: بقیه اسناد) بسازد.

ب) تابعی بنویسید که در زمان اجرای پرسمان (Query time)، ابتدا فقط در لایه اول جستجو کند؛ اگر تعداد نتایج مشترک بین کلمات پرسمان به حدنصاب K نرسید، الگوریتم به طور هوشمندانه لایه دوم را تنها برای تکمیل ظرفیت بازگشایی کند.

مسئله 7.34: تحلیل پیچیدگی هرس خوشه‌ای سلسله‌مراتبی (Hierarchical Cluster Pruning Complexity Analysis) ★

الگوریتم هرس خوشه‌ای (Cluster Pruning) را در نظر بگیرید که در آن به جای یک بار انتخاب تصادفی، اسناد را در یک ساختار درختی (سلسله‌مراتبی) خوشه‌بندی کرده‌ایم.

اگر N تعداد کل اسناد باشد، اثبات کنید که در زمان پردازش پرسمان، جستجو در این ساختار سلسله‌مراتبی چگونه از مرتبه زمانی $O(N)$ در جستجوی خطی، به چه مرتبه زمانی جدیدی کاهش می‌یابد؟ در طول این اثبات، گلوگاه‌های احتمالی (مانند انتخاب نادرست رهبر در ریشه درخت) را تحلیل کرده و یک راهکار برای دور زدن این گلوگاه ارائه دهید.

پاسخنامه مسئله 7.1: بازیابی دقیق در برابر غیردقیق (Exact vs. Inexact Top K)

بخش 1) مقدمه و تعریف مفاهیم پایه

در سیستم‌های بازیابی اطلاعات، هدف اصلی یافتن **K** سند برتر (**Top K**) است که بیشترین میزان ارتباط را با پرسمان (**Query**) کاربر دارند. پیش از بررسی دلایل، ابتدا باید تفاوت این دو رویکرد را به روشنی درک کنیم:

رویکرد دقیق (Exact Top K): در این روش، سیستم امتیاز (**Score**) تمام اسنادی که حداقل شامل یکی از واژگان پرسمان هستند را محاسبه می‌کند. سپس همه این اسناد را بر اساس امتیاز مرتب کرده و دقیقاً **K** سندی که بالاترین امتیاز ریاضی را دارند، به عنوان خروجی باز می‌گرداند.

رویکرد غیردقیق یا تقریبی (Inexact Top K): در این روش، سیستم از تکنیک‌های ابتکاری و میان‌بر (مانند لیست‌های قهرمان یا **Champion Lists**) و حذف نمایه‌ها یا **Index Elimination** استفاده می‌کند. هدف این است که بدون محاسبه امتیاز برای تک‌تک اسناد، مجموعه‌ای متشکل از **K** سند استخراج شود که به احتمال بسیار بالا، همان اسناد برتر هستند.

بخش 2) تحلیل مسئله و دلایل ترجیح رویکرد غیردقیق

موتورهای جستجوی مدرن و سیستم‌های مقیاس‌پذیر، همواره رویکرد غیردقیق را ترجیح می‌دهند و می‌پذیرند که خروجی سیستم ممکن است کاملاً منطبق بر خروجی مدل **Exact** نباشد. دو دلیل اصلی و کلیدی برای این معماری به شرح زیر است:

1. کارایی محاسباتی، کاهش تأخیر (**Latency**) و مدیریت منابع:

تشریح مشکل: در مجموعه‌های دادگان عظیم (مانند شبکه جهانی وب)، یک پرسمان ساده ممکن است در میلیون‌ها سند وجود داشته باشد. محاسبه دقیق معیارهایی مانند شباهت کسینوسی (**Cosine Similarity**) یا **BM25** برای تک‌تک این میلیون‌ها سند و متعاقباً مرتب‌سازی (**Sorting**) آن‌ها، بار پردازشی (**CPU**) و مصرف حافظه (**RAM**) بسیار بالایی به سیستم تحمیل می‌کند.

راهکار و مزیت: کاربران سیستم‌های بازیابی اطلاعات انتظار دارند نتایج جستجو در کسری از ثانیه (**Sub-second**) به آن‌ها نمایش داده شود. رویکرد **Inexact** با هرس کردن فضای جستجو و ارزیابی نکردن اسنادی که شانس کمی برای قرار گرفتن در بین **K** سند برتر دارند، زمان پاسخ‌گویی را به شکل چشمگیری کاهش داده و هزینه‌های زیرساختی و پردازشی سرورها را بهینه می‌کند.

2. ماهیت تخمینی توابع امتیازدهی و مفهوم «ارتباط» (**Relevance**):

تشریح مشکل: باید توجه داشت که فرمول‌های وزن‌دهی مانند **Tf-Idf**، تنها یک مدل‌سازی آماری و تقریب ریاضی از مفهوم پیچیده و معنایی «ارتباط» هستند. به بیان علمی، سندی که بالاترین امتیاز ریاضی را در مدل **Exact** کسب می‌کند، لزوماً از دیدگاه شناختی کاربر نهایی، مرتبط‌ترین سند نیست. سیستم امتیازدهی در ذات خود دارای خطای معنایی است.

راهکار و مزیت: از آنجایی که خود سیستم پایه‌ای امتیازدهی یک تقریب (**Approximation**) است، صرف هزینه گزاف پردازشی برای یافتن دقیق‌ترین جواب ریاضی در یک مدل تقریبی، ارزش افزوده‌ای ایجاد نمی‌کند. تفاوت بین اسنادی که توسط مدل **Inexact** بازیابی می‌شوند با اسناد مدل **Exact**، از نظر کاربر نهایی عموماً غیرقابل تشخیص است و تأثیری در افت کیفیت تجربه کاربری (**User Experience**) ندارد.

بخش 3) نتیجه‌گیری نهایی

در طراحی و مهندسی سیستم‌های بازیابی اطلاعات، همواره یک موازنه (**Trade-off**) اساسی میان «کارایی و سرعت» و «دقت مطلق ریاضی» وجود دارد.

رویکرد **Inexact Top K** یک اصل مهم مهندسی را آموزش می‌دهد: در مقیاس‌های کلان، رسیدن به یک جواب "به اندازه کافی خوب" و مرتبط" در زمان بسیار کوتاه، به مراتب ارزشمندتر از یافتن جواب "مطلقاً دقیق ریاضی" در زمانی طولانی و غیرقابل قبول است.

پاسخنامه مسئله 7.2: حذف نمایه مبتنی بر فراوانی سند

(Index Elimination with Document Frequency)

بخش 1) مقدمه و ارتباط فراوانی سند با ارزش اطلاعاتی واژگان

در مباحث بنیادین بازیابی اطلاعات، فراوانی سند یا **df** (**Document Frequency**) بیانگر تعداد اسنادی در مجموعه دادگان است که یک واژه خاص حداقل یک بار در آن‌ها ظاهر شده است. کلماتی که دارای **df** بسیار بالایی هستند (مانند حروف اضافه، ربط و ضمایر که معمولاً در دسته کلمات توقف یا **Stop Words** قرار می‌گیرند)، تقریباً در تمام اسناد حضور دارند. بر اساس قانون زیپف (**Zipf's Law**) و مفهوم فراوانی معکوس سند (**idf**)، قدرت تمایزدهندگی (**Discriminative Power**) این کلمات برای یافتن اسناد مرتبط، بسیار ناچیز و نزدیک به صفر است.

بخش 2) مراحل و مکانیزم تکنیک حذف نمایه

گام اول: تحلیل ریاضیاتی ارزش کلمات

محاسبه وزن هر واژه برای رتبه‌بندی اسناد، معمولاً بر پایه مدل‌هایی مانند **TF-IDF** انجام می‌شود. در این مدل، ارزش **idf** از طریق فرمول زیر محاسبه می‌گردد (که در آن **N** برابر با کل اسناد موجود در مجموعه است):

$$idf = \log \left(\frac{N}{df} \right)$$

برای واژگانی با **df** بالا (واژگانی که در اکثر اسناد وجود دارند)، کسر **N/df** به عدد 1 نزدیک شده و در نتیجه، لگاریتم آن و مقدار نهایی **idf** به صفر میل می‌کند.

گام دوم: اعمال حذف نمایه (Index Elimination)

از آنجا که سهم این کلمات در امتیاز نهایی (**Score**) اسناد از نظر ریاضی قابل اغماض است، سیستم بازیابی اطلاعات در زمان پردازش پرسمان (**Query Processing**)، به صورت آگاهانه از واکشی و پردازش این کلمات صرف‌نظر می‌کند. به عبارت دیگر، اگر پرسمان شامل ترکیب کلماتی با **df** بالا و پایین باشد، سیستم تنها بر روی کلماتی با **df** پایین‌تر و ارزش معنایی بیشتر متمرکز می‌شود.

بخش 3) تاثیر مستقیم بر افزایش سرعت پردازش

نادیده گرفتن این کلمات با **df** بالا، از دو طریق اصلی موجب بهینه‌سازی و افزایش سرعت پردازش می‌شود:

1. کاهش شدید عملیات ورودی/خروجی (Disk I/O):

در ساختار نمایه معکوس (**Inverted Index**)، لیست‌های رکورد (**Posting Lists**) مربوط به کلمات پربسامد، طولانی‌ترین لیست‌های موجود هستند. خواندن و انتقال این حجم عظیم از داده‌ها از حافظه دیسک به حافظه اصلی (**RAM**) گلوگاه اصلی سیستم است. با حذف این واژگان، زمان واکشی داده‌ها به شکل چشمگیری کاهش می‌یابد.

2. کاهش بار محاسباتی پردازنده (CPU Cycles):

برای محاسبه امتیاز اسناد، سیستم باید عملیات اشتراک‌گیری (**Intersection**) یا اجتماع را روی لیست‌های رکورد انجام دهد. پیمایش لیست‌های طولانی نیازمند محاسبات سنگین است. با کنار گذاشتن این کلمات، تعداد اسناد نامزدی (**Candidate Documents**) که پردازنده باید امتیاز شباهت کسینوسی (**Cosine Similarity**) آن‌ها را محاسبه و سپس مرتب‌سازی (**Sorting**) کند، به شدت افت می‌کند.

بخش 4) نتیجه‌گیری نهایی

تکنیک حذف نمایه مبتنی بر **df** یک رویکرد کاملاً منطقی در معماری موتورهای جستجو است. این روش با حذف بخش عمده‌ای از بار پردازشی بی‌ارزش، سرعت پاسخ‌گویی سیستم را به صورت نمایی ارتقا می‌دهد، بدون آنکه افت محسوسی در کیفیت و دقت مجموعه اسناد برتر (**Top K**) ایجاد نماید.

پاسخنامه مسئله 7.3: مفهوم و ساختار لیست‌های قهرمان (Champion Lists)

بخش 1) مقدمه و تعریف پایه

در سیستم‌های بازیابی اطلاعات، «لیست قهرمان» (که در برخی منابع با نام‌های **Fancy List** یا **Top Docs** نیز شناخته می‌شود)، یک ساختار داده کمکی و بهینه‌سازی شده است. این لیست شامل مجموعه کوچکی از اسناد است که برای یک واژه خاص، بالاترین امتیاز (وزن) را دارند. هدف اصلی از ایجاد لیست قهرمان، تسریع فرآیند پاسخ‌گویی به پرسمان در رویکرد بازیابی غیردقیق (**Inexact Top K**) است، تا سیستم مجبور نباشد در زمان اجرای پرسمان، تمام اسناد دربردارنده یک واژه را ارزیابی کند.

بخش 2) مراحل ساخت لیست قهرمان پیش از زمان اجرا (Offline Phase)

یکی از ویژگی‌های کلیدی لیست‌های قهرمان این است که بار محاسباتی آن‌ها به فاز پیش‌پردازش (زمانی که سیستم در حال نمایه‌سازی اسناد است و هنوز پرسمانی از سمت کاربر ارسال نشده) منتقل می‌شود. ساخت این لیست‌ها گام‌به‌گام به شرح زیر است:

گام اول (محاسبه اوزان): سیستم برای هر واژه (**t**) در واژه‌نامه (**Dictionary**)، امتیاز یا وزن آن واژه را در تمام اسنادی که شامل آن هستند، محاسبه می‌کند. این وزن معمولاً بر اساس معیار **TF-IDF** یا صرفاً فراوانی واژه در سند (**tf**) به دست می‌آید.

گام دوم (مرتب‌سازی کاهشی): لیست رکوردها (**Posting List**) مربوط به آن واژه، بر اساس امتیازهای محاسبه‌شده در گام اول، به صورت نزولی (از بیشترین امتیاز به کمترین) مرتب می‌شود.

گام سوم (برش و انتخاب): سیستم یک حد آستانه به نام r تعیین می‌کند. سپس، تنها r سند اول لیست مرتب‌شده (که دارای بالاترین امتیازها هستند) را انتخاب می‌کند.

گام چهارم (ذخیره‌سازی): این r سند برتر به عنوان «لیست قهرمان» برای آن واژه خاص، در کنار لیست اصلی (یا به عنوان یک لایه دسترسی سریع‌تر) در حافظه ذخیره می‌شوند. مقدار r باید به گونه‌ای انتخاب شود که همواره $r > K$ باشد (که در آن K تعداد اسناد نهایی مورد انتظار کاربر است).

بخش 3) تفاوت‌های بنیادین لیست قهرمان با نمایه معکوس معمولی (Standard Inverted Index)

درک تفاوت این دو ساختار برای تسلط بر معماری موتورهای جستجو ضروری است. این تفاوت‌ها در دو بعد اصلی بررسی می‌شوند:

1. حجم و جامعیت داده‌ها (Data Coverage):

نمایه معکوس معمولی: یک ساختار جامع است. لیست رکوردهای یک واژه در نمایه معمولی، شامل شناسه (**DocID**) تمامی اسنادی است که آن واژه در آن‌ها حداقل یک‌بار ظاهر شده است.

لیست قهرمان: یک ساختار گزینشی و هرس‌شده است. این لیست تنها شامل r سند برتر است و از نگهداری اسنادی که اهمیت کمتری دارند، چشم‌پوشی می‌کند.

2. معیار مرتب‌سازی (Sorting Criterion):

نمایه معکوس معمولی: در لیست‌های رکورد استاندارد، شناسه‌ها معمولاً بر اساس **DocID** (به صورت صعودی) مرتب می‌شوند. این نوع مرتب‌سازی برای انجام کارآمد عملیات اشتراک‌گیری (**Intersection**) در پرسman‌های بولی (**Boolean Queries**) با استفاده از الگوریتم‌های ادغام، کاملاً ضروری است.

لیست قهرمان: در این لیست، شناسه‌ها بر اساس امتیاز یا وزن (**Score/Weight**) مرتب می‌شوند. این معماری به سیستم اجازه می‌دهد در زمان اجرای پرسman کاربر (**Online Phase**)، به سرعت به مرتبط‌ترین اسناد دسترسی پیدا کند و نیازی به پیمایش کل لیست برای یافتن اسناد دارای امتیاز بالا نداشته باشد.

بخش 4) نتیجه‌گیری نهایی

استفاده از لیست‌های قهرمان، یک تکنیک مهندسی هوشمندانه برای انتقال هزینه محاسباتی از زمان پاسخ‌گویی آنلاین (**Online**) به زمان پردازش آفلاین (**Offline**) است. با این روش، سیستم در زمان دریافت پرسman ابتدا به سراغ لیست‌های قهرمان واژگان می‌رود و تنها در صورتی که تعداد اسناد یافت شده برای رسیدن به K سند برتر کافی نباشد، به لیست‌های رکورد طولانی و استاندارد مراجعه می‌کند.

پاسخنامه مسئله 7.4: امتیاز کیفیت ایستا (Static Quality Scores)

بخش 1) مقدمه و مفهوم کیفیت ایستا

در سیستم‌های بازیابی اطلاعات پیشرفته، رتبه‌بندی نهایی اسناد تنها بر اساس میزان تطابق متنی آن‌ها با پرسman (مانند امتیاز **TF-IDF**) انجام نمی‌شود. اسناد دارای ویژگی‌های ذاتی و مستقلی هستند که ارزش، اعتبار و کیفیت کلی آن‌ها را نشان می‌دهد. به امتیازی که بر اساس این ویژگی‌های ذاتی و کاملاً مستقل از پرسman کاربر (**Query-Independent**) محاسبه می‌شود، «امتیاز

کیفیت ایستا» (Static Quality Score) می‌گویند. این امتیاز معمولاً با نماد **g(d)** نمایش داده می‌شود و در فاز پیش‌پردازش (Offline) محاسبه و ذخیره می‌گردد.

بخش 2) دو نمونه بارز از امتیازات کیفیت ایستا

1. امتیاز پیج‌رنک (PageRank):

تعریف و منطق: در شبکه جهانی وب، پیج‌رنک یکی از مشهورترین معیارهای کیفیت ایستا است. این الگوریتم بر اساس ساختار گراف وب کار می‌کند و فرض آن بر این است که اگر صفحات وب معتبر و مهم به یک صفحه خاص لینک داده باشند (**In-links**)، آن صفحه نیز دارای ارزش و اعتبار بالایی است.

کاربرد: مستقل از اینکه کاربر چه کلمه‌ای را جستجو کرده است، صفحه‌ای که پیج‌رنک بالاتری دارد، به عنوان یک منبع معتبرتر شناخته شده و امتیاز ایستای بالاتری دریافت می‌کند.

2. تعداد ارجاعات (Citation Count) یا تازگی سند (Document Freshness):

تعریف و منطق: در پایگاه‌های داده علمی و آکادمیک، تعداد دفعاتی که به یک مقاله ارجاع (**Citation**) داده شده است، نشان‌دهنده کیفیت و مرجع بودن آن سند است. همچنین در سیستم‌های بازیابی اخبار، «عمر سند» یا تازگی آن یک معیار ایستای حیاتی است؛ خبری که دقایقی پیش منتشر شده، ذاتاً ارزش بیشتری نسبت به خبر سال گذشته دارد.

کاربرد: این معیارها بدون نیاز به دانستن پرسمان، یک عدد ثابت به عنوان کیفیت پایه برای هر مقاله یا خبر تولید می‌کنند.

بخش 3) نحوه ادغام امتیاز ایستا با امتیاز TF-IDF در رتبه‌بندی نهایی

برای استخراج لیست نهایی اسناد برتر، سیستم باید امتیاز کیفی ایستا (اعتبار ذاتی سند) را با امتیاز پویای متنی (میزان ارتباط با پرسمان، مانند **TF-IDF**) ترکیب کند. هدف، رسیدن به یک امتیاز نهایی یا **NetScore** است.

بخش 4) ساختار ریاضی ادغام

متداول‌ترین و استانداردترین رویکرد ریاضی برای این ادغام، استفاده از ترکیب خطی (**Linear Combination**) است. اگر امتیاز ایستای سند را **g(d)** و امتیاز تطابق متنی آن با پرسمان **q** را **Score(q, d)** در نظر بگیریم (که معمولاً بر پایه **TF-IDF** محاسبه می‌شود)، فرمول رتبه‌بندی نهایی به شکل زیر خواهد بود:

$$\text{NetScore}(q, d) = \alpha \cdot g(d) + (1 - \alpha) \cdot \text{Score}(q, d)$$

پارامتر α : یک ضریب تنظیم‌کننده (**Tuning Parameter**) بین بازه **[0, 1]** است که وزن و اهمیت کیفیت ایستا را در برابر ارتباط متنی کنترل می‌کند.

تابع Score: در مدل فضای برداری، این بخش همان مجموع وزن‌های **TF-IDF** برای واژگان مشترک پرسمان و سند است:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf-idf}_{t,d}$$

بخش 5) نتیجه‌گیری محاسباتی

از آنجایی که **g(d)** یک مقدار ثابت است و از پیش در سیستم محاسبه شده، موتور جستجو در زمان اجرای پرسمان **(Online Phase)** تنها کافی است امتیاز **TF-IDF** را محاسبه کرده و سپس آن را با مقدار آماده‌ی **g(d)** جمع کند. این معماری تضمین می‌کند که اسناد بازیابی‌شده، هم دارای بالاترین ارتباط موضوعی با نیاز کاربر هستند و هم از معتبرترین منابع موجود استخراج شده‌اند، در حالی که سربار محاسباتی جدیدی نیز به زمان پاسخ‌گویی اضافه نمی‌شود.

پاسخنامه مسئله 7.5: ترکیب امتیاز ایستا و لیست‌های رکورد

(Combining Static Score and Postings)

بخش 1) مقدمه و تغییر پارادایم در ساختار نمایه

در معماری سنتی موتورهای جستجو، لیست‌های رکورد (**Postings Lists**) در نمایه معکوس بر اساس شناسه سند (**DocID**) به صورت صعودی مرتب می‌شوند. این نوع مرتب‌سازی برای اجرای سریع عملیات اشتراک‌گیری (**Intersection**) حیاتی است. اما زمانی که هدف ما صرفاً یافتن **K** سند برتر (**Top K**) بر اساس ترکیبی از ارتباط متنی و اعتبار ذاتی سند است، تغییر معیار مرتب‌سازی به یک «امتیاز کیفیت ایستا» (مانند **PageRank**)، استراتژی بسیار قدرتمندی برای بهینه‌سازی سرعت ارائه می‌دهد.

بخش 2) مراحل و مکانیزم افزایش سرعت بازیابی

گام اول: مرتب‌سازی نزولی در فاز آفلاین (Offline Sorting)

به جای استفاده از **DocID**، سیستم در زمان ساخت نمایه، لیست رکورد مربوط به هر واژه را بر اساس امتیاز کیفیت ایستای اسناد، یعنی **g(d)**، به صورت نزولی (**Descending**) مرتب می‌کند. در این ساختار، اولین شناسه‌ای که در لیست رکورد یک واژه ظاهر می‌شود، متعلق به سندی است که فارغ از محتوایش، بالاترین اعتبار جهانی (مثلاً بالاترین پیج‌رنک) را در میان تمام اسناد حاوی آن واژه دارد. هر چه در طول لیست به سمت جلو حرکت کنیم، با اسنادی با کیفیت ذاتی پایین‌تر روبرو می‌شویم.

گام دوم: پردازش پرسمان و پیمایش لیست‌ها (Online Processing)

زمانی که یک پرسمان چندکلمه‌ای (**Multi-term Query**) دریافت می‌شود، سیستم پوینترهایی (**Pointers**) را در ابتدای لیست رکوردهای مربوط به واژگان پرسمان قرار می‌دهد و شروع به خواندن آن‌ها می‌کند. به دلیل مرتب‌سازی نزولی، سیستم از همان ابتدا با معتبرترین و باکیفیت‌ترین اسناد (اسنادی با بالاترین **g(d)**) مواجه می‌شود.

گام سوم: شرط توقف زودهنگام (Early Termination)

کلید اصلی افزایش سرعت در این روش، امکان «توقف زودهنگام» است. سیستم نیازی ندارد کل لیست‌های رکورد را تا انتها پیمایش کند. فرآیند به این شکل پیش می‌رود:

1. سیستم در حین پیمایش لیست‌ها، یک صف اولویت (Priority Queue) یا (Min-Heap) با ظرفیت K تشکیل می‌دهد تا K سند برترِ فعلی را نگهداری کند. فرض کنید کمترین امتیازِ نهایی در این لیستِ K تایی، برابر با S_{min} باشد.

2. با حرکت در طول لیست رکورد، مقدار $g(d)$ برای اسنادِ بررسی‌نشده به طور پیوسته کاهش می‌یابد.

3. سیستم به طور مستمر یک کران بالا (Upper Bound) برای بالاترین امتیازِ ممکن که اسنادِ باقی‌مانده در لیست می‌توانند کسب کنند، محاسبه می‌کند. اگر فرض کنیم بیشترین امتیازِ متنی ممکن برابر با $MaxScore(q, d)$ باشد، بیشترین امتیازِ نهایی برای اسنادِ ندیده برابر است با:

4. **نقطه توقف:** به محض اینکه مقدار **Max Possible Score** از S_{min} کوچکتر شود، از نظر ریاضی غیرممکن است که اسنادِ باقی‌مانده در انتهای لیست بتوانند امتیازی بالاتر از K سند برترِ فعلی کسب کنند. در این نقطه، سیستم پیمایش را متوقف کرده و نتیجه را بازمی‌گرداند.

بخش 3) نتیجه‌گیری و دستاوردهای عملکردی

با ادغام امتیاز ایستا در ساختار لیست‌های پستی و بهره‌گیری از تکنیک توقف زود هنگام، موتور جستجو موفق می‌شود بدون پردازشِ کاملِ داده‌ها، دقیقاً همان خروجی‌ای را تولید کند که در صورت پردازشِ کلِ لیست به دست می‌آمد. این رویکرد به معنای کاهشِ چشمگیرِ عملیات خواندن از دیسک (Disk I/O) و صرفه‌جویی عظیم در چرخه‌های پردازشی (CPU Cycles) است که در نهایت منجر به بازیابیِ درنگ‌تایم (Real-time) و بسیار سریع برای پرسمان‌های پیچیده می‌گردد.

پاسخنامه مسئله 7.6: مرتب‌سازی بر اساس تأثیر (Impact Ordering)

بخش 1) مقدمه و مفهوم مرتب‌سازی بر اساس تأثیر

در ساختار استاندارد نمایه معکوس (Inverted Index)، رکوردهای موجود در لیست پستی (Postings List) بر اساس شناسه سند (DocID) به صورت صعودی مرتب می‌شوند تا عملیات اشتراک‌گیری (Intersection) تسهیل گردد. اما در تکنیک «مرتب‌سازی بر اساس تأثیر» (Impact Ordering)، معیار مرتب‌سازی تغییر بنیادینی می‌کند و فاکتور اصلی، میزان اهمیت، وزن یا "تأثیر" یک واژه در یک سند خاص است.

بخش 2) مبای مرتب‌سازی لیست‌های پستی

در این روش، رکوردهای مربوط به هر واژه (t) در لیست پستی، بر اساس وزن آن واژه در سند (که عموماً همان فراوانی واژه یا $tf_{t,d}$ و یا وزن نهایی $tf-idf_{t,d}$ است) به صورت نزولی (Descending) مرتب می‌شوند.

از منظر ریاضی، اگر لیست پستی واژه t شامل اسناد $d_1, d_2, \dots, d_n$ باشد و وزن واژه در سند را با $wt_{t,d}$ نشان دهیم، ترتیب قرارگیری آن‌ها در لیست به گونه‌ای است که رابطه زیر همواره برقرار باشد:

$$wt_{t,d_1} \geq wt_{t,d_2} \geq \dots \geq wt_{t,d_n}$$

در این ساختار معماری، سندی که واژه t در آن بیشترین تأثیر و بالاترین وزن ریاضی را دارد، دقیقاً در ابتدای لیست (Head of the list) قرار می‌گیرد. با پیمایش به سمت انتهای لیست (Tail)، میزان اهمیت واژه در اسناد به طور یکنواخت کاهش

می‌یابد. (در برخی پیاده‌سازی‌های صنعتی، به جای مرتب‌سازی مطلق، از گروه‌بندی یا بلوک‌بندی رکوردهایی با وزن یکسان استفاده می‌شود).

بخش 3) مزیت کلیدی در برخورد با پرسمان‌های تک‌کلمه‌ای (Single-term Queries)

هنگامی که کاربر یک پرسمان متشکل از تنها یک کلمه ($q = \{t\}$) را به سیستم ارسال می‌کند، مرتب‌سازی بر اساس تأثیر بالاترین سطح از کارایی مهندسی را به نمایش می‌گذارد. مزایای این امر در دو گام تحلیلی زیر خلاصه می‌شود:

1. تطابق کامل امتیاز نهایی با وزن تک‌واژه:

در پرسمان‌های تک‌کلمه‌ای، نیازی به جمع زدن وزن چندین کلمه نیست. بنابراین، امتیاز نهایی و میزان ارتباط هر سند با پرسمان، دقیقاً برابر با همان وزن آن تک‌واژه در سند است:

$$\text{Score}(q, d) = wt_{t,d}$$

2. واکنشی مستقیم با مرتبه زمانی $O(K)$ و توقف فوری:

از آنجایی که هدف نهایی موتور جستجو یافتن **K** سند برتر (**Top K**) است و لیست پستی این واژه از پیش (در فاز **Offline**) بر اساس امتیاز $wt_{t,d}$ به صورت نزولی مرتب شده است، سیستم به هیچ‌گونه محاسبه ریاضی، ارزیابی تک‌تک اسناد یا مرتب‌سازی (**Sorting**) در زمان اجرا نیاز ندارد.

موتور جستجو به سادگی **K** رکورد اول را از ابتدای لیست می‌خواند و بی‌درنگ عملیات خواندن را متوقف می‌کند (**Early Termination**). در این حالت، به جای پیمایش کل لیست که نیازمند مرتبه زمانی $O(N)$ است (که در آن **N** طول کل لیست پستی است)، عملیات تنها با مرتبه زمانی $O(K)$ انجام می‌پذیرد.

بخش 4) نتیجه‌گیری نهایی

مرتب‌سازی بر اساس تأثیر، با تبدیل یک مسئله پیچیده «محاسبات و مرتب‌سازی در زمان اجرا» به یک عمل بسیار ساده‌ی «خواندن متوالی از حافظه»، پرسمان‌های تک‌کلمه‌ای را در بهینه‌ترین حالت ممکن پاسخ می‌دهد. این رویکرد از هدررفت چرخه‌های پردازنده جلوگیری کرده و زمان پاسخ‌گویی (**Latency**) را برای این دسته از پرسمان‌ها به حداقل ممکن کاهش می‌دهد.

پاسخنامه مسئله 7.7: هرس خوشه‌ای مقدماتی (Basic Cluster Pruning)

بخش 1) مقدمه و ضرورت استفاده از هرس خوشه‌ای

در مدل فضای برداری (**Vector Space Model**)، محاسبه شباهت کسینوسی میان پرسمان کاربر و تک‌تک اسناد موجود در مجموعه‌های کلان (با حجم **N** سند)، از نظر پردازشی به شدت پرهزینه است و به مرتبه زمانی $O(N)$ نیاز دارد. الگوریتم «هرس خوشه‌ای» (**Cluster Pruning**) یک تکنیک تقریب‌زنی ساختاریافته است که با دسته‌بندی اسناد پیش از زمان اجرا، فضای جستجو را به شدت محدود کرده و نیاز به پیمایش کل اسناد را از بین می‌برد.

در این معماری، مجموعه اسناد به دو گروه «رهبران» و «پیروان» تقسیم می‌شوند:

بخش 2) مکانیزم انتخاب و نقش رهبران (Leaders)

معیار انتخاب: در نسخه مقدماتی (**Basic**) این الگوریتم، اگر مجموعه داده شامل **N** سند باشد، سیستم دقیقاً $\sqrt{N}$ سند را به صورت کاملاً تصادفی (**Random**) به عنوان «رهبر» انتخاب می‌کند. استفاده از انتخاب تصادفی یک استراتژی هوشمندانه و سریع است که تضمین می‌کند (با احتمال بالا) رهبران در سراسر فضای برداری اسناد پراکنده شوند، بدون آنکه نیازی به اجرای الگوریتم‌های سنگین خوشه‌بندی مانند **K-Means** باشد.

نقش رهبران: رهبران در واقع نمایندگان (**Representatives**) هر خوشه در فضای برداری هستند. نقش آن‌ها ایجاد یک لایه فیلترینگ اولیه در زمان اجرای پرسمان است. هنگامی که یک پرسمان دریافت می‌شود، سیستم ابتدا امتیاز شباهت کسینوسی پرسمان را فقط با این $\sqrt{N}$ رهبر محاسبه می‌کند تا نزدیک‌ترین رهبر (یا رهبران) به پرسمان مشخص شود.

بخش 3) مکانیزم اتصال و نقش پیروان (Followers)

تعریف و اتصال: سایر اسناد موجود در مجموعه (یعنی $N - \sqrt{N}$ سند باقی‌مانده) به عنوان «پیرو» شناخته می‌شوند. در فاز پیش‌پردازش (**Offline Phase**)، سیستم شباهت هر سند پیرو را با تمام رهبران محاسبه کرده و آن پیرو را به نزدیک‌ترین رهبر (رهبری که بیشترین شباهت کسینوسی را با آن دارد) متصل می‌کند.

نقش پیروان: پیروان اعضای تشکیل‌دهنده خوشه‌ها هستند. هر رهبر به همراه پیروان متصل به خود، یک خوشه (**Cluster**) را تشکیل می‌دهد.

بخش 4) فرآیند هرس و تاثیر آن در زمان اجرا (Online Processing)

زمانی که پرسمان جدیدی وارد سیستم می‌شود، الگوریتم مراحل زیر را طی می‌کند:

1. مقایسه پرسمان با تمام $\sqrt{N}$ رهبر و یافتن شبیه‌ترین رهبر به پرسمان.
2. **مرحله هرس:** سیستم به جای بررسی تمام اسناد مجموعه، سایر خوشه‌ها را به طور کامل هرس (حذف) کرده و شباهت پرسمان را تنها با پیروان متعلق به همان رهبر منتخب محاسبه می‌کند. فرض بر این است که اگر پرسمان به یک رهبر شبیه است، به احتمال قوی به پیروان آن رهبر نیز شباهت دارد.

بخش 5) نتیجه‌گیری ریاضی و عملکردی

با اجرای این معماری، تعداد محاسبات شباهت کسینوسی از **N** محاسبه (در حالت عادی) به حدود $2\sqrt{N}$ محاسبه (متشکل از $\sqrt{N}$ محاسبه برای یافتن رهبر و به طور متوسط $\sqrt{N}$ محاسبه برای بررسی پیروان آن رهبر) کاهش می‌یابد. این تقلیل مرتبه زمانی از $O(N)$ به $O(\sqrt{N})$ ، سرعت پاسخ‌گویی موتور جستجو را به شکلی خیره‌کننده افزایش می‌دهد، در حالی که دقت بازیابی (**Top K**) همچنان در سطح قابل قبولی باقی می‌ماند.

پاسخنامه مسئله 7.8: نمایه‌های چندلایه (Tiered Indexes)

بخش 1 مقدمه و معماری نمایه‌های چندلایه

در سیستم‌های بازیابی اطلاعات مقیاس‌پذیر، «نمایه‌های چندلایه» یا لایه‌بندی‌شده (**Tiered Indexes**) تعمیمی بر مفهوم لیست‌های قهرمان (**Champion Lists**) هستند. در این معماری، به جای داشتن تنها یک لیست اصلی و یک لیست فشرده، لیست پستی (**Postings List**) هر واژه به چندین لایه یا سطح متوالی شکسته می‌شود. این لایه‌بندی بر اساس یک آستانه (**Threshold**) مشخص، مانند فراوانی واژه در سند (**tf**)، امتیاز **tf-idf**، یا یک امتیاز کیفیت ایستا (مانند **PageRank**) صورت می‌پذیرد. هدف از این ساختار، پاسخ‌گویی به پرسمان در سریع‌ترین و بالاترین لایه ممکن و اجتناب از پردازش داده‌های کم‌اهمیت است.

بخش 2) تحلیل ساختار با یک مثال دولایه‌ای (Two-Tier Architecture)

برای درک بهتر، یک معماری ساده متشکل از دو لایه را برای واژه‌ای مانند "الگوریتم" در نظر می‌گیریم و معیار لایه‌بندی را فراوانی واژه (**tf**) قرار می‌دهیم:

لایه اول (Tier 1) لایه طلایی یا بااهمیت بالا:

این لایه تنها شامل اسنادی است که واژه "الگوریتم" در آن‌ها حضور بسیار پررنگی دارد. برای مثال، سیستم شرط $tf > 20$ را برای این لایه تنظیم می‌کند. بنابراین، لایه اول صرفاً شناسه اسنادی را در خود جای می‌دهد که کلمه مورد نظر بیش از 20 بار در آن‌ها تکرار شده باشد. این لایه بسیار کوچک، فشرده و در زمان اجرا به سرعت قابل بارگذاری در حافظه اصلی است.

لایه دوم (Tier 2) لایه استاندارد یا تکمیلی:

این لایه دربرگیرنده باقی‌مانده اسنادی است که واژه مورد نظر در آن‌ها وجود دارد، اما اهمیت کمتری نسبت به لایه اول دارند. در مثال ما، اسنادی با شرط $0 < tf \leq 20$ در این لایه قرار می‌گیرند. حجم این لایه بسیار بزرگ‌تر از لایه اول است.

در زمان نمایه‌سازی (**Offline Phase**)، این لایه‌ها به صورت مجزا در ساختار نمایه ذخیره می‌شوند.

بخش 3) شرط مراجعه سیستم به لایه دوم (Fallback Condition)

هنگامی که یک پرسمان از سوی کاربر دریافت می‌شود، موتور جستجو فرآیند بازیابی را استراتژیک و از لایه اول آغاز می‌کند. هدف سیستم یافتن **K** سند برتر (**Top K**) است. مراجعه به لایه دوم (یا لایه‌های پایین‌تر در نمایه‌های چندلایه) تنها تحت شرایط زیر رخ می‌دهد:

1. کفایت نکردن نتایج در لایه اول (Insufficient Candidates):

پس از آنکه سیستم عملیات بازیابی (و در صورت نیاز، اشتراک‌گیری بین لایه‌های اول و ژگان پرسمان) را انجام داد، تعداد اسناد یافت شده را می‌شمارد. اگر تعداد اسناد مرتبط یافت‌شده در لایه اول، کمتر از تعداد نتایج مورد نیاز کاربر (یعنی کمتر از **K**) باشد، سیستم با کمبود رکورد مواجه شده است.

2. سقوط به لایه پایین‌تر (Dropping Down a Tier):

تنها در صورتی که شرط فوق برقرار باشد، سیستم برای جبران کمبود اسناد و رسیدن به سقف **K** سند، فرآیند جستجو را در لایه دوم از سر می‌گیرد. در این حالت، سیستم لایه دوم و ژگان را واکشی کرده و پردازش را تا یافتن تعداد مطلوب نتایج ادامه می‌دهد.

بخش 4) نتیجه‌گیری نهایی

معماری نمایه‌های چندلایه بر پایه اصل «صرفه‌جویی در منابع» بنا شده است. از آنجایی که اکثر پرسمان‌های کاربران با همان اسنادِ باکیفیتِ موجود در لایه اول (**Tier 1**) به طور کامل پاسخ داده می‌شوند (یعنی شرط $K \geq$ برآورده می‌شود)، موتور جستجو در بیشتر مواقع هرگز نیازی به لمس کردن و پردازش لایه‌های عظیم دوم و سوم نخواهد داشت. این مکانیزم تضمین می‌کند که بار ورودی/خروجی دیسک (**Disk I/O**) بهینه بماند و تأخیر در پاسخ‌گویی به حداقل ممکن برسد.

پاسخنامه مسئله 7.9: مجاورت عبارات پرسمان (Query-term Proximity)

بخش 1) مقدمه و بررسی محدودیت مدل‌های مبتنی بر فراوانی

مدل‌های کلاسیک بازیابی اطلاعات، مانند مدل فضای برداری (**Vector Space Model**) و معیار وزن‌دهی **tf-idf**، بر پایه فرض «کیسه کلمات» (**Bag of Words**) استوار هستند. در این معماری، یک سند صرفاً به عنوان مجموعه‌ای از کلمات مستقل و درهم‌ریخته در نظر گرفته می‌شود و سیستم تنها به «حضور» و «فراوانی» یک واژه اهمیت می‌دهد، نه به «محل قرارگیری» یا ترتیب آن‌ها در جمله. این محدودیت باعث می‌شود که ساختار نحوی و روابط معنایی بین کلمات در متن نادیده گرفته شود.

بخش 2) دلیل محاسبه مجاورت فیزیکی (Proximity)

در پرسمان‌های چندکلمه‌ای (**Multi-term Queries**)، هدف کاربر عموماً یافتن یک مفهوم واحد، یک نام خاص یا رابطه مستقیم بین آن کلمات است. محاسبه «مجاورت فیزیکی» (که به معنای اندازه‌گیری تعداد کلمات حائل بین کلمات پرسمان در داخل متن سند است) بر این اصل منطقی و زبان‌شناختی استوار است: **کلماتی که در متن به یکدیگر نزدیک‌ترند، با احتمال بسیار بالاتری به یک مفهوم معنایی مشترک اشاره دارند و مرتبط با نیت کاربر هستند.**

موتورهای جستجوی مدرن برای غلبه بر خطای معنایی مدل کیسه کلمات، علاوه بر محاسبه **tf-idf**، فاصله مکانی کلمات را نیز از طریق ساختار «نمایه‌های موقعیتی» (**Positional Indexes**) استخراج کرده و از آن به عنوان یک ضریب تقویت‌کننده (**Boost**) برای بهبود رتبه‌بندی استفاده می‌کنند.

بخش 3) مثال تحلیلی و کاربردی

فرض کنید کاربری پرسمان دو کلمه‌ای "بازیابی اطلاعات" (**Information Retrieval**) را جستجو کرده است. موتور جستجو دو سند زیر را در پایگاه داده خود می‌یابد:

سند الف (مجاورت کم - فاصله فیزیکی زیاد): "مدیریت پایگاه‌های اطلاعات در سازمان‌های بزرگ بسیار مهم است.

همچنین داشتن سیستم‌های پشتیبان برای بازیابی داده‌های از دست رفته در زمان بحران کاملاً ضروری است."

سند ب (مجاورت بالا - کلمات متصل): "درس بازیابی اطلاعات یکی از دروس پایه و حیاتی در مقطع کارشناسی

مهندسی کامپیوتر است."

تحلیل خروجی سیستم: اگر رتبه‌بندی صرفاً بر اساس مدل **tf-idf** باشد، ممکن است سند «الف» به دلیل طولانی‌تر بودن یا تکرار مجدد یکی از کلمات، امتیاز ریاضی بالاتری کسب کند. اما از دیدگاه معنایی و نیاز کاربر، سند «ب» دقیقاً به همان مفهوم تخصصی اشاره دارد و مرتبط‌تر است.

سیستم مبتنی بر مجاورت، با محاسبه «پنجره کلمات»، متوجه می‌شود که در سند «ب» این دو کلمه دقیقاً مجاور یکدیگرند (فاصله صفر یا 1)، در حالی که در سند «الف» چندین کلمه بی‌ربط بین آن‌ها فاصله انداخته است. در نتیجه، الگوریتم به سند

«ب» امتیاز مجاورت بسیار بالاتری تخصیص داده و جایگاه آن را در لیست نتایج (Top K) ارتقا می‌دهد.

بخش 4) نتیجه‌گیری ساختاری

در معماری نهایی سیستم‌های جستجو، الگوریتم‌های رتبه‌بندی یک امتیاز مجاورت (Proximity Score) را محاسبه می‌کنند. این امتیاز معمولاً بر اساس عرضِ کوچکترین پنجره متنی (Smallest Window) که تمام کلمات پرسمان را در بر می‌گیرد، به دست می‌آید. سپس این امتیاز از طریق یک تابع ترکیب خطی، با امتیاز متنی (tf-idf) و امتیاز کیفی ایستا (مانند PageRank) جمع می‌گردد. اعمال این ضریب، دقت (Precision) سیستم را به شدت افزایش داده و مانع از نمایش اسنادی می‌شود که کلمات پرسمان در آن‌ها به صورت تصادفی و پراکنده توزیع شده‌اند.

پاسخنامه مسئله 7.10: تعامل فضای برداری و عملگرهای پرسمان

(Vector Space and Query Operator Interaction)

بخش 1) مقدمه

بله، ترکیب عملگرهای منطقی بولی (مانند AND, OR, NOT) با مدل فضای برداری (Vector Space Model - VSM) نه تنها امکان‌پذیر است، بلکه شالوده اصلی بسیاری از موتورهای جستجوی مدرن و سیستم‌های پایگاه داده متنی را تشکیل می‌دهد. در حالی که مدل بولی صرفاً بر اساس «حضور یا عدم حضور» واژگان تصمیم‌گیری می‌کند (خروجی باینری) و مدل فضای برداری بر اساس «میزان ارتباط متنی» رتبه‌بندی انجام می‌دهد، ادغام این دو معماری می‌تواند دقت (Precision) و کارایی سیستم را به طور همزمان بهینه کند.

بخش 2) رویکردهای ترکیب مدل بولی و فضای برداری

این تعامل به صورت مفهومی از طریق 2 رویکرد متفاوت مهندسی و ریاضیاتی پیاده‌سازی می‌شود:

بخش 3) رویکرد اول: معماری دو مرحله‌ای (Two-Stage Architecture) - فیلتر و رتبه‌بندی

این روش رایج‌ترین و عملیاتی‌ترین شیوه در صنعت بازیابی اطلاعات است. فرآیند در این ساختار به صورت گام‌به‌گام زیر انجام می‌شود:

گام اول (فیلتر بولی محدودکننده): در این مرحله، پرسمان کاربر به عنوان یک عبارت بولی اکید پردازش می‌شود. سیستم با استفاده از نمایه معکوس (Inverted Index) و عملگرهای منطقی (مانند عملیات اشتراک‌گیری برای عملگر AND)، زیرمجموعه‌ای از اسناد را استخراج می‌کند که دقیقاً با شروط بولی مطابقت دارند. در این گام هیچ‌گونه محاسبه ریاضی برای رتبه‌بندی صورت نمی‌گیرد.

گام دوم (رتبه‌بندی برداری): پس از فیلترینگ اولیه، سیستم اکنون یک مجموعه به مراتب کوچک‌تر از اسناد نامزد (Candidate Documents) در اختیار دارد. در این مرحله، مدل فضای برداری وارد عمل شده و وزن‌دهی tf-idf و شباهت کسینوسی (Cosine Similarity) را صرفاً برای همین زیرمجموعه محاسبه می‌کند.

خروجی نهایی: اسنادی بازگردانده می‌شوند که شروط مطلق بولی را ارضا کرده‌اند، اما بر اساس میزان ارتباط موضوعی (VSM Score) به صورت نزولی مرتب شده‌اند.

بخش 4) رویکرد دوم: مدل بولی توسعه یافته (Extended Boolean Model)

این رویکرد که توسط جرارد سالتون (**Gerard Salton**) پایه گذاری شد، یک مدل یکپارچه و پیشرفته ریاضی است که منطق صلب بولی را با وزن دهی فضای برداری در هم می آمیزد.

مفهوم ریاضی (p-norm): در این مدل، عملگرهای بولی دیگر به صورت صفر و 1 مطلق ارزیابی نمی شوند، بلکه منطقی «نرم» (**Soft Boolean**) پیدا می کنند. اسناد بر اساس فاصله هندسی آن ها (با استفاده از نرم های L_p) تا نقاط ایده آل در فضای برداری ارزیابی می گردند.

تحلیل عملگرها:

در یک پرسمان دارای عملگر **OR** (مانند $t_1 \vee t_2$)، سیستم فاصله بردار سند را از نقطه مبدأ (**0, 0**) می سنجد. هر چه وزن یکی از واژگان (یا هر دو) در سند بیشتر باشد، فاصله از مبدأ بیشتر شده و سند امتیاز بالاتری کسب می کند.

در پرسمان دارای عملگر **AND** (مانند $t_1 \wedge t_2$)، سیستم فاصله سند را از نقطه ایده آل (**1, 1**) (نقطه ای که هر دو واژه دارای بیشترین وزن ممکن هستند) محاسبه می کند. سندی که هر دو واژه را با بالاترین وزن داشته باشد، کمترین فاصله را تا نقطه ایده آل خواهد داشت و در صدر قرار می گیرد.

بخش 5) نتیجه گیری ساختاری

ترکیب این دو مکانیزم، یک پارادایم تحلیلی و قدرتمند ایجاد می کند. در این ساختار ترکیبی، عملگرهای بولی نقش یک «فیلترکننده صلب» را ایفا می کنند که به کاربر (یا مهندس سیستم) اجازه می دهند دایره جستجو را با قاطعیت محدود کنند؛ در موازات آن، مدل فضای برداری نقش «رتبه بندی کننده منعطف» را بر عهده می گیرد تا اطمینان حاصل شود که در میان نتایج فیلتر شده، مرتبط ترین اسناد در بالاترین جایگاه لیست خروجی (**Top K**) به کاربر نمایش داده می شوند.

پاسخنامه مسئله 7.11: سناریوی تنظیم آستانه حذف نمایه در دامنه های تخصصی

(Threshold Tuning in Index Elimination)

بخش 1) مقدمه و تحلیل ریشه مشکل (Root Cause Analysis)

در سیستم های بازیابی اطلاعات عمومی (مانند جستجوی وب)، واژگانی که فراوانی سند (**df**) بسیار بالایی دارند، معمولاً فاقد ارزش معنایی و تمایزدهندگی هستند. اما در یک پایگاه داده تخصصی (مانند کتابخانه پزشکی)، واژگانی نظیر «درمان»، «بیماری» یا «تشخیص»، با وجود داشتن **df** بسیار بالا و حضور در اکثر اسناد، حامل «نیت اصلی کاربر» (**User Intent**) هستند.

اگر سیستم با یک رویکرد ساده انگارانه و صرفاً بر اساس آستانه ریاضی **df** این واژگان را حذف کند (**Index Elimination**)، پرسمان «درمان سرطان ریه» به «سرطان ریه» تقلیل می یابد. در این حالت، سیستم به جای بازیابی پروتکل های درمانی، ممکن است مقالاتی درباره پاتولوژی یا آمار شیوع این سرطان را به عنوان نتایج برتر بازگرداند که به معنای شکست کامل سیستم در درک نیاز کاربر است.

بخش 2) استراتژی های اصلاحی و راهکارهای مهندسی (Solutions)

برای جلوگیری از این فاجعه معنایی در دامنه های تخصصی، باید از استراتژی های ترکیبی و هوشمندانه زیر استفاده کرد:

1. غیرفعال‌سازی حذف نمایه بر اساس طول پرسمان (Query-Length Dependent Elimination):

تشریح منطق: کاربرانی که پرسمان‌های کوتاه (**Short Queries**) وارد می‌کنند (مثلاً 2 تا 4 کلمه)، تک‌تک کلمات را با دقت و با هدف خاصی انتخاب کرده‌اند. در یک پرسمان 3 کلمه‌ای، حذف حتی یک کلمه، 33% از بار معنایی را از بین می‌برد.

اقدام عملی: سیستم باید به گونه‌ای پیکربندی شود که تکنیک حذف نمایه، فقط برای پرسمان‌های طولانی (مثلاً بیشتر از 5 کلمه) فعال گردد. برای پرسمان «درمان سرطان ریه» (3 کلمه)، سیستم هیچ کلمه‌ای را فارغ از میزان **df** آن حذف نمی‌کند.

2. لیست سفید دامنه-محور (Domain-Specific Whitelisting) و فرهنگ لغات تخصصی:

تشریح منطق: آستانه‌های ریاضی صرف، برای دامنه‌های تخصصی پاسخگو نیستند. سیستم نیازمند دخالت دانش بشری (**Domain Expertise**) است.

اقدام عملی: پیش از اجرای الگوریتم‌های هرس، یک «لیست سفید» (**Whitelist**) از واژگان کلیدی پزشکی (مانند درمان، دارو، جراحی، عوارض) تهیه می‌شود. سیستم موظف است این کلمات را از الگوریتم **Index Elimination** مستثنی کند، حتی اگر فراوانی معکوس سند (**idf**) آن‌ها بسیار پایین باشد.

3. رویکرد حذف نرم با نمایه‌های چندلایه (Soft Elimination via Tiered Indexes):

تشریح منطق: به جای حذف قطعی (**Hard Dropping**) واژه «درمان» پیش از اجرای جستجو، سیستم از یک معماری چندلایه استفاده می‌کند.

اقدام عملی:

- مرحله اول:** موتور جستجو ابتدا تمام کلمات پرسمان (از جمله واژه پریسامد «درمان») را با استفاده از عملگر **AND** در لایه اول پردازش می‌کند.
- مرحله دوم (Fallback):** تنها در صورتی که تعداد اسناد یافت‌شده شامل هر 3 کلمه بسیار کم باشد، سیستم به صورت خودکار به لایه پایین‌تر رفته و شرط وجود کلمه «درمان» را آزاد می‌کند. با این روش، اولویت همواره با اسنادی است که دقیقاً به «درمان» پرداخته‌اند.

4. تحلیل عبارات به جای کلمات منفرد (Phrase Recognition):

تشریح منطق: تغییر واحد پردازش از «تک‌کلمه» به «مفهوم» (**Concept**).

اقدام عملی: موتور جستجو با استفاده از تکنیک‌های پردازش زبان طبیعی (**NLP**)، عبارت «سرطان ریه» را به عنوان یک موجودیت واحد (**Single Entity**) شناسایی می‌کند. در این حالت، سیستم به جای جستجوی مستقل 3 کلمه، پرسمان را به صورت «سرطان ریه» **AND** «درمان» پردازش کرده که این امر وابستگی به محاسبه وزن کلمات منفرد را به شدت کاهش می‌دهد.

بخش 3 نتیجه‌گیری ساختاری

طراحی سیستم بازبازی اطلاعات برای دامنه‌های تخصصی (**Vertical Search Engines**) نیازمند تغییر پارادایم از «آمار محض» به «معناشناسی کاربردی» است. در چنین سیستم‌هایی، استراتژی‌های بهینه‌سازی سرعت (مانند **Index Elimination**) نباید هرگز به

قیمت از بین رفتن یکپارچگی معنایی پرسمان و افت کیفیت دقت (**Precision**) تمام شوند. استفاده از آستانه‌های پویای مبتنی بر طول پرسمان و نمایه‌های لایه‌بندی‌شده، بهترین نقطه تعادل میان سرعت و دقت را در این سناریوها فراهم می‌کنند.

پاسخنامه مسئله 7.12: سناریوی به‌روزرسانی لیست قهرمان (Champion List Update Scenario)

بخش 1) مقدمه و تشریح چالش معماری

در موتورهای جستجوی مقیاس‌پذیر، ساختار نمایه (**Index**) به دلیل حجم عظیم داده‌ها عموماً ایستا (**Static**) است و به‌روزرسانی ساختارهایی مانند «لیست قهرمان» (**Champion List**) نیازمند پردازش‌های سنگین در فاز آفلاین (**Offline**) است که معمولاً به صورت دوره‌ای (مثلاً هفتگی) انجام می‌شود. چالش اصلی این است که اگر یک سند بسیار بارز (مانند یک خبر فوری یا مقاله کلیدی) پیش از زمان بازسازی نمایه‌ها به سیستم اضافه شود، در لیست‌های قهرمان فعلی حضور نخواهد داشت. برای رفع این مشکل بدون نیاز به بازسازی کل نمایه، از معماری «نمایه‌سازی پویا و دوگانه» (**Dynamic/Dual Index Architecture**) استفاده می‌شود.

بخش 2) مراحل و مکانیزم تضمین حضور مقالات جدید در نتایج (Top K)

گام اول: ایجاد نمایه کمکی در حافظه اصلی (In-Memory Auxiliary Index)

به جای تزریق مستقیم مقالات جدید به نمایه اصلی (**Main Index**) که فرآیندی پرهزینه است، سیستم یک نمایه کوچک، موقت و بسیار سریع در حافظه اصلی (**RAM**) ایجاد می‌کند. تمامی هزاران مقاله‌ای که روزانه وارد سیستم می‌شوند، بلافاصله در این «نمایه کمکی پویا» نمایه‌سازی می‌شوند. از آنجایی که حجم این نمایه نسبتاً کوچک است، سیستم نیازی به ساخت لیست قهرمان برای آن ندارد و می‌تواند لیست‌های رکورد (**Postings Lists**) آن را در زمان اجرا به طور کامل پردازش کند.

گام دوم: پردازش موازی پرسمان (Concurrent Query Processing)

هنگامی که کاربری یک پرسمان ارسال می‌کند، موتور جستجو به صورت همزمان (**Parallel**) دو درخواست مجزا ارسال می‌کند:

1. درخواست به نمایه اصلی: برای واکشی اسناد از طریق «لیست‌های قهرمان» (مربوط به مقالات قدیمی‌تر).

2. درخواست به نمایه کمکی: برای واکشی اسناد از طریق پیمایش کامل لیست‌های رکورد موقت (مربوط به

مقالات جدید و تازه‌نشریافته).

گام سوم: اعمال فاکتور تازگی به عنوان امتیاز ایستا (Freshness Boost)

برای تضمین اینکه مقاله جدید و مهم در رقابت با مقالات قدیمی‌تر شکست نخورد، سیستم یک «امتیاز کیفیت ایستا» (**Static Quality Score**) مبتنی بر زمان به آن اختصاص می‌دهد. مقالاتی که به تازگی منتشر شده‌اند، یک ضریب تقویت (**Boost**) در امتیاز **tf-idf** خود دریافت می‌کنند. این ضریب باعث می‌شود که مقاله جدید به دلیل ارزش زمانی و اهمیت موضوعی بالا، امتیاز ریاضی قابل توجهی کسب کند.

گام چهارم: ادغام نتایج و استخراج لیست نهایی (Merging and Top K Selection)

پس از آنکه هر دو نمایه اسناد نامزد (**Candidate Documents**) خود را همراه با امتیازاتشان بازگردانند، سیستم نتایج را در یک صف اولویت (**Min-Heap**) با ظرفیت **K** ادغام می‌کند. به دلیل آنکه مقاله جدید ما دارای ارتباط متنی بالا و

همچنین ضریب تازگی (**Freshness Boost**) است، امتیاز نهایی آن به راحتی از حد آستانه اسناد قدیمی موجود در لیست قهرمان عبور کرده و جایگاه خود را در میان **K** سند برتر تثبیت می‌کند.

بخش 3) چرخه حیات نمایه و نتیجه‌گیری

در پایان هفته و در زمان مقرر آفلاین، سیستم عملیات «ادغام نمایه‌ها» (**Index Merging**) را انجام می‌دهد. در این فرآیند، اطلاعات نمایه کمکی به طور کامل به نمایه اصلی منتقل شده، لیست‌های قهرمان جدید بر اساس ترکیب کل داده‌ها بازسازی می‌شوند و نمایه کمکی خالی می‌گردد تا آماده دریافت مقالات هفته آینده باشد.

این معماری یک راهکار مهندسی ظریف است که موازنه بی‌نقصی میان «کارایی سیستم» (با استفاده از لیست‌های قهرمان ایستا) و «به‌روز بودن نتایج» (با استفاده از نمایه کمکی پویا) ایجاد می‌کند و تضمین می‌نماید که کاربران هرگز اطلاعات حیاتی و جدید را از دست ندهند.

پاسخنامه مسئله 7.13: مدیریت پرسمان‌های چندکلمه‌ای در مرتب‌سازی بر اساس تأثیر

(Handling Multi-term Queries in Impact Ordering)

بخش 1) مقدمه و چالش مسئله

در تکنیک «مرتب‌سازی بر اساس تأثیر» (**Impact Ordering**)، لیست‌های پستی به جای مرتب‌سازی صعودی بر اساس شناسه سند (**DocID**)، به صورت نزولی بر اساس وزن واژه (مانند **tf** یا **tf-idf**) مرتب می‌شوند. در پرسمان‌های تک‌کلمه‌ای، سیستم به سادگی **K** سند اول را واکنشی کرده و متوقف می‌شود. اما در پرسمان‌های چندکلمه‌ای (مانند یک پرسمان **3** کلمه‌ای)، چالش اساسی این است که یک سند خاص ممکن است در لیست پستی کلمه اول در صدر قرار داشته باشد، اما در لیست کلمه دوم در انتها باشد و در لیست کلمه سوم اصلاً حضور نداشته باشد.

بنابراین، برای اعمال «توقف زودهنگام» (**Early Termination**) بدون از دست دادن دقت، سیستم باید از رویکردی مبتنی بر محاسبه کران بالا (**Upper Bound**) و الگوریتم‌های آستانه (مانند **Fagin's Threshold Algorithm** یا الگوریتم **WAND**) استفاده کند.

بخش 2) مراحل الگوریتم و مکانیزم تشخیص توقف زودهنگام

سیستم جستجو برای مدیریت این سناریو مراحل ریاضی و منطقی زیر را طی می‌کند:

1. تشکیل صف اولویت برای نتایج فعلی (**Top-K Heap**):

سیستم یک صف اولویت (**Min-Heap**) با ظرفیت **K** ایجاد می‌کند تا **K** سند برتری که تا این لحظه پردازش شده‌اند را در خود نگه دارد. کمترین امتیاز نهایی در میان این **K** سند، به عنوان «حد آستانه» یا S_{min} شناخته می‌شود. هر سند جدیدی که امتیازی بالاتر از S_{min} کسب کند، وارد صف شده و S_{min} به‌روزرسانی (افزایش) می‌یابد.

2. پیمایش موازی لیست‌ها و محاسبه کران بالا (**Upper Bound**):

سیستم برای هر یک از سه واژه پرسمان، یک اشاره‌گر (**Pointer**) در ابتدای لیست پستی مربوط به آن واژه قرار می‌دهد و شروع به پیمایش می‌کند.

به دلیل مرتب‌سازی نزولی لیست‌ها، وزن‌هایی که در زیر هر اشاره‌گر قرار دارند، **بیشترین وزن ممکن** برای اسناد باقی‌مانده در آن لیست هستند. سیستم به طور پیوسته مجموع وزن‌های فعلی در زیر اشاره‌گرهای هر سه لیست را محاسبه کرده و آن را کران بالا (**UB**) می‌نامد:

$$UB = \sum_i wt(t_i, ptr_i)$$

در این رابطه، $wt(t_i, ptr_i)$ وزن واژه **i** در سندی است که اشاره‌گر در حال حاضر روی آن قرار دارد. از آنجا که لیست‌ها به صورت نزولی مرتب شده‌اند، با پیشروی اشاره‌گرها، مقدار **UB** به طور یکنواخت و پیوسته کاهش می‌یابد.

3. ارزیابی شرط توقف زود هنگام (Early Termination Check):

کلید حل مسئله در این گام نهفته است. سیستم در هر مرحله از پیمایش، مقدار کران بالای اسناد ندیده (**UB**) را با کمترین امتیاز موجود در صف اولویت (S_{min}) مقایسه می‌کند.

شرط توقف: به محض اینکه نامعادله زیر برقرار شود، سیستم عملیات پیمایش را بلافاصله متوقف می‌کند:

$$UB \leq S_{min}$$

بخش 3) منطق ریاضی پشت شرط توقف

زمانی که **UB** از S_{min} کوچکتر یا مساوی شود، از نظر ریاضی **کاملاً غیرممکن** است که هیچ‌یک از اسناد پردازش‌نشده (که در ادامه لیست‌ها قرار دارند)، حتی اگر در هر سه لیست به طور همزمان حضور داشته باشند، بتوانند مجموع امتیازی بالاتر از S_{min} کسب کنند. بنابراین، ارزیابی ادامه لیست‌ها هیچ تغییری در **K** سند برتر موجود در صف اولویت ایجاد نخواهد کرد.

بخش 4) نتیجه‌گیری نهایی

با استفاده از تکنیک محاسبه مداوم کران بالا (**UB**) و مقایسه آن با آستانه پویای نتایج فعلی (S_{min})، موتور جستجو می‌تواند در لیست‌های مبتنی بر **Impact Ordering** نیز توقف زود هنگام را با دقت **100%** اجرا کند. این معماری تضمین می‌کند که سیستم ضمن حفظ سلامت و دقت ریاضی در یافتن اسناد برتر (**Exact Top K**)، از پردازش بیهوده بخش عظیمی از لیست‌های پستی جلوگیری کرده و کارایی را به حداکثر می‌رساند.

پاسخنامه مسئله 7.14: سناریوی افت عملکرد هرس خوشه‌ای

(Cluster Pruning Degradation Scenario)

بخش 1) مقدمه و تحلیل خروجی سیستم در وضعیت بحران (System Output Analysis)

در معماری هرس خوشه‌ای (**Cluster Pruning**)، پایه و اساس یافتن اسناد مرتبط، محاسبه شباهت کسینوسی (**Cosine Similarity**) میان پرسمان (**q**) و بردارهای رهبران (**L**) است. صورت مسئله به یک پرسمان نادر (**Rare Query**) اشاره

دارد که هیچ یک از کلمات آن در بردار 1000 رهبر انتخاب شده وجود ندارد.

در این سناریو، خروجی و رفتار سیستم به شرح زیر خواهد بود:

محاسبه ریاضی: از آنجا که هیچ واژه مشترکی بین پرسمان و رهبران وجود ندارد، حاصل ضرب داخلی (**Dot Product**) در فرمول شباهت کسینوسی برای تمامی 1000 رهبر دقیقاً برابر با صفر خواهد بود:

$$\text{sim}(q, L_i) = 0$$

خروجی نهایی: سیستم در یافتن «نزدیکترین رهبر» کاملاً شکست می خورد. بسته به نوع پیاده سازی نرم افزاری، سیستم یا لیست تهی (**Zero Results**) را برمی گرداند، و یا در بدترین حالت، به صورت تصادفی یکی از رهبران (و پیروانش) را انتخاب کرده و اسنادی کاملاً نامرتب (**Irrelevant Documents**) را به کاربر نمایش می دهد که منجر به افت شدید شاخص بازخوانی (**Recall**) و نارضایتی کاربر می گردد.

بخش 2) راهکارهای پیشنهادی در معماری سیستم (Architectural Solutions)

برای حل مشکل «از دست رفتن پرسمان های نادر» و ایجاد پایداری در سیستم، معماری هرس خوشه ای باید با مکانیزم های ترکیبی زیر اصلاح شود:

1. پیاده سازی مکانیزم بازگشت مبتنی بر آستانه (Threshold-based Fallback):

تشریح منطق: سیستم نباید به طور کورکورانه همواره از هرس خوشه ای استفاده کند. باید یک حد آستانه حداقل شباهت (مثلاً ϵ) تعریف شود.

نحوه عملکرد: هنگامی که سیستم شباهت پرسمان را با تمام رهبران محاسبه می کند، بررسی می کند که آیا بیشترین شباهت یافت شده (**Max Similarity**) از ϵ بزرگتر است یا خیر. اگر بزرگتر بود، فرآیند هرس خوشه ای عادی ادامه می یابد. اما اگر مانند سناریوی صفر مطلق در این مسئله باشد، سیستم فوراً الگوریتم خوشه ای را متوقف کرده و به عنوان نقشه جایگزین (**Fallback**)، پرسمان را به یک نمایه معکوس استاندارد (**Standard Inverted Index**) ارسال می کند تا جستجوی دقیق (**Exact Search**) انجام شود.

2. مسیریابی هوشمند پرسمان بر اساس ارزش واژگان (Smart Query Routing):

تشریح منطق: کلماتی که در هیچ یک از رهبران (که نماینده کل مجموعه هستند) حضور ندارند، قطعاً کلماتی بسیار نادر با فراوانی سند (**df**) بسیار پایین و فراوانی معکوس سند (**idf**) بسیار بالا هستند.

نحوه عملکرد: پیش از آنکه پرسمان وارد فاز مقایسه با رهبران شود، مازول پردازش پرسمان (**Query Parser**) مقدار **idf** کلمات آن را بررسی می کند. اگر پرسمان منحصرراً از کلماتی با **idf** بسیار بالا تشکیل شده باشد، سیستم تشخیص می دهد که این یک "پرسمان در دُم توزیع" (**Tail Query**) است. در این حالت، سیستم به صورت پیش دستانه (**Proactive**) معماری خوشه ای را دور زده (**Bypass**) و مستقیماً از نمایه معکوس برای واکنشی نتایج استفاده می کند. این کار از هدررفت منابع برای مقایسه بی حاصل با رهبران جلوگیری می کند.

3. افزایش ضریب هم پوشانی رهبران در فاز نمایه سازی (b_1 و b_2):

در معماری پیشرفته تر هرس خوشه ای، به جای آنکه هر سند پیرو تنها به یک رهبر متصل شود، به b_1 رهبر نزدیک به خود متصل می گردد (ایجاد خوشه های دارای هم پوشانی). اگرچه این روش به تنهایی مشکل صفر بودن شباهت

پرسمان با رهبران را حل نمی‌کند، اما شانس اینکه کلمات نادر در مرزهای خوشه‌ها توزیع بهتری پیدا کنند را افزایش می‌دهد.

بخش 3) نتیجه‌گیری ساختاری

الگوریتم هرس خوشه‌ای، ابزاری قدرتمند برای افزایش سرعت در پرسمان‌های متداول (**Head Queries**) است، اما ذاتاً برای مدیریت پرسمان‌های نادر نقطه‌ضعف دارد. یک موتور جستجوی استاندارد و دانشگاهی، هرگز به یک الگوریتم واحد تکیه نمی‌کند؛ بلکه از یک معماری هیبریدی (**Hybrid Architecture**) استفاده می‌کند که در آن، سرعت بالای هرس خوشه‌ای با دقت و جامعیت نمایه معکوس (به عنوان شبکه ایمنی یا **Safety Net**) ترکیب می‌شود.

پاسخنامه مسئله 7.15: طراحی نمایه‌های چندلایه برای تجارت الکترونیک (Tiered Index Design for E-commerce)

بخش 1) مقدمه و منطق لایه‌بندی در دامنه تجارت الکترونیک

در معماری موتورهای جستجوی وب عمومی، لایه‌بندی نمایه‌ها (**Tiered Indexes**) معمولاً صرفاً بر اساس فراوانی واژه (**tf**) یا امتیاز پیج‌رنک (**PageRank**) انجام می‌شود. اما در دامنه تجارت الکترونیک، مفهوم «ارتباط» (**Relevance**) مستقیماً با «قابلیت فروش» و «رضایت مشتری» گره خورده است. بنابراین، تخصیص یک محصول (سند) به لایه‌های مختلف باید بر اساس ترکیبی از امتیازات کیفیت ایستا (**Static Quality Scores**) شامل موجودی، اعتبار محصول و همچنین موقعیت و اهمیت واژگان (**Metadata/TF**) در فاز آفلاین صورت گیرد.

در این معماری، هدف این است که پرسمان کاربر در سریع‌ترین زمان ممکن با محصولاتی پاسخ داده شود که هم مرتبط هستند و هم قابلیت خریداری دارند.

بخش 2) طراحی ساختار 3 لایه‌ای و پارامترهای تخصیص

برای تخصیص محصولات به این سه لایه، ما یک «امتیاز کیفیت محصول» (**Product Quality Score**) شامل موجودی کالا (**Inventory**)، میانگین امتیاز کاربران (**User Rating**)، و میزان فروش/بازدید (**Popularity**) در نظر می‌گیریم. همچنین موقعیت کلمه (مترادف با **tf** در فیلدهای مختلف) نیز لحاظ می‌شود.

1. لایه اول (لایه طلایی یا محصولات استراتژیک):

پارامترها و شروط تخصیص:

- موجودی (**Inventory**): کالا حتماً موجود باشد (**Inventory > 0**).
- امتیاز کاربر (**Rating**): میانگین امتیاز کالا بالا باشد (مثلاً **4.0 ≥** از 5).
- اهمیت متنی (**Term Weight**): واژه مورد نظر دقیقاً در «عنوان محصول» (**Product Title**) یا ویژگی‌های اصلی (مانند برند) قرار داشته باشد.

منطق معماری: این لایه شامل بهترین، محبوب‌ترین و آماده‌ترین محصولات برای فروش است. از آنجایی که بیش از 80 درصد پرسمان‌های کاربران (**Head Queries**) با هدف خرید فوری کالاهای محبوب انجام می‌شود،

جستجو در این لایه بسیار کوچک و فشرده، بالاترین نرخ تبدیل (**Conversion Rate**) را با کمترین زمان تأخیر (**Latency**) به همراه دارد.

2. لایه دوم (لایه استاندارد یا کاتالوگ عمومی):

پارامترها و شروط تخصیص:

- موجودی (**Inventory**): کالا موجود باشد.
- امتیاز کاربر (**Rating**): کالا دارای امتیاز متوسط (مثلاً بین 3.0 تا 4.0) باشد یا هنوز امتیازی دریافت نکرده باشد (کالای جدید).
- اهمیت متنی (**Term Weight**): واژه مورد نظر به جای عنوان، در «توضیحات محصول» (**Product Description**) یا نظرات کاربران با فراوانی متوسط (**tf**) یافت شود.

منطق معماری: سیستم زمانی به این لایه مراجعه می‌کند (**Fallback**) که لایه اول نتواند **K** محصول مرتبط را برای کاربر پیدا کند (مثلاً جستجوی یک ویژگی خاص که در عنوان ذکر نمی‌شود ولی در توضیحات وجود دارد). این لایه حجم بزرگتری دارد اما همچنان کالاهای قابل فروش را تضمین می‌کند.

3. لایه سوم (لایه حاشیه‌ای، ناموجود و آرشیو):

پارامترها و شروط تخصیص:

- موجودی (**Inventory**): کالاهایی که ناموجود (**Out of Stock**) یا توقف تولید هستند.
- امتیاز کاربر (**Rating**): کالاهایی با امتیاز بسیار پایین (مثلاً < 3.0).
- اهمیت متنی (**Term Weight**): واژه مورد نظر تنها یک یا دو بار (**tf** بسیار پایین) در بخش‌های کم‌اهمیت متن پنهان شده باشد.

منطق معماری: چرا کالای ناموجود یا بی‌کیفیت را نمایه‌سازی می‌کنیم؟ در فروشگاه‌های آنلاین، گاهی کاربران به دنبال مدل‌های قدیمی برای مقایسه، یافتن لوازم جانبی آن‌ها، یا خواندن مشخصات فنی هستند. این لایه (که بزرگترین حجم داده را دارد) تنها زمانی پردازش می‌شود که پرسمان کاربر بسیار نادر (**Long-tail Query**) باشد و لایه‌های 1 و 2 هیچ خروجی مفیدی نداشته باشند. هدف این لایه، جلوگیری از نمایش فاجعه‌بار «صفحه بدون نتیجه» (**Zero-Result Page**) است.

بخش 3 نتیجه‌گیری الگوریتم در زمان اجرا (Online Phase)

موتور جستجوی فروشگاه همواره پردازش پرسمان را از لایه 1 آغاز می‌کند. محاسبه گران‌قیمت وزن‌دهی‌های پویا و شباهت کسینوسی در ابتدا فقط روی همین لایه کوچک انجام می‌شود. تنها در صورتی که تعداد محصولات استخراج‌شده شرط $K <$ را نشان دهد، سیستم به صورت آبشاری (**Cascading**) به لایه 2 و در نهایت به لایه 3 سقوط می‌کند تا ظرفیت **K** نتیجه برتر تکمیل شود. این مکانیزم مهندسی، ضمن حفظ کیفیت و دقت، بار سرورهای پایگاه داده فروشگاه را به شدت کاهش می‌دهد.

پاسخنامه مسئله 7.16: وزن‌دهی به مجاورت در سیستم‌های خبری (**Proximity Weighting**)

بخش 1) مقدمه و بیان هدف

در مدل کلاسیک فضای برداری (**Vector Space Model**)، اسناد بر اساس فرض «کیسه کلمات» (**Bag of Words**) ارزیابی می‌شوند؛ به این معنا که تنها «حضور» و «فراوانی» واژگان در امتیاز نهایی تاثیرگذار است و ترتیب یا فاصله فیزیکی کلمات در متن نادیده گرفته می‌شود. در دامنه‌هایی مانند خبرخوان‌ها، زمانی که کاربر ترکیب "حمله سایبری بانک" را جستجو می‌کند، نیت او یافتن خبری است که این مفاهیم در یک گزاره‌ی معنایی مشترک (و در نتیجه نزدیک به یکدیگر) بیان شده باشند.

طراحی یک تابع امتیازدهی مبتنی بر مجاورت (**Proximity-based Scoring**)، این خلأ معنایی را با جریمه کردن کلماتی که در سند پراکنده‌اند و پاداش دادن به کلماتی که به هم متصل هستند، برطرف می‌کند.

بخش 2) تعریف متغیرهای سیستم و مدل ریاضی

برای طراحی این تابع به شکلی دقیق و ساختاریافته، ابتدا متغیرهای سیستم را بر اساس استانداردهای بازیابی اطلاعات تعریف می‌کنیم:

مجموعه پرسمان را به صورت $q = \{t_1, t_2, t_3\}$ در نظر می‌گیریم (که در آن $t_1 =$ حمله، $t_2 =$ سایبری، $t_3 =$ بانک است). طول پرسمان $|q| = 3$ است.

امتیاز پایه شباهت کسینوسی (**tf-idf**) را با $S_{vsm}(q, d)$ نمایش می‌دهیم.

متغیر $w_{min}(q, d)$ را به عنوان «کوچکترین پنجره متنی» (**Smallest Window Size**) تعریف می‌کنیم که هر 3 کلمه پرسمان را (به هر ترتیبی) در داخل متن سند d در بر می‌گیرد.

نکته محاسباتی: اگر هر 3 کلمه دقیقاً و بدون هیچ کلمه اضافی کنار هم باشند، $w_{min} = 3$ خواهد بود. اگر 1 کلمه حائل بین آن‌ها باشد، $w_{min} = 4$ و به همین ترتیب محاسبه می‌شود.

بخش 3) طراحی تابع امتیازدهی مجاورت (Proximity Function)

برای اعمال دقیق شرط مطرح شده در صورت مسئله (تخصیص امتیاز ویژه منحصراً به اسنادی که این 3 کلمه در آن‌ها در یک پنجره 5 کلمه‌ای قرار دارند)، یک تابع پله‌ای-پیوسته (**Step-Continuous Function**) به نام $S_{prox}(q, d)$ طراحی می‌کنیم:

$$S_{prox}(q, d) = \begin{cases} \frac{1}{w_{min}(q, d) - 2} & \text{if } w_{min}(q, d) \leq 5 \\ 0 & \text{otherwise} \end{cases}$$

تحلیل رفتار تابع:

اگر کلمات کاملاً متصل باشند ($w_{min} = 3$)، مخرج کسر برابر با 1 شده و تابع بیشترین پاداش ممکن (مقدار 1.0) را برمی‌گرداند.

اگر کلمات دقیقاً در یک پنجره 5 کلمه‌ای باشند ($w_{min} = 5$)، تابع پاداش کمتری (مقدار 0.33) برای این مجاورت اعمال می‌کند.

به محض اینکه فاصله کلمات از مرز 5 کلمه تجاوز کند، تابع مقدار 0 را برمی‌گرداند و سند مربوطه به عنوان یک سند فاقد مجاورت معنایی در نظر گرفته می‌شود.

بخش 4) تابع ترکیب و رتبه‌بندی نهایی (Final Scoring Function)

اکنون برای استخراج لیست نهایی اسناد برتر (**Top-K Ranking**)، امتیاز پایه کسینوسی را با امتیاز مجاورت ادغام می‌کنیم. استانداردترین الگو برای این ادغام، استفاده از یک ترکیب خطی (**Linear Combination**) با ضرایب وزن‌دهی کنترل‌شونده است:

$$\text{FinalScore}(q, d) = \alpha \cdot S_{vsm}(q, d) + \beta \cdot S_{prox}(q, d)$$

متغیر α : ضریب اهمیت ارتباط متنی پایه در فضای برداری.

متغیر β : ضریب تقویت (**Boost Weight**) برای مجاورت کلمات.

این ضرایب معمولاً به صورت تجربی و از طریق اجرای الگوریتم‌های یادگیری رتبه‌بندی (**Learning to Rank**) در فاز آموزش سیستم، بهینه و تنظیم (**Tune**) می‌شوند.

بخش 5) مثال تحلیلی و مکانیزم اجرا

فرض کنید **2** سند خبری در سیستم وجود دارند که هر دو شامل این **3** کلمه هستند و امتیاز کسینوسی S_{vsm} تقریباً یکسانی کسب کرده‌اند:

سند الف: "... هشدار درباره حمله بزرگ سایبری به بانک مرکزی ..."

محاسبه پنجره: کلمات هدف در عبارت (حمله**(1)**، بزرگ**(2)**، سایبری**(3)**، به**(4)**، بانک**(5)**) قرار دارند. طول کوچکترین پنجره دقیقاً **5** کلمه است ($w_{min} = 5$).

خروجی مجاورت: شرط $5 \leq$ برقرار است؛ بنابراین تابع فعال شده و امتیاز $S_{prox} = 1/3$ محاسبه و در متغیر β ضرب می‌شود.

سند ب: "... رئیس بانک ملی امروز استعفا داد ... [**50** کلمه فاصله] ... پلیس فتا از یک حمله سایبری خبر داد ..."

محاسبه پنجره: فاصله بین کلمات بسیار زیاد است.

خروجی مجاورت: تابع پاداش فعال نمی‌شود و $S_{prox} = 0$ خواهد بود.

بخش 6) نتیجه‌گیری

با اعمال این فرمول‌بندی ریاضی، سیستم موتور جستجوی خبرخوان به طور خودکار «سند الف» را به دلیل دارا بودن مجاورت فیزیکی و همبستگی معنایی بالا، با اختلاف معناداری نسبت به «سند ب» ارتقا (**Boost**) داده و در صدر لیست نتایج جستجو به کاربر نمایش می‌دهد که این فرآیند دقیقاً تضمین‌کننده رفع نیاز اطلاعاتی کاربر است.

پاسخنامه مسئله 7.17: طراحی تابع امتیازدهی ترکیبی (Designing Combined Scoring Function)

بخش 1) مقدمه و تحلیل متغیرها

برای طراحی یک تابع امتیازدهی (**Scoring Function**) پایدار که بتواند میان ارتباط متنی، اعتبار ذاتی و تازگی سند موازنه ایجاد کند، ابتدا باید هر سه فاکتور را در یک بازه استاندارد (نرمال‌سازی شده) تعریف کنیم. دو فاکتور اول در صورت مسئله نرمال شده‌اند، اما فاکتور زمان نیاز به مدل‌سازی ریاضی دارد.

شباهت کسینوسی (S_{cos}): میزان ارتباط متنی پرسمان و سند $\in [0, 1]$

امتیاز پیچرنک (PR): میزان اعتبار و کیفیت ایستای سند $\in [0, 1]$

تازگی سند (**Freshness**): برای تبدیل تاریخ انتشار به یک امتیاز بین 0 و 1، از یک «تابع زوال نمایی» (**Exponential Decay Function**) استفاده می‌کنیم. اگر عمر سند را با Δt (اختلاف زمان حال و تاریخ انتشار) نشان دهیم:

$$F(d) = e^{-\lambda \Delta t}$$

در این فرمول، λ ثابت زوال است. هرچه سند جدیدتر باشد $\Delta t \rightarrow 0$ ، مقدار $F(d)$ به 1 نزدیک می‌شود و با گذشت زمان به سمت 0 میل می‌کند.

بخش 2) چالش طراحی و شرط اساسی مسئله

شرط کلیدی این است: «اسناد قدیمی اما بسیار معتبر، باید اسناد جدید بی‌ربط را مغلوب کنند.»

اگر از یک تابع ترکیب خطی ساده (جمع کردن فاکتورها) استفاده کنیم، یک سند کاملاً بی‌ربط ($S_{cos} \approx 0$) که همین امروز منتشر شده ($F(d) = 1$) ممکن است با یک سند بسیار مرتبط و معتبر اما قدیمی رقابت کند. برای جلوگیری از این خطای سیستماتیک، باید از یک «معماری مبتنی بر دروازه» (**Gated Architecture**) یا ترکیب ضربی استفاده کرد.

بخش 3) طراحی تابع نهایی

برای تضمین شرط مسئله، شباهت کسینوسی را به عنوان یک «ضریب فیلترکننده» (**Multiplier**) در نظر می‌گیریم و دو فاکتور کیفیت (پیچرنک و تازگی) را با یکدیگر به صورت خطی ترکیب کرده و در آن ضرب می‌کنیم:

$$\text{NetScore}(q, d) = S_{cos}(q, d) \times (\alpha \cdot PR(d) + \beta \cdot F(d))$$

بخش 4) تشریح ضرایب و منطق تابع

پارامترهای تنظیم (α و β): ضرایبی هستند که مجموع آن‌ها برابر با 1 است ($\alpha + \beta = 1$). برای اولویت دادن به اعتبار علمی نسبت به صرف جدید بودن، باید وزن پیچرنک بسیار بیشتر از تازگی تنظیم شود (مثلاً $\alpha = 0.8$ و $\beta = 0.2$).

نقش شباهت کسینوسی به عنوان دروازه منطقی: از آنجایی که S_{cos} در کل عبارت ضرب می‌شود، اگر سندی نامرتب باشد (مقدار S_{cos} نزدیک به صفر)، حاصل ضرب نهایی صرف‌نظر از میزان تازگی یا پیچرنک به شدت افت کرده و به صفر میل می‌کند.

بخش 5) اثبات کارایی تابع با یک مثال تحلیلی:

فرض کنید $\alpha = 0.8$ و $\beta = 0.2$ تنظیم شده است. دو سند زیر را در نظر بگیرید:

سند الف (قدیمی، بسیار معتبر و مرتبط):

ارتباط متنی بالایی دارد: $S_{cos} = 0.9$

مرجع معتبری است: $PR = 0.95$

10 سال پیش منتشر شده است: $F \approx 0.05$ (بسیار قدیمی)

محاسبه امتیاز:

$$\text{NetScore}_A = 0.9 \times (0.8 \times 0.95 + 0.2 \times 0.05) = 0.9 \times 0.77 = 0.693$$

سند ب (جدید، بی‌ربط و نامعتبر):

ارتباط متنی ضعیفی دارد: $S_{cos} = 0.1$

وبلاگ ناشناخته‌ای است: $PR = 0.1$

امروز منتشر شده است: $F = 1.0$ (کاملاً جدید)

محاسبه امتیاز:

$$\text{NetScore}_B = 0.1 \times (0.8 \times 0.1 + 0.2 \times 1.0) = 0.1 \times 0.28 = 0.028$$

بخش 6) نتیجه‌گیری

همان‌طور که محاسبات نشان می‌دهد، $\text{NetScore}_A \gg \text{NetScore}_B$. طراحی این ساختار ریاضی (قرار دادن ارتباط متنی به عنوان ضریب تعدیل‌کننده و تنظیم دقیق وزن‌های ایستا)، به صورت مطلق تضمین می‌کند که تازگی یک سند به تنهایی نمی‌تواند بر عدم ارتباط موضوعی آن غلبه کند و ساختار کیفی موتور جستجو در برابر اسناد زرد (**Clickbait**) و بی‌ربط مصون می‌ماند.

پاسخنامه مسئله 7.18: مدیریت حافظه و گلوگاه‌های سیستم بازیابی یکپارچه

(Memory Management and System Bottlenecks)

بخش 1) مقدمه و تحلیل بحران حافظه در معماری یکپارچه

در یک موتور جستجوی یکپارچه (**Integrated IR System**)، ساختارهای داده‌ای متعددی برای تسریع فرآیند پاسخ‌گویی در حافظه اصلی (**RAM**) مقیم می‌شوند. حافظه **RAM** به دلیل سرعت بسیار بالای دسترسی (از مرتبه نانو ثانیه)، کلیدی‌ترین منبع برای کاهش تأخیر (**Latency**) است. اما در شرایط بحران حافظه (**Memory Exhaustion**)، مهندس سیستم باید تصمیم بگیرد کدام بخش از داده‌ها به حافظه دیسک (که سرعت دسترسی آن از مرتبه میلی‌ثانیه است و یک گلوگاه **I/O** محسوب می‌شود) منتقل گردد (**Paging/Eviction**).

بخش 2) تصمیم معماری (انتخاب ساختار داده برای انتقال)

در بین ساختارهای موجود (حذف نمایه، لیست قهرمان، و نمایه‌های چندلایه)، «پایین‌ترین لایه از نمایه‌های چندلایه» (**Lowest Tier of the Tiered Index**) - مثلاً لایه سوم یا نمایه استاندارد پشتیبان) باید به عنوان اولین کاندیدا برای انتقال به هارد دیسک انتخاب شود.

لیست‌های قهرمان (**Champion Lists**) و لایه اولِ نمایه‌ها (**Tier 1**) تحت هیچ شرایطی نباید در مرحله اول به دیسک منتقل شوند.

بخش 3) استدلال علمی و گام‌به‌گام برای این تصمیم

1. الگوی توزیع پرسمان‌ها و اصل محلیت (Query Distribution and Locality):

در سیستم‌های بازیابی اطلاعات، اکثر پرسمان‌های کاربران (بیش از 80 تا 90 درصد) در دسته پرسمان‌های متداول (**Head Queries**) قرار می‌گیرند.

سیستم طوری طراحی شده است که با استفاده از «لیست‌های قهرمان» یا «لایه اول نمایه»، بتواند بلافاصله **K** سند برتر را برای این پرسمان‌ها پیدا کند. بنابراین، نرخ اصابت (**Hit Rate**) به لایه‌های بالایی بسیار زیاد است. انتقال این لایه‌های پرمراجعه به دیسک، به معنای تحمیل جریمه زمانی (**Disk Seek Time**) به اکثریت کاربران و فروپاشی کامل زمان پاسخ‌گویی (**Latency**) است.

2. تحلیل امید ریاضی زمان پاسخ‌گویی (Expected Latency):

زمان پاسخ‌گویی مورد انتظار در یک سیستم چندلایه با استفاده از امید ریاضی (**Expected Value**) به صورت زیر مدل‌سازی می‌شود:

$$E[\text{Latency}] \approx P(\text{Tier}_1) \cdot T_{\text{RAM}} + P(\text{Tier}_{\text{low}}) \cdot T_{\text{Disk}}$$

متغیر $P(\text{Tier}_1)$ احتمال پاسخ‌دهی در لایه اول است (که مقدار آن بسیار نزدیک به 1 است).

متغیر $P(\text{Tier}_{\text{low}})$ احتمال سقوط به لایه‌های پایین‌تر (**Fallback**) برای یافتن کسر نتایج است (که مقدار آن بسیار نزدیک به صفر است).

از آنجا که سیستم به ندرت برای یافتن **K** سند به پایین‌ترین لایه مراجعه می‌کند، قرار دادن آن لایه در دیسک، مقدار T_{Disk} (زمان کند هارد دیسک) را تنها در کسر بسیار کوچکی از پرسمان‌ها (پرسمان‌های نادر یا **Long-tail Queries**) ضرب می‌کند. در نتیجه، میانگین تأخیر کل سیستم کمترین آسیب را می‌بیند.

3. نرخ آزادسازی فضا به ازای هزینه (Space/Cost Ratio):

لیست‌های قهرمان: ساختارهایی بسیار فشرده و کوچک هستند. انتقال آن‌ها به دیسک، فضای **RAM** قابل توجهی را آزاد نمی‌کند، اما هزینه تأخیر فاجعه‌باری دارد.

پایین‌ترین لایه نمایه: این لایه شامل اسنادی با پایین‌ترین امتیاز (مثل **tf** کم یا **PageRank** پایین) و اسناد نامرتب‌تر است که بزرگترین حجم داده‌ها (اغلب بیش از 70 درصد کل نمایه) را تشکیل می‌دهد. انتقال این بخش به دیسک، فوراً حجم عظیمی از **RAM** را آزاد کرده و سیستم را از وضعیت بحران خارج می‌کند، در حالی که تأثیر آن بر سرعت جستجوهای روزمره تقریباً نامحسوس است.

بخش 4) نتیجه‌گیری ساختاری

در معماری یک موتور جستجوی استاندارد، استراتژی مدیریت حافظه بر پایه «ارزش احتمالاتی داده‌ها» استوار است. لایه‌های بالایی و لیست‌های قهرمان، خط مقدم پاسخ‌گویی هستند و باید همواره در سریع‌ترین حافظه (**RAM**) حفظ شوند. انتقال لایه‌های

پاسخنامه مسئله 7.19: فیلتر بولین پیش از رتبه‌بندی برداری

(Boolean Filter before Vector Ranking)

بخش 1) مقدمه و تشریح معماری پایپ‌لاین (Pipeline Architecture)

در سیستم‌های بازیابی اطلاعات مقیاس‌پذیر، محاسبه ضرب نقطه‌ای (**Dot Product**) یا شباهت کسینوسی میان بردار پرسمان و بردار تک‌تک اسناد در یک مجموعه عظیم، از نظر محاسباتی بسیار پرهزینه و غیرمنطقی است. برای حل این مشکل، از یک «معماری دو مرحله‌ای» (**Two-Stage Architecture**) استفاده می‌شود. در این پایپ‌لاین، مدل قطعی و سریع «بولین» به عنوان یک فیلتر اولیه (**Candidate Generator**) عمل کرده و سپس مدل منعطف «فضای برداری» (**Vector Space Model**) وظیفه رتبه‌بندی دقیق (**Ranker**) را بر روی یک زیرمجموعه بسیار کوچک‌تر بر عهده می‌گیرد.

بخش 2) مراحل اجرای پایپ‌لاین برای پرسمان: (Apple OR Samsung) AND Laptop

مرحله اول: فیلترینگ سریع بولی (Fast Boolean Filtering)

در این مرحله، سیستم به هیچ وجه درگیر محاسبه وزن‌ها (**tf-idf**) یا رتبه‌بندی نمی‌شود و صرفاً منطق مجموعه‌ها را روی لیست‌های پستی (**Postings Lists**) اجرا می‌کند.

1. **واکشی لیست‌ها:** سیستم لیست پستی سه واژه **Apple**، **Samsung** و **Laptop** را از نمایه معکوس (**Inverted Index**) در حافظه بارگذاری می‌کند.

2. **اجرای عملگر OR (اجتماع):** ابتدا سیستم لیست‌های **Apple** و **Samsung** را با یکدیگر ادغام (**Merge**) می‌کند. از نظر الگوریتمی، این کار معادل عملیات اجتماع روی دو مجموعه است. خروجی این بخش، لیستی از شناسه‌ها (**DocIDs**) است که حداقل یکی از دو برند در آن‌ها وجود دارد.

3. **اجرای عملگر AND (اشتراک):** در گام بعدی، لیست حاصل از مرحله قبل، با لیست پستی **Laptop** اشتراک‌گیری (**Intersection**) می‌شود. در این عملیات، تنها اسنادی باقی می‌مانند که کلمه **Laptop** را حتماً در کنار حداقل یکی از کلمات **Apple** یا **Samsung** داشته باشند.

دستآورد مرحله اول: خروجی این فاز، یک مجموعه نامرتب اما به شدت کوچک‌شده از اسناد است که آن را $D_{filtered}$ می‌نامیم. حجم این مجموعه کسری بسیار ناچیز از کل پایگاه داده است.

مرحله دوم: رتبه‌بندی برداری و محاسبه ضرب نقطه‌ای (Vector Ranking)

اکنون که فضای جستجو به شدت هرس شده است، پایپ‌لاین وارد فاز محاسبات ریاضی سنگین می‌شود.

1. **تشکیل بردارها:** سیستم تنها برای اسناد موجود در مجموعه $D_{filtered}$ ، بردار سند ($\vec{d}$) را استخراج می‌کند.

2. **محاسبه ضرب نقطه‌ای:** امتیاز نهایی هر سند صرفاً برای همین زیرمجموعه از طریق فرمول ضرب نقطه‌ای (**Dot Product**) محاسبه می‌گردد:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} w_{t,q} \cdot w_{t,d}$$

3. **مرتب‌سازی و استخراج خروجی:** در نهایت، اسناد موجود در مجموعه $D_{filtered}$ بر اساس امتیاز محاسبه‌شده به صورت نزولی مرتب شده و **K** سند برتر (**Top K**) به کاربر نمایش داده می‌شوند.

بخش 3) تحلیل کارایی و توجیه مهندسی

دلیل استفاده از این پایپ‌لاین ترکیبی، موازنه بی‌نظیر میان «زمان پردازش» و «دقت» است. عملیات بولی (اشتراک و اجتماع روی لیست‌های مرتب‌شده بر اساس **DocID**) با استفاده از اشاره‌گرها در مرتبه زمانی خطی و با سرعت بسیار بالا در پردازنده (**CPU**) انجام می‌شود. با اجرای این فیلتر، موتور جستجو خود را از محاسبه ضرب نقطه‌ای برای میلیون‌ها سند نامرتب معاف کرده و توان پردازشی گران‌قیمت خود را صرفاً به رتبه‌بندی چند هزار سندی اختصاص می‌دهد که با قطعیت می‌دانیم شرایط پایه‌ای پرسمان کاربر را دارا هستند.

پاسخنامه مسئله 7.20: موازنه میان سرعت و دقت (Speed vs. Accuracy Trade-off)

بخش 1) مقدمه و تحلیل موازنه مهندسی

در طراحی سیستم‌های بازیابی اطلاعات، کاهش 50 درصدی زمان پاسخ‌گویی (**Latency**) در ازای پذیرش 5% افت در دقت (**Accuracy**)، یک موازنه (**Trade-off**) کاملاً استاندارد و منطقی در مقیاس صنعتی است. برای دستیابی به این هدف، سیستم باید از پردازش بخش عظیمی از اسناد که شانس کمی برای قرار گرفتن در لیست نتایج نهایی دارند، صرف‌نظر کند.

برای پاسخ به نیاز مدیر محصول، دو تکنیک کلیدی از معماری بازیابی غیردقیق (**Inexact Retrieval**) که قابلیت اعمال با شدت بالا (**Aggressive Tuning**) را دارند، به شرح زیر مهندسی و پیاده‌سازی می‌شوند:

بخش 2) تکنیک اول: اعمال تهاجمی حذف نمایه (Aggressive Index Elimination)

مکانیزم اجرا برای افزایش سرعت: در حالت استاندارد، سیستم تنها کلماتی با فراوانی سند (**df**) بسیار بالا (مانند حروف اضافه) را حذف می‌کند. برای کاهش چشمگیر زمان پاسخ‌دهی، ما «آستانه حذف» (**Elimination Threshold**) را به شدت ارتقا می‌دهیم؛ به طوری که کلمات دارای فراوانی متوسط نیز از فرآیند پردازش حذف شوند. با این کار، طولانی‌ترین لیست‌های پستی (**Postings Lists**) به طور کامل نادیده گرفته شده و بار ورودی/خروجی دیسک (**Disk I/O**) و محاسبات پردازنده به شدت افت می‌کند.

عوارض جانبی (Side Effect): پرسمان‌هایی که شامل مفاهیم خاص اما با کلمات نسبتاً رایج هستند، دچار افت دقت می‌شوند، زیرا کلمات کلیدی آن‌ها پردازش نخواهد شد.

کنترل و مدیریت عوارض: برای حفظ سقف 5% افت دقت، از مکانیزم «حذف وابسته به طول پرسمان» (**Query-Length Dependent Elimination**) استفاده می‌کنیم. به این معنا که این فیلتر تهاجمی تنها بر روی

پرسمان‌های طولانی (مثلاً بیش از 4 کلمه) اعمال می‌گردد. در پرسمان‌های کوتاه (که حذف یک کلمه به معنای از دست رفتن بخش عمده‌ای از بار معنایی است)، این فیلتر غیرفعال می‌شود تا دقت در پرسش‌های حیاتی فدای سرعت نگردد.

بخش 3) تکنیک دوم: تحدید شدید ظرفیت در نمایه‌های چندلایه (Aggressive Tiering & Early Termination)

مکانیزم اجرا برای افزایش سرعت: در معماری نمایه‌های چندلایه (**Tiered Indexes**)، ظرفیت لایه اول (**Tier 1**) را به شدت کاهش می‌دهیم. به عنوان مثال، حد آستانه امتیاز ورود به این «لایه طلایی» را بسیار سخت‌گیرانه تنظیم می‌کنیم تا حجم آن نصف شود. همچنین، شرط سقوط به لایه دوم (**Fallback to Tier 2**) را محدودتر می‌سازیم. در این ساختار، سیستم جستجو در اکثریت مواقع، تنها یک مجموعه بسیار فشرده و کوچک را در حافظه اصلی (**RAM**) پیمایش کرده و عملیات را بلافاصله متوقف می‌کند.

عوارض جانبی: در برخی پرسمان‌های پیچیده‌تر، ممکن است تعداد نتایج باکیفیت در لایه اول کمتر از سقف **K** باشد. با توجه به محدود شدن شرط مراجعه به لایه دوم، سیستم ممکن است لیست نتایجی با ارتباط معنایی پایین‌تر برگرداند.

کنترل و مدیریت عوارض: برای جلوگیری از فروپاشی کیفیت، از ترکیب «امتیاز کیفیت ایستا» (**Static Quality Score**) مانند پیچ‌رنک) با وزن متنی برای تخصیص اسناد به لایه اول استفاده می‌کنیم. این معماری تضمین می‌کند که حتی با وجود کوچک بودن لایه اول، اسنادی که در آن قرار دارند از بالاترین اعتبار جهانی برخوردارند. در نتیجه، نیاز اکثریت کاربران (**Head Queries**) با بالاترین کیفیت برآورده شده و خطای سیستم فراتر از تلورانس 5 درصدی نخواهد رفت.

بخش 4) نتیجه‌گیری ساختاری

دستیابی به سرعت نمایی در سیستم‌های مقیاس‌پذیر، همواره نیازمند هرس کردن هدفمند فضای جستجو است. ترکیب «حذف واژگان با **df** بالا در پرسمان‌های طولانی» و «کاهش حجم لایه اول نمایه»، زمان محاسبات گران‌قیمت ضرب نقطه‌ای (**Dot Product**) را به کسر کوچکی از حالت اولیه تقلیل می‌دهد. این معماری به شما اجازه می‌دهد دقیقاً روی مرز موازنه حرکت کرده و زمان پاسخ‌دهی را نصف کنید، بدون آنکه ساختار آموزشی و علمی موتور جستجو دچار آسیب جدی شود.

پاسخنامه مسئله 7.21: الگوریتم پویا برای حدنصاب لیست قهرمان

★ (Dynamic Thresholding for Champion Lists)

بخش 1) مقدمه و تبیین منطق الگوریتم

در روش سنتی، اختصاص یک اندازه ثابت (مثلاً $r = 50$) به لیست قهرمان (**Champion List**) تمامی واژگان، فارغ از ارزش اطلاعاتی و آماری آن‌ها، یک رویکرد مهندسی غیربهبینه است. در مدل فضای برداری، واژگان نادر (دارای فراوانی معکوس سند یا **idf** بالا) قدرت تمایزدهندگی (**Discriminative Power**) بسیار بالایی دارند و نقش تعیین‌کننده‌ای در امتیاز نهایی شباهت کسینوسی ایفا می‌کنند.

الگوریتم پویا بر این اصل بنیادین استوار است: هرچه ارزش اطلاعاتی یک واژه بیشتر باشد، سیستم باید حاشیه امنیت بزرگتری برای آن در نظر بگیرد و اسناد بیشتری را در لیست قهرمان آن حفظ کند تا شانس بازیابی دقیق‌ترین نتایج افزایش یابد.

بخش 2) طراحی الگوریتم پویا در فاز پیش‌پردازش (Offline Phase)

برای تخصیص پویای ظرفیت، متغیر r_t (طول لیست قهرمان برای واژه t) را به عنوان تابعی از idf_t تعریف می‌کنیم. مراحل اجرای الگوریتم در زمان ساخت نمایه به شرح زیر است:

گام اول: محاسبه پارامترهای پایه

ابتدا برای هر واژه t در واژه‌نامه، مقدار فراوانی سند (df_t) و سپس ارزش اطلاعاتی آن (idf_t) محاسبه می‌گردد:

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

که در آن N تعداد کل اسناد مجموعه است.

گام دوم: اعمال تابع تخصیص ظرفیت

سیستم برای محاسبه طول لیست قهرمان (r_t) از فرمول پویای زیر استفاده می‌کند:

$$r_t = \min(K + \lambda \cdot idf_t, df_t)$$

تحلیل متغیرهای فرمول:

- متغیر K:** تعداد نتایج برتری است که سیستم متعهد به بازگرداندن آن‌هاست (مثلاً $K = 10$).
- پارامتر λ :** یک ضریب تنظیم‌کننده (**Tuning Parameter**) مثبت است که شدت تاثیر **idf** را کنترل می‌کند.
- تابع min:** این تابع به عنوان یک شرط مرزی عمل می‌کند تا تضمین نماید ظرفیت درخواستی لیست قهرمان، هرگز از تعداد کل اسنادی که آن واژه را در خود دارند (df_t) فراتر نرود.

گام سوم: مرتب‌سازی و برش

پس از تعیین مقدار اختصاصی r_t برای واژه t ، لیست پستی اصلی آن واژه بر اساس وزن **tf-idf** به صورت نزولی مرتب شده و دقیقاً به اندازه r_t سند اول جدا شده و به عنوان لیست قهرمان در حافظه اصلی ذخیره می‌گردد.

بخش 3) تحلیل سود و زیان معماری پویا (Pros and Cons Analysis)

الف) مزایا و دستاوردهای مهندسی (سود):

1. تخصیص هوشمندانه و بهینه‌سازی حافظه اصلی (RAM):

در این معماری، واژگان متداول (مانند "است" یا "از" با **idf** نزدیک به صفر)، ظرفیتی در حدود همان **K** سند دریافت می‌کنند، زیرا حضور آن‌ها در پرسمان تعیین‌کننده نیست. در عوض، فضای آزاد شده به واژگان تخصصی اختصاص می‌یابد. این کار از هدررفت حافظه برای ذخیره اسناد بی‌ارزش جلوگیری می‌کند.

2. ارتقای چشمگیر شاخص دقت (Precision) در پرسمان‌های نادر:

در پرسمان‌های تخصصی و طولانی، حفظ اسناد بیشتر برای کلمات کلیدی دارای **idf** بالا، تضمین می‌کند که مهم‌ترین اسناد در همان مرحله اول (بدون نیاز به مراجعه به نمایه اصلی روی دیسک) در دسترس سیستم قرار داشته و استخراج شوند.

1. پیچیدگی مدیریت حافظه (Memory Fragmentation):

در روش سنتی (اندازه ثابت)، مدیریت آرایه‌ها در حافظه ساده و یکپارچه است. اما در روش پویا، سیستم باید با ساختارهای داده‌ای با طول‌های کاملاً متغیر (**Variable-length Arrays**) کار کند که می‌تواند منجر به تکه‌تکه شدن حافظه (**Fragmentation**) و پیچیدگی در برنامه‌نویسی مدیریت حافظه شود.

2. خطر افت عملکرد در پرسمان‌های عمومی (Head Queries):

اگر کاربری پرسمانی کاملاً متشکل از کلمات متداول (با **idf** پایین) وارد کند، ظرفیت لیست‌های قهرمان برای این کلمات بسیار کوچک خواهد بود. این کوچکی مفرط می‌تواند باعث شود که اشتراک‌گیری (**Intersection**) میان این لیست‌ها، خروجی کمتر از **K** سند تولید کند. در نتیجه، سیستم مجبور به سقوط (**Fallback**) به لیست‌های پستی اصلی روی هارد دیسک می‌شود که تأخیر (**Latency**) جستجو را افزایش می‌دهد.

پاسخنامه مسئله 7.22: طراحی پایپ‌لاین هرس خوشه‌ای دو مرحله‌ای

★ (Two-Stage Cluster Pruning)

بخش 1) مقدمه و تحلیل بن‌بست محاسباتی

در الگوریتم پایه هرس خوشه‌ای (**Basic Cluster Pruning**)، انتخاب $\sqrt{N}$ رهبر برای مجموعه‌های نسبتاً کوچک کارآمد است. اما با رشد تصاعدی داده‌ها و رسیدن به مقیاس‌های کلان (مانند $N = 10^9$ سند)، اندازه $\sqrt{N}$ برابر با حدود **31,622** گره خواهد شد. در فاز پردازش آنلاین (**Online Phase**)، مقایسه بردار پرسمان با بیش از **31** هزار رهبر و سپس مقایسه با **31** هزار پیرو در خوشه منتخب (مجموعاً بیش از **63** هزار عملیات ضرب نقطه‌ای)، تأخیر (**Latency**) غیرقابل قبولی به سیستم تحمیل می‌کند.

برای شکستن این گلوگاه، ساختار مسطح به یک معماری سلسله‌مراتبی و دو مرحله‌ای (**Two-level Hierarchical Clustering**) ارتقا می‌یابد.

بخش 2) معماری پایپ‌لاین و طراحی ساختار (Offline Phase)

بر اساس اصول بهینه‌سازی ریاضی در ساختمان داده‌ها، برای کمینه‌سازی هزینه جستجو در یک درخت با عمق **3** (سطح بالا، سطح میانی و سطح برگ‌ها)، تعداد انشعابات در هر گره نباید برابر با ریشه دوم ($\sqrt{N}$) باشد، بلکه باید برابر با ریشه سوم کل اسناد ($N^{1/3}$) تنظیم شود.

برای مجموعه داده $N = 10^9$ سند، پایپ‌لاین در زمان ساخت نمایه به این شکل لایه‌بندی می‌شود:

محاسبه ضریب انشعاب: $N^{1/3} = (10^9)^{1/3} = 1000$ گره.

1. سطح اول (ابر-رهبران - **Super-leaders**): سیستم دقیقاً **1000** سند را به صورت تصادفی به عنوان ابر-رهبر

انتخاب می‌کند. تمامی اسناد دیگر به نزدیک‌ترین ابر-رهبر متصل می‌شوند. در این حالت، **1000** ابر-خوشه (**Super-cluster**) داریم که هر کدام شامل **1** میلیون سند است.

2. سطح دوم (رهبران محلی - **Local Leaders**): در داخل هر ابر-خوشه، سیستم **1000** سند را به عنوان رهبر محلی

انتخاب می‌کند. (مجموعاً در کل سیستم $1000 \times 1000 = 10^6$ رهبر محلی خواهیم داشت).

3. سطح سوم (پیروان - Followers): در هر ابر-خوشه، اسناد باقی‌مانده به نزدیک‌ترین رهبر محلی متصل می‌شوند.

بنابراین، به هر رهبر محلی دقیقاً 1000 سند پیرو متصل می‌گردد.

بخش 3) فرآیند اجرای پرسمان (Online Processing)

هنگامی که پرسمان کاربر دریافت می‌شود، سیستم در 3 گام و با سرعت خیره‌کننده‌ای فضای جستجو را هرس می‌کند:

1. گام اول (پیمایش سطح اول): پرسمان صرفاً با 1000 ابر-رهبر مقایسه می‌شود تا نزدیک‌ترین ابر-رهبر شناسایی

گردد. ← تعداد بررسی: 1000 گره

2. گام دوم (پیمایش سطح دوم): پرسمان به داخل ابر-خوشه منتخب ارسال شده و تنها با 1000 رهبر محلی موجود

در آن خوشه مقایسه می‌شود. ← تعداد بررسی: 1000 گره

3. گام سوم (پیمایش سطح سوم): پرسمان با 1000 پیرو متصل به رهبر محلی منتخب مقایسه می‌شود. ← تعداد

بررسی: 1000 گره

مجموع عملیات آنلاین: $1000 + 1000 + 1000 = 3000$ مقایسه.

بخش 4) اثبات ریاضی پیچیدگی زمانی (Time Complexity Proof)

برای یک معماری سلسله‌مراتبی 3 لایه، هزینه زمانی برابر با مجموع گره‌های پیمایش شده است. با استفاده از نامساوی حسابی-

هندسی (AM-GM Inequality) خواهیم داشت:

$$T = s_1 + s_2 + s_3 \geq 3\sqrt[3]{s_1 \cdot s_2 \cdot s_3} = 3\sqrt[3]{N}$$

بخش 5) نتیجه‌گیری نهایی

کمترین مقدار هزینه زمانی دقیقاً زمانی رخ می‌دهد که تساوی در رابطه AM-GM برقرار باشد (یعنی $s_1 = s_2 = s_3 = N^{1/3}$).

بنابراین، سیستم با ایجاد انشعابات برابر با ریشه سوم مجموعه، به بهینه‌ترین حالت محاسباتی خود می‌رسد و پیچیدگی زمانی فرآیند

جستجو از $O(N^{1/2})$ در هرس مقدماتی، با موفقیت به $O(N^{1/3})$ در معماری دو مرحله‌ای ارتقا می‌یابد.

پاسخنامه مسئله 7.23: پیاده‌سازی کد نمایه چندلایه (Tiered Index Code Implementation)

بخش 1) مقدمه و تحلیل منطق الگوریتم

در پیاده‌سازی معماری نمایه‌های چندلایه، هدف اصلی بهینه‌سازی زمان اجرا و مدیریت هوشمندانه منابع است. زمانی که

الگوریتم برای یافتن K سند برتر به لایه اول (Tier 1) مراجعه می‌کند، ممکن است عملیات اشتراک‌گیری (Intersection) تعداد

اسنادی کمتر از K تولید کند. در این سناریو (Fallback)، چالش مهندسی این است که بدون تکرار پردازش‌های موفق لایه اول،

کمبود ظرفیت از لایه دوم (Tier 2) جبران شود.

از منظر تئوری مجموعه‌ها، اگر پرسمان شامل کلمات q_1 و q_2 باشد، اسناد لایه اول برابر است با:

$$T1_{q_1} \cap T1_{q_2}$$

اگر این مقدار کافی نباشد، ما به دنبال اسنادی هستیم که در مجموعه کل یعنی رابطه زیر حضور دارند:

$$(T1_{q_1} \cup T2_{q_1}) \cap (T1_{q_2} \cup T2_{q_2})$$

برای حفظ دستاوردهای لایه اول، مجموعه یافت‌شده در مرحله اول را از خروجی نهایی کسر کرده و تنها اسناد جدید را به انتهای لیست اضافه می‌کنیم.

بخش 2) پیاده‌سازی

```
def get_top_k_tiered(query_terms, k_limit, tier1_idx, tier2_idx):  
    """  
    Retrieves top K documents using a two-tier index architecture.  
    Ensures Tier 1 processing is preserved during Tier 2 fallback.  
    """  
  
    if not query_terms or k_limit <= 0:  
        return []  
  
    # Step 1: Process Tier 1 exclusively  
    t1_sets = []  
    for term in query_terms:  
        # Fetch posting list for each term in Tier 1; default to empty set  
        term_docs = set(tier1_idx.get(term, []))  
        t1_sets.append(term_docs)  
  
    # Perform intersection for Tier 1  
    t1_result = set.intersection(*t1_sets) if t1_sets else set()  
  
    # Early termination if Tier 1 satisfies the K requirement  
    if len(t1_result) >= k_limit:  
        return list(t1_result)[:k_limit]  
  
    # Step 2: Fallback to Tier 2 (Cascading)  
    # Combine Tier 1 and Tier 2 posting lists logically using Union  
    combined_sets = []  
    for term in query_terms:  
        t1_docs = set(tier1_idx.get(term, []))  
        t2_docs = set(tier2_idx.get(term, []))  
        # Logical union of both tiers for the specific term  
        combined_sets.append(t1_docs | t2_docs)  
  
    # Perform full intersection on the combined logical index  
    full_result = set.intersection(*combined_sets) if combined_sets else set()  
  
    # Isolate newly discovered documents to preserve Tier 1 processing  
    new_candidate_docs = full_result - t1_result  
  
    # Step 3: Merge and extract Top K  
    final_docs = list(t1_result) + list(new_candidate_docs)  
  
    return final_docs[:k_limit]
```

بخش 3) تحلیل گام‌به‌گام و پیچیدگی زمانی (Time Complexity)

1. گام اول (پردازش اختصاصی لایه اول):

تابع ابتدا تلاش می‌کند تمام کلمات پرسمان را منحصراً در `tier1_idx` پیدا کند. استفاده از ساختار داده `set` در پایتون، عملیات اشتراک‌گیری `set.intersection` را با استفاده از توابع بهینه‌شده زبان **C** در پس‌زمینه اجرا می‌کند که پیچیدگی زمانی آن به طور متوسط $\mathcal{O}(\min(\text{len}(S_i)))$ است. اگر طول مجموعه `t1_result` بزرگتر یا مساوی **K** باشد، تابع با موفقیت در همین نقطه متوقف شده (**Early Termination**) و هزینه پردازش لایه دوم به طور کامل حذف می‌گردد.

2. گام دوم (سقوط به لایه دوم و ترکیب منطقی):

در صورتی که شرط توقف زودهنگام ارضا نشود، تابع وارد فاز ترکیب می‌شود. عملگر اجتماع $\cup$ ، لیست‌های پستی لایه اول و دوم را برای هر کلمه با یکدیگر ادغام می‌کند. نکته کلیدی مهندسی در دستور `new_candidate_docs = full_result - t1_result` نهفته است؛ این عملیات تفاضل مجموعه، تضمین می‌کند که اسناد ارزشمندی که پیش‌تر در لایه اول کشف و تایید شده‌اند، مجدداً ارزیابی یا جابجا نمی‌شوند و مستقیماً در صدر لیست نهایی جای می‌گیرند.

3. گام سوم (تکمیل ظرفیت):

در نهایت، اسناد یافت‌شده در لایه اول (که دارای بالاترین کیفیت ایستا یا اهمیت متنی هستند) در ابتدای لیست قرار گرفته و اسناد جدید استخراج‌شده از لایه دوم به انتهای آن متصل می‌شوند تا ظرفیت عدد K با حفظ بالاترین دقت (**Precision**) تکمیل گردد.

پاسخنامه مسئله 7.24: چالش مرتب‌سازی تأثیر در حافظه محدود (کوانتیزه‌کردن وزن‌ها)

بخش 1) مقدمه و تبیین گلوگاه حافظه

در پیاده‌سازی تکنیک «مرتب‌سازی بر اساس تأثیر» (**Impact Ordering**)، ذخیره‌سازی مقادیر دقیق و اعشاری وزن‌ها (مانند **tf-idf** که معمولاً نیازمند 32 یا 64 بیت حافظه به ازای هر رکورد است)، در سیستم‌های کلان به سرعت منجر به پر شدن حافظه نهان (**Cache**) و حافظه اصلی (**RAM**) می‌شود. برای غلبه بر این چالش، از تکنیک **کوانتیزه‌کردن (Quantization)** استفاده می‌شود. این تکنیک با تبدیل مقادیر پیوسته اعشاری به مقادیر گسسته و صحیح، نیاز به حافظه را به 3 الی 4 بیت تقلیل می‌دهد، در حالی که ساختار کلی رتبه‌بندی را حفظ می‌کند.

بخش 2) بخش اول: معماری ساخت بلوک‌های کوانتیزه‌شده (Index Construction)

برای تبدیل لیست‌های پستی استاندارد به لیست‌های بلوکی و کوانتیزه‌شده در فاز پیش‌پردازش (**Offline Phase**)، سیستم مراحل زیر را طی می‌کند:

1. نگاشت و گسسته‌سازی مقادیر (Score Mapping):

ابتدا وزن دقیق **tf-idf** برای تمام جفت‌های (سند-واژه) محاسبه می‌شود. سپس سیستم کل بازه وزن‌ها را به L سطح یا بازه مجزا تقسیم می‌کند (در این مسئله $L = 5$). هر وزن اعشاری به یک عدد صحیح $v \in \{1, 2, 3, 4, 5\}$ نگاشت می‌شود. سطح 5 به معنای بیشترین تأثیر (**Highest Impact**) و سطح 1 به معنای کمترین تأثیر است.

2. ساختاردهی بلوکی لیست‌های پستی (Block-based Postings):

لیست پستی مربوط به هر واژه (t)، به جای یک لیست یکپارچه، به 5 «بلوک مجزا» (B_5, B_4, B_3, B_2, B_1) شکسته می‌شود:

بلوک 5 (B_5): شامل شناسه تمام اسنادی است که وزن آن‌ها در این واژه کوانتیزه شده و برابر 5 است.

بلوک 4 (B_4): شامل شناسه اسنادی است که وزن آن‌ها 4 است، و به همین ترتیب تا بلوک 1.

3. مزیت کلیدی معماری (مرتب‌سازی درون‌بلوکی):

مهم‌ترین دستاورد این معماری این است که چون تمام اسناد درون یک بلوک (مثلاً B_4) دارای ارزش (وزن) یکسانی هستند، دیگر نیازی به مرتب‌سازی نزولی بر اساس وزن درون آن بلوک نیست. بنابراین، سیستم شناسه اسناد (DocIDs) درون هر بلوک را به صورت صعودی مرتب می‌کند. این کار به موتور جستجو اجازه می‌دهد تا عملیات اشتراک‌گیری (Intersection) بسیار سریع را که در مرتب‌سازی تأثیر خالص از دست رفته بود، مجدداً به دست آورد.

بخش 3) الگوریتم پیمایش برای پرسمان‌های چندکلمه‌ای (Traversal Algorithm)

برای پرسمان‌های چندکلمه‌ای، الگوریتم به جای پیمایش سند-به-سند (DAAT)، از رویکرد پیمایش سطح-به-سطح (Impact-at-a-Time) استفاده می‌کند تا بهینه‌ترین استفاده را از ساختار بلوکی ببرد. مراحل الگوریتم در زمان اجرا به این شرح است:

1. مقداردهی اولیه (Initialization):

سیستم یک صف اولویت (Min-Heap) برای نگهداری K سند برتر ایجاد می‌کند و کمترین امتیاز در این صف را به عنوان آستانه (S_{min}) در نظر می‌گیرد. همچنین یک ساختار «جمع‌کننده» (Accumulator) در حافظه کش برای ردیابی امتیازات اسنادی که در حال پردازش هستند، تخصیص می‌یابد.

2. پردازش نزولی بلوک‌ها (Processing by Impact Level):

الگوریتم پردازش را از بالاترین سطح تأثیر (یعنی سطح $v = 5$) برای تمام کلمات پرسمان آغاز می‌کند.

سیستم بلوک‌های B_5 مربوط به کلمات پرسمان را می‌خواند. اگر سندی در این بلوک‌ها مشاهده شد، مقدار 5 به امتیاز آن سند در جمع‌کننده اضافه می‌شود.

پس از اتمام سطح 5، سیستم به سطح 4 ($v = 4$) می‌رود و بلوک‌های B_4 را می‌خواند و امتیازات را به‌روزرسانی می‌کند.

3. محاسبه کران بالا و توقف زودهنگام (Upper Bound & Early Termination):

پس از پردازش هر سطح (مثلاً پس از اتمام سطح 4 و قبل از ورود به سطح 3)، الگوریتم محاسبه می‌کند که در بهترین حالت، یک سند پردازش‌نشده (سندی که تا این لحظه در هیچ‌یک از بلوک‌های 5 و 4 دیده نشده است) چه امتیازی می‌تواند کسب کند.

اگر پرسمان دارای 3 کلمه باشد و سیستم در حال ورود به سطح 3 باشد، کران بالا (UB) برای اسناد کاملاً جدید برابر است با:

$$UB = |q| \times v_{\text{current}} = 3 \times 3 = 9$$

(یعنی در خوش‌بینانه‌ترین حالت، سند جدید در هر سه کلمه وجود دارد و بالاترین وزن باقیمانده یعنی 3 را کسب می‌کند).

شرط توقف: سیستم مقدار UB را با S_{min} (پایین‌ترین امتیاز در میان K سند برتر فعلی) مقایسه می‌کند. به محض اینکه رابطه $UB \leq S_{min}$ برقرار شود، الگوریتم بلافاصله متوقف می‌گردد (Early Termination).

بخش 4) نتیجه‌گیری ساختاری

این الگوریتم تضمین می‌کند که اسناد دارای بالاترین ارزش کیفی (اسناد موجود در بلوک‌های 5 و 4) در همان مراحل اولیه کشف و امتیازدهی می‌شوند. ترکیب «کوانتیزه کردن وزن‌ها»، «مرتب‌سازی صعودی درون‌بلوکی» و «توقف زودهنگام مبتنی بر کران بالا»، به موتور جستجو اجازه می‌دهد با حداقل استفاده از حافظه کش، پرسرمان‌های چندکلمه‌ای را با دقتی معادل (و سرعتی به مراتب بیشتر از) سیستم‌های مبتنی بر وزن‌های اعشاری پاسخ دهد.

پاسخنامه مسئله 7.25: مدیریت تداخل توابع امتیازدهی و نرمال‌سازی ویژگی‌ها در مدل‌های یادگیری ماشین (Feature Normalization in ML Scoring)

بخش 1) مقدمه و تحلیل ریشه مشکل (Root Cause Analysis)

در سیستم‌های رتبه‌بندی مبتنی بر یادگیری ماشین (Learning to Rank - LTR)، مدل‌ها تمایل دارند به ویژگی‌هایی (Features) که واریانس بالا و همبستگی مستقیمی با خروجی مورد انتظار دارند، وزن بیشتری اختصاص دهند. اضافه شدن متغیر خام «تعداد کلیک» (Raw Click Count) باعث ایجاد پدیده انحراف به سمت طعمه کلیک (Clickbait Bias) شده است.

ریشه مشکل در 2 خطای مدل‌سازی نهفته است: اولاً، تعداد کلیک خام به صورت نمایی رشد می‌کند و اثر ارتباط متنی (Vector Similarity) را می‌بلعد. ثانیاً، مدل فاقد متغیری است که «رضایت پس از کلیک» را بسنجد؛ در نتیجه، مقالات کوتاه و زرد که عناوین جذابی دارند، به اشتباه به عنوان اسناد معتبر طبقه‌بندی می‌شوند.

برای حل این بحران و ایجاد تعادل میان طول سند، اعتبار علمی و تعامل کاربر، تابع امتیازدهی باید طی 4 گام مهندسی زیر اصلاح و نرمال‌سازی شود:

بخش 2) گام اول: نرمال‌سازی لگاریتمی متغیر کلیک (Click Feature Normalization)

مقادیر خام کلیک دارای توزیع دُم‌دار (Heavy-tailed) هستند. سندی با 100,000 کلیک لزوماً 100 برابر مرتبط‌تر از سندی با 1,000 کلیک نیست. برای جلوگیری از تسلط این متغیر بر تابع امتیازدهی، باید از هموارسازی لگاریتمی (Logarithmic Smoothing) استفاده کرد.

متغیر کلیک خام در مدل یادگیری ماشین باید با فرمول زیر جایگزین شود:

$$\text{Normalized_Clicks} = \log(1 + \text{Raw_Clicks})$$

اضافه کردن عدد 1 تضمین می‌کند که برای اسناد بدون کلیک، مقدار لگاریتم 0 شود (جلوگیری از خطای لگاریتم 0). این تغییر، اثر کلیک‌های نجومی را مهار کرده و آن را به یک مقیاس خطی و کنترل‌شده تبدیل می‌کند.

بخش 3) گام دوم: ارتقای جریمه طول سند با مدل BM25

در مقالات علمی و طولانی، تکرار کلمات به طور طبیعی بیشتر است، اما اگر سیستم از نرمال‌سازی طول نامناسب استفاده کند، اسناد طولانی جریمه می‌شوند. برای رفع این مشکل، باید فاکتور ارتباط متنی در مدل ماشین لرنینگ را بر اساس مکانیزم نرمال‌سازی طول در الگوریتم BM25 تنظیم کرد.

جریمه طول سند (L_d) در این مدل با فرمول زیر کنترل می‌شود:

$$L_d = (1 - b) + b \cdot \frac{|d|}{\text{avg_dl}}$$

پارامتر $|d|$ طول سند فعلی و avg_dl میانگین طول اسناد در کل مجموعه است.

اقدام اصلاحی: با کاهش دادن پارامتر تنظیم‌کننده b (مثلاً تنظیم آن روی $b = 0.3$)، سیستم جریمه کمتری برای مقالات طولانی و علمی در نظر می‌گیرد و به آن‌ها اجازه می‌دهد با اسناد کوتاه رقابت کنند.

بخش 4) گام سوم: معرفی متغیر زمان توقف به عنوان عامل تعادل (Dwell Time / Reading Time)

کلیک کردن صرفاً نشان‌دهنده «جذابیت عنوان» است، نه «کیفیت محتوا». برای شناسایی اسناد **Clickbait**، باید متغیر جدیدی به نام «زمان توقف» (**Dwell Time**) که نشان‌دهنده مدت زمان حضور کاربر در صفحه است، به مدل اضافه شود. برای اینکه این متغیر عادلانه باشد، باید نسبت به طول سند نرمال‌سازی شود تا نرخ واقعی مطالعه (**Reading Rate**) به دست آید:

$$\text{Reading_Rate} = \frac{\text{Dwell_Time}}{|d|}$$

مقالات علمی طولانی، زمان توقف بسیار بالایی دارند که نشان‌دهنده رضایت کاربر و مطالعه عمیق است، در حالی که اسناد زرد و کوتاه دارای نرخ پرش (**Bounce Rate**) بالا و زمان توقف نزدیک به 0 هستند.

بخش 5) گام چهارم: بازنویسی تابع امتیازدهی ترکیبی (Restructured Scoring Function)

اکنون مدل یادگیری ماشین با استفاده از رگرسیون یا درخت‌های تصمیم، تابع نهایی (NetScore) را بر اساس ترکیب خطی (یا غیرخطی) متغیرهای نرمال‌شده بازنویسی می‌کند:

$$\text{NetScore} = w_1 \cdot \text{Text_Relevance} + w_2 \cdot \log(1 + \text{Clicks}) + w_3 \cdot \text{Reading_Rate}$$

بخش 6) نتیجه‌گیری ساختاری

با اعمال این اصلاحات، پدیده **Clickbait** به طور کامل خنثی می‌شود. مقالات زرد کوتاه، اگرچه ممکن است در فاکتور دوم ($\log(\text{Clicks})$) امتیاز بگیرند، اما در فاکتور سوم (زمان توقف نرمال‌شده) و فاکتور اول (ارتباط متنی غنی) به شدت شکست می‌خورند. در مقابل، مقالات علمی و طولانی به دلیل داشتن کلمات کلیدی غنی، زمان توقف بالا و جریمه طول کنترل‌شده، مجدداً به صدر نتایج بازمی‌گردند و تعادل میان «تعامل کاربر» و «کیفیت علمی» در موتور جستجو برقرار می‌گردد.

پاسخنامه مسئله 7.26: بهینه‌سازی محاسبات مجاورت عبارات

(Optimizing Term Proximity Computations)

بخش 1) مقدمه و تحلیل پیچیدگی

در نمایه‌های موقعیتی (**Positional Indexes**)، لیست موقعیت‌های یک کلمه در یک سند، آرایه‌ای مرتب‌شده از اعداد صحیح است که جایگاه (**Offset**) آن کلمه را در متن نشان می‌دهد. اگر پرسمان شامل 3 کلمه باشد و ما از روش «جستجوی جامع» (**Brute-force**) برای یافتن تمام ترکیبات ممکن استفاده کنیم، پیچیدگی زمانی برابر با حاصل ضرب طول لیست‌ها، یعنی $O(|P_1| \times |P_2| \times |P_3|)$ خواهد بود که برای اسناد طولانی به شدت ناکارآمد است.

برای رساندن این زمان محاسباتی به مرتبه خطی $O(L)$ (که در آن $L = |P_1| + |P_2| + |P_3|$ است)، از «الگوریتم پیمایش چندنشانگری» (**Multi-pointer Traversal Algorithm**) یا تکنیک پنجره لغزان مبتنی بر اشاره‌گر استفاده می‌کنیم.

بخش 2) طراحی الگوریتم (مبتنی بر 3 نشانگر)

فرض کنید 3 لیست موقعیت مرتب‌شده به نام‌های P_1 و P_2 و P_3 برای 3 کلمه پرسمان در یک سند خاص در اختیار داریم.

گام اول: مقداردهی اولیه (Initialization)

3 نشانگر (**Pointer**) به نام‌های ptr_1 و ptr_2 و ptr_3 ایجاد کرده و همه آن‌ها را روی عدد 0 (اشاره به اولین عنصر هر لیست) تنظیم می‌کنیم.

1 متغیر برای نگهداری اندازه کوچکترین پنجره به نام `min_window_size` تعریف کرده و مقدار اولیه آن را بی‌نهایت (∞) در نظر می‌گیریم.

گام دوم: حلقه پیمایش و یافتن کران‌های پنجره

الگوریتم وارد یک حلقه تکرار (**Loop**) می‌شود که تا رسیدن به انتهای حداقل یکی از لیست‌ها ادامه می‌یابد. در هر تکرار، مراحل زیر انجام می‌شود:

1. **شناسایی مرزها:** اعدادی که 3 نشانگر در حال حاضر به آن‌ها اشاره می‌کنند را می‌خوانیم. کمترین مقدار را Min_{pos} و بیشترین مقدار را Max_{pos} می‌نامیم.

نکته منطقی: Min_{pos} موقعیت اولین کلمه و Max_{pos} موقعیت آخرین کلمه در پنجره فعلی است.

2. **محاسبه طول پنجره:** اندازه پنجره فعلی با استفاده از فرمول زیر محاسبه می‌شود:

$$\text{Current Window} = Max_{pos} - Min_{pos} + 1$$

3. **به‌روزرسانی کوچکترین پنجره:** اگر **Current Window** از `min_window_size` کوچکتر باشد، مقدار `min_window_size` را با این عدد جدید جایگزین می‌کنیم.

گام سوم: استراتژی حرکت نشانگرها

برای اینکه در تکرار بعدی حلقه، شانس یافتن پنجره‌ای کوچکتر را داشته باشیم، باید فاصله بین Min_{pos} و Max_{pos} را کاهش دهیم.

قانون حرکت: تنها نشانگری که می‌تواند پنجره را کوچکتر کند، نشانگری است که به Min_{pos} (کوچکترین عدد) اشاره دارد. اگر نشانگر Max_{pos} را جلو ببریم، پنجره قطعاً بزرگتر می‌شود.

بنابراین، الگوریتم به سادگی بررسی می‌کند که کدام لیست شامل مقدار Min_{pos} است، و نشانگر همان لیست را 1 واحد به جلو ($Position + 1$) حرکت می‌دهد.

گام چهارم: شرط توقف (Termination)

فرآیند فوق تا زمانی ادامه می‌یابد که یکی از نشانگرها از مرز لیست موقعیت خود خارج شود (**Out of Bounds**). به محض رسیدن یکی از نشانگرها به انتهای لیست، جستجو متوقف شده و مقدار `min_window_size` به عنوان خروجی نهایی بازگردانده می‌شود.

بخش 3) اثبات ریاضی پیچیدگی زمانی $O(L)$

برای اثبات اینکه الگوریتم طراحی‌شده دقیقاً دارای پیچیدگی خطی $O(L)$ است، به رفتار نشانگرها در حلقه تکرار دقت می‌کنیم:

در هر بار اجرای حلقه، الگوریتم مقدار مینیمم و ماکزیمم بین 3 عدد را پیدا می‌کند که به زمان ثابت $O(1)$ نیاز دارد.

در انتهای هر تکرار، دقیقاً 1 نشانگر (نشانگر کمترین مقدار) یک گام به جلو حرکت می‌کند.

از آنجایی که نشانگرها هرگز به عقب برنمی‌گردند، حداکثر تعداد دفعاتی که می‌توان نشانگرها را به جلو حرکت داد، برابر است با مجموع تعداد عناصر موجود در هر 3 لیست.

اگر طول لیست‌ها را به ترتیب $|P_1|$ و $|P_2|$ و $|P_3|$ بنامیم، حلقه نهایتاً پس از $L = |P_1| + |P_2| + |P_3|$ بار تکرار متوقف خواهد شد.

بخش 4) نتیجه‌گیری ساختاری

این الگوریتم ظریف و قدرتمند نشان می‌دهد که چگونه می‌توان با استفاده از ویژگی «مرتب‌بودن» لیست‌های موقعیت و هدایت هوشمندانه نشانگرها، یکی از سنگین‌ترین پردازش‌های موتورهای جستجو (محاسبه مجاورت عبارات) را از مرتبه نمایی/چندجمله‌ای به مرتبه کاملاً خطی کاهش داد، تا زمان پاسخ‌گویی موتور جستجو در پرسمان‌های پیچیده حفظ گردد.

پاسخنامه مسئله 7.27: حل مشکل هرس خوشه‌ای در داده‌های نامتوازن

(Balanced Cluster Pruning)

بخش 1) مقدمه و تحلیل بحران محاسباتی (Bottleneck Analysis)

الگوریتم هرس خوشه‌ای (**Cluster Pruning**) بر این فرض آماری استوار است که پیروان (**Followers**) به صورت نسبتاً یکنواخت میان رهبران (**Leaders**) توزیع می‌شوند. در یک مجموعه با N سند و $\sqrt{N}$ رهبر، امید ریاضی تعداد پیروان هر رهبر برابر با $\sqrt{N}$ است. این توزیع متقارن، پیچیدگی زمانی پرسمان را در مرتبه اندیکاتور $O(\sqrt{N})$ تضمین می‌کند.

هنگامی که توزیع داده‌ها به شدت چوله (**Skewed**) باشد (تجمع 80% اسناد حول 5% از رهبران)، ساختار سیستم با 2 بحران اساسی مواجه می‌گردد:

فروپاشی پیچیدگی زمانی: اگر پرسمان کاربر به یکی از این «رهبران کلان» (**Mega-leaders**) شباهت داشته باشد،

سیستم مجبور است پرسمان را با $0.8N$ سند پیرو مقایسه کند. در این حالت، زمان اجرا از مرتبه $O(\sqrt{N})$ به مرتبه

خطی $O(N)$ میل می‌کند که دقیقاً به معنای از دست رفتن تمام مزایای هرس خوشه‌ای است.

واریانس شدید تأخیر (Latency Variance): برخی پرسمان‌ها (متصل به رهبران کوچک) در چند میلی‌ثانیه پاسخ داده می‌شوند و برخی دیگر (متصل به رهبران کلان) ثانیه‌ها طول می‌کشند. این عدم پایداری، کیفیت خدمات (QoS) موتور جستجو را نقض می‌کند.

بخش 2) مکانیزم‌های ریاضی برای تخصیص مجدد و توازن بخشی (Balancing Mechanisms)

برای حل این بحران، نمی‌توان اسناد را به صورت تصادفی به رهبران خلوت‌تر منتقل کرد، زیرا این کار باعث اتصال سند به رهبر نامرتب شده و دقت جستجو (Precision) را به شدت کاهش می‌دهد. در طراحی الگوریتم‌های استاندارد، از 2 مکانیزم ریاضی زیر برای بهینه‌سازی توأمان «تراکم» و «شباهت» استفاده می‌شود:

راهکار اول: تخصیص مبتنی بر جریمه تراکم (Regularized Similarity Function)

در این روش، تابع شباهت کسینوسی در زمان ساخت نمایه (Offline Phase) با یک «ترم منظم‌سازی» (Regularization Term) ترکیب می‌شود. هدف این است که اتصال به رهبران شلوغ، هزینه ریاضی داشته باشد. تابع امتیازدهی برای اتصال سند d به رهبر L_i به شکل زیر بازنویسی می‌شود:

$$\text{Score}(d, L_i) = \cos(\vec{d}, \vec{L}_i) - \lambda \frac{|C_i|}{C_{max}}$$

متغیر $\cos(\vec{d}, \vec{L}_i)$: شباهت موضوعی و متنی میان سند و رهبر.

متغیر $|C_i|$: تعداد پیروانی که تا این لحظه به رهبر i متصل شده‌اند.

متغیر C_{max} : میانگین یا ظرفیت ایده‌آل یک خوشه (مثلاً $\sqrt{N}$).

پارامتر λ : ضریب تنظیم (Tuning Parameter).

تحلیل عملکرد: با هر سندی که به یک رهبر اضافه می‌شود، مقدار $|C_i|$ افزایش یافته و جریمه سنگین‌تری اعمال می‌گردد. در نتیجه، سند بعدی به جای اتصال به این رهبر اشباع‌شده، به «دومین رهبر شبیه خود» که خلوت‌تر است، متمایل می‌شود. این مکانیزم تعادل را به صورت پیوسته (Soft Clustering) برقرار می‌کند.

راهکار دوم: کران‌بندی ظرفیت با صف اولویت (Capacity Bounding with Priority Queues)

در این روش، یک کران بالای سخت (Hard Limit) برای حداکثر تعداد پیروان هر خوشه تعریف می‌گردد.

$$C_{max} = \alpha \sqrt{N}$$

ضریب α (مثلاً $\alpha = 1.5$) میزان انعطاف‌پذیری سیستم را نشان می‌دهد. مراحل اجرای این الگوریتم به شرح زیر است:

- در فاز پیش‌پردازش، شباهت تک‌تک اسناد با تمام رهبران محاسبه شده و هر سند دارای یک صف اولویت (Min-Heap) از نزدیک‌ترین رهبران خود است.
- اسناد به صورت حریصانه (Greedy) به شبیه‌ترین رهبر (اولویت اول) متصل می‌شوند.
- به محض اینکه تعداد پیروان یک رهبر به مرز C_{max} رسید، آن خوشه «قفل» (Locked) می‌شود.

4. اسنادی که پس از قفل شدن، قصد ورود به آن خوشه را دارند، به صورت خودکار به رهبر موجود در «اولویت دوم» یا «اولویت سوم» خود در صف ارجاع داده می‌شوند.

بخش 3) نتیجه‌گیری ساختاری

استفاده از مکانیزم جریمه تراکم یا کران‌بندی ظرفیت، یک مسئله بهینه‌سازی صلب را به یک مدل توزیع هوشمند تبدیل می‌کند. در این ساختارهای ریاضی، اگر سندی نتواند به شبیه‌ترین رهبر خود متصل شود، به جای رها شدن در یک خوشه نامرتب، به دومین یا سومین رهبر شبیه به خود (که همچنان ارتباط معنایی بالایی با سند دارد) متصل می‌گردد. این استراتژی، ضمن حفظ یکپارچگی معنایی داده‌ها، تضمین می‌کند که اندازه هیچ خوشه‌ای از حد آستانه فراتر نرفته و موتور جستجو همواره در زمان استاندارد $O(\sqrt{N})$ به پرسمان‌ها پاسخ خواهد داد.

پاسخنامه مسئله 7.28: معماری یکپارچه برای پرس‌وجوهای پیچیده

(Unified Architecture for Complex Queries)

بخش 1) مقدمه و منطق طراحی

برای پاسخ‌گویی به یک پرسمان ترکیبی و پیچیده (شامل عبارت دقیق، عملگرهای بولی و فیلتر متادیتا) در زمان کمتر از 100 میلی‌ثانیه، سیستم نمی‌تواند تمام این شروط را به صورت همزمان در مرحله رتبه‌بندی برداری محاسبه کند. کلید حل این مسئله، طراحی یک «پایپ‌لاین پردازشی آبشاری» (Cascading Pipeline) است. در این معماری، فیلترهای قطعی و ارزان‌قیمت در ابتدای مسیر قرار می‌گیرند تا فضای جستجو را به شدت هرس کنند و محاسبات گران‌قیمت (مانند شباهت کسینوسی و مجاورت) تنها روی یک زیرمجموعه بسیار کوچک و فیلترشده اعمال گردد.

بخش 2) نمودار جریان داده (Data Flow Diagram)

ساختار آبشاری سیستم به صورت شماتیک در 5 فاز متوالی زیر طراحی می‌شود:

[دریافت پرسمان کاربر]



فاز 1: پردازشگر پرسمان و حذف نمایه (Query Parser & Index Elimination)

(شناسایی توکن‌ها، حذف واژگان بی‌ارزش، استخراج شروط)



فاز 2: فیلتر فراداده / متادیتا (Metadata Filtering)

(اعمال شرط $date > 2023$ روی ساختار $B-Tree$ و تولید ماسک بیتی)



فاز 3: فیلتر بولی روی نمایه معکوس (Boolean Retrieval)

(اشتراک و اجتماع روی لیست‌های پستی $python$ ، R ، $machine$ ، $learning$)



فاز 4: ارزیابی مجاورت و عبارت (Positional / Phrase Check)

(بررسی مجاورت *machine* و *learning* تنها برای اسناد باقی‌مانده)



فاز 5: تابع امتیازدهی نهایی (Final Scoring Function)

(رتبه‌بندی برداری و محاسبه *Top-K* روی خروجی فاز 4)



[ارسال نتایج به کاربر]

بخش 3) تشریح دقیق جایگاه و عملکرد کامپوننت‌های درخواستی

دانشجویان باید دقت کنند که ترتیب اجرای این مراحل، تضمین‌کننده تأخیر (Latency) زیر 100 میلی‌ثانیه است:

1. جایگاه و نقش حذف نمایه (Index Elimination)

محل اجرا: بلافاصله پس از دریافت پرسمان (فاز 1).

نحوه عملکرد: پردازشگر پرسمان، تک‌تک کلمات را ارزیابی می‌کند. در پرسمان فعلی ("machine learning" AND (python OR R))، تمامی کلمات کلیدی تخصصی هستند و فراوانی معکوس سند (idf) بالایی در پایگاه داده‌های علمی دارند (حتی حرف "R" در این دامنه یک موجودیت باارزش است و یک Stop-word محسوب نمی‌شود).

تصمیم‌الگوریتم: سیستم تشخیص می‌دهد که این پرسمان فاقد کلمات عمومی (مانند in, the, with) است و طول آن نیز کوتاه است. بنابراین، الگوریتم حذف نمایه در اینجا بای‌پس (Bypass) می‌شود و هیچ کلمه‌ای را حذف نمی‌کند تا بار معنایی پرسمان‌های تخصصی دچار آسیب نشود. (در صورت وجود کلمه‌ای مثل "using"، در همین مرحله پیش از ورود به دیسک حذف می‌شد).

2. جایگاه فیلتر بولین روی Inverted Index

محل اجرا: فاز 2 و 3 (پیش از هرگونه محاسبه ریاضیاتی وزن‌ها).

نحوه عملکرد (تکنیک Bit-masking):

1. ابتدا شرط **date > 2023** روی یک نمایه ساختاریافته (مثل B-Tree) اجرا می‌شود. خروجی آن، یک آرایه بیتی (Bit-mask) از شناسه اسنادی است که در بازه زمانی مجاز قرار دارند. این عملیات در کسری از میلی‌ثانیه انجام می‌شود.

2. سیستم لیست‌های پستی کلمات **python** و **R** را واکنشی کرده و عملگر اجتماع (OR) را با مرتبه زمانی خطی روی آن‌ها اجرا می‌کند.

3. سپس سیستم عملگر اشتراک (AND) را میان سه مجموعه اجرا می‌کند:

[لیست پستی machine] ∩ [لیست پستی learning] ∩ [خروجی python OR R] ∩ [ماسک بیتی تاریخ]

دستاورد: سیستم بدون محاسبه حتی یک فرمول **tf-idf**، میلیون‌ها سند نامرتبط یا تاریخ‌گذشته را دور می‌ریزد و مجموعه نامزدهای جستجو ($D_{filtered}$) را به چند هزار یا چند صد سند قطعی محدود می‌کند.

3. جایگاه اجرای تابع امتیازدهی نهایی (Final Scoring Function)

محل اجرا: فاز 5 (آخرین مرحله پایپ‌لاین).

زیرمجموعه هدف: تابع امتیازدهی نهایی به هیچ‌وجه روی کل پایگاه داده اجرا نمی‌شود. این تابع منحصراً روی زیرمجموعه‌ی بسیار کوچکی اجرا می‌شود که توانسته‌اند از فیلتر بولی (فاز 3) و فیلتر عبارت دقیق (فاز 4 - تأیید اینکه **machine** دقیقاً قبل از **learning** آمده است) با موفقیت عبور کنند.

نحوه عملکرد: اکنون برای این چند صد سند باقی‌مانده، سیستم وزن‌های **tf-idf** (یا الگوریتم **BM25**) را با امتیازات کیفیت ایستا (مانند **PageRank** یا تعداد ارجاعات) ترکیب کرده و ضرب نقطه‌ای (**Dot Product**) را محاسبه می‌کند.

خروجی: اسناد بر اساس امتیاز نهایی مرتب شده و **K** سند برتر به کاربر نمایش داده می‌شوند.

بخش 4) نتیجه‌گیری برای پاسخنامه

جدا کردن فیلترهای صلب (تاریخ و بولین) از فیلترهای نرم (امتیازدهی کسینوسی)، شالوده معماری سیستم‌های توزیع‌شده است. با این معماری ترکیبی، بار پردازشی (**CPU Cycles**) و خواندن از دیسک (**Disk I/O**) به حداقل مطلق می‌رسد و سیستم می‌تواند سنگین‌ترین پرسman‌های تحلیلی را در کمتر از 100 میلی‌ثانیه پردازش و رتبه‌بندی کند.

پاسخنامه مسئله 7.29: تحلیل پیچیدگی زمانی ترکیب روش‌ها

(Time Complexity Analysis of Combined Methods)

بخش 1) مقدمه و تعریف متغیرهای تحلیلی

برای ارائه یک اثبات ریاضی دقیق، ابتدا باید متغیرهای درگیر در زمان اجرای سیستم بازیابی اطلاعات را مدل‌سازی کنیم:

N : تعداد کل اسناد موجود در پایگاه داده.

q : تعداد کلمات موجود در پرسman کاربر.

K : تعداد اسناد برتر مورد نیاز برای نمایش به کاربر (**Top-K**).

r : طول ثابت لیست قهرمان (**Champion List**) به ازای هر واژه.

q' : تعداد کلمات پرسman که پس از عبور از فیلتر آستانه فراوانی (**IDF Threshold**) باقی می‌مانند ($q' \leq q$).

هدف این است که نشان دهیم چگونه اعمال همزمان این دو تکنیک، وابستگی سیستم به متغیر سنگین N را در فاز اجرای برخط (**Online**) از بین می‌برد.

بخش 2) گام اول: تحلیل پیچیدگی زمانی در فضای برداری استاندارد (Standard VSM)

در یک معماری استاندارد مبتنی بر نمایه معکوس (بدون هیچ‌گونه هرس)، در بدترین حالت (**Worst-case Scenario**)، کلمات پرسman متداول هستند و هر کلمه در تمامی N سند مجموعه حضور دارد. فرآیند بازیابی استاندارد شامل دو بخش است:

1. **محاسبه امتیازات:** سیستم باید برای هر کلمه پرسمان، امتیاز شباهت کسینوسی را با تمام اسناد لیست پستی آن محاسبه و جمع (**Accumulate**) کند. این عملیات نیازمند $q \cdot N$ ضرب و جمع ریاضی است.

2. **استخراج Top-K:** پس از محاسبه امتیاز N سند نامزد، سیستم باید این امتیازات را در یک صف اولویت (**Min-Heap**) با ظرفیت K پردازش کند که هزینه آن برای N سند برابر $N \log K$ است.

بنابراین، معادله سقف عملیاتی در مدل پایه به شکل زیر است:

$$T_{\text{standard}} = \mathcal{O}(q \cdot N + N \log K)$$

بخش 3) گام دوم: استخراج معادله بدترین حالت در سیستم ترکیبی (Combined System)

اکنون معماری جدید را که مجهز به دو فیلتر متوالی است، ارزیابی می‌کنیم:

فاز 1: اثر هرس مبتنی بر آستانه (Index Elimination)

الگوریتم ابتدا کلماتی که دارای ارزش اطلاعاتی پایینی هستند (**IDF** کمتر از حد آستانه) را از پرسمان حذف می‌کند. در بدترین حالت ممکن، پرسمان کاربر تماماً از کلمات کاملاً تخصصی و نادر تشکیل شده است. در این شرایط هیچ کلمه‌ای حذف نمی‌شود و طول پرسمان پردازشی همان q باقی می‌ماند ($q' = q$).

فاز 2: اثر تحدید لیست‌های قهرمان (Champion Lists)

به جای پیمایش لیست‌های پستی به طول N ، سیستم منحصراً به لیست‌های قهرمان مراجعه می‌کند. طبق فرض مسئله، سقف طول هر لیست قهرمان برابر با r است. در بدترین حالت (یعنی زمانی که اشتراک لیست‌های قهرمان کلمات مختلف، تهی باشد)، سیستم مجبور است دقیقاً $q \cdot r$ سند کاملاً متمایز را واکشی کند.

فاز 3: محاسبه امتیاز و استخراج نهایی

هزینه محاسبه امتیاز: سیستم تنها برای این اسناد فیلترشده ضرب نقطه‌ای را محاسبه می‌کند. تعداد کل ارزیابی‌ها در بدترین حالت برابر است با $q \cdot r$.

هزینه مرتب‌سازی (Heap Operations): هر یک از این $q \cdot r$ سند نامزد، وارد صف اولویت (**Min-Heap**) با ظرفیت K می‌شوند. عملیات درج/به‌روزرسانی در این صف $\log K$ هزینه دارد.

با تجمیع این مقادیر، معادله نهایی سقف عملیات محاسباتی در بدترین حالت به این شکل تقلیل می‌یابد:

$$T_{\text{combined}} = \mathcal{O}(q \cdot r \log K)$$

بخش 4) مقایسه ریاضی و نتیجه‌گیری ساختاری (Mathematical Comparison)

برای درک عمق بهینه‌سازی، دو معادله به دست آمده را در قالب یک نسبت مجانبی (**Asymptotic Ratio**) مقایسه می‌کنیم:

$$\frac{T_{\text{standard}}}{T_{\text{combined}}} = \frac{\mathcal{O}(N(q + \log K))}{\mathcal{O}(q \cdot r \log K)} \sim \mathcal{O}\left(\frac{N}{r}\right)$$

در سیستم‌های مقیاس‌پذیر صنعتی، همواره متغیر N (ده‌ها میلیارد سند) به شدت بزرگتر از متغیرهای r (مثلاً 1000)، K (مثلاً 10) و q (مثلاً 3 تا 5) است $(N \gg r, K, q)$.

دست‌آورد بنیادین این اثبات این است که با ادغام «حذف نمایه» و «لیست قهرمان»، پیچیدگی زمانی موتور جستجو کاملاً از حجم داده‌ها (N) مستقل (Decoupled) می‌شود. در مدل استاندارد، با دو برابر شدن اسناد وب، زمان پاسخ‌گویی نیز دو برابر می‌گردد (رابطه خطی). اما در مدل ترکیبی، زمان پاسخ‌گویی به جای N ، توسط کران بالای ثابتی به نام r مهار شده است $(O(q \cdot r \log K))$. این استقلال ریاضی، دقیقاً همان معماری پنهانی است که به موتورهای جستجو اجازه می‌دهد با وجود رشد نمایی حجم اطلاعات، زمان پاسخ‌گویی برخط (Online Latency) را در محدوده میلی‌ثانیه تثبیت کنند.

پاسخنامه مسئله 7.30: شبیه‌سازی سیستم رتبه‌بندی نهایی (Final Ranking System Simulation)

بخش 1) مقدمه و تبیین فلسفه معماری ماژول

محاسبه مجاورت کلمات (Term Proximity) در نمایه‌های موقعیتی (Positional Indexes)، یکی از سنگین‌ترین و گران‌ترین پردازش‌ها از نظر چرخه‌های پردازنده (CPU Cycles) است، زیرا نیازمند پیمایش موازی آرایه‌های موقعیت با استفاده از چندین نشانگر فعال است. اگر سیستم بخواهد این محاسبات را برای تمام اسناد دربردارنده کلمات پرسمان انجام دهد، موتور جستجو دچار گلو‌سنگینی پردازنده و افزایش شدید تأخیر (Latency) می‌شود.

طراحی این ماژول بر پایه استراتژی «هرسِ آبشاری بار محاسباتی» (Cascading Cost Reduction) بنا شده است. در این پایپ‌لاین 3 مرحله‌ای، ماژول Scoring_Engine ابتدا فضای جستجو را با فیلترهای ارزان (بولین و لیست قهرمان) محدود کرده، سپس امتیاز متنی و ایستای میان‌رتبه را محاسبه می‌کند و در نهایت، پردازش سنگین مجاورت را منحصراً روی 30 سند برتر اعمال می‌نماید. این تفکیک معماری، کارایی محاسباتی سیستم را در مقیاس کلان تضمین می‌کند.

بخش 2) پیاده‌سازی: شبکه‌کد ماژول Scoring_Engine

```
scoring_engine.py

def compute_smallest_window(position_lists):
    """
    Helper function using the multi-pointer linear algorithm O(L)
    to find the smallest window containing all query terms.
    """
    pointers = [0] * len(position_lists)
    min_window = float('inf')

    while True:
        try:
            current_positions = [position_lists[i][pointers[i]] for i in range(len(position_lists))]
        except IndexError:
            break # Terminated when any pointer runs out of bounds

        min_pos = min(current_positions)
        max_pos = max(current_positions)
        current_window = max_pos - min_pos + 1

        if current_window < min_window:
            min_window = current_window

        # Move the pointer that points to the minimum position
        min_index = current_positions.index(min_pos)
        pointers[min_index] += 1

    return min_window

def Scoring_Engine(query_terms, docs_metadata, postings_lists, alpha=0.6, beta=0.3):
    """
    Advanced 3-Stage Ranking Pipeline to manage CPU consumption.
    Inputs:
    - query_terms: List[str]
```

```
- docs_metadata: Dict[doc_id, dict] -> contains 'pagerank'
- postings_lists: Dict[term, Dict[doc_id, List[int]]] -> Champion Lists with positions
Output:
- List of Top-10 Document IDs sorted by final score.
"""
if not query_terms:
    return []

# STAGE 1: Initial Screening (Boolean AND Intersection over Champion Lists)
candidate_docs = None
for term in query_terms:
    if term in postings_lists:
        # Extract doc_ids present in the term's Champion List
        term_champ_docs = set(postings_lists[term].keys())
        if candidate_docs is None:
            candidate_docs = term_champ_docs
        else:
            candidate_docs = candidate_docs.intersection(term_champ_docs)
    else:
        # If a query term doesn't exist in the corpus, intersection is empty
        return []

if not candidate_docs:
    return []

# STAGE 2: Net_Score Calculation (Vector Space Model + PageRank)

stage2_ranking = []

for doc_id in candidate_docs:
    # Calculate a simulated Cosine/TF score based on term frequencies
    cosine_sim = 0
    for term in query_terms:
        tf = len(postings_lists[term][doc_id])
        cosine_sim += tf # Simplified VSM summation for demonstration

    # Normalize cosine score simulation (Mock normalization between 0 and 1)
    cosine_sim_norm = min(cosine_sim / 20.0, 1.0)

    # Fetch Static Quality Score (PageRank)
    pagerank = docs_metadata.get(doc_id, {}).get('pagerank', 0.0)

    # Linear combination for Net_Score
    net_score = (alpha * cosine_sim_norm) + ((1 - alpha) * pagerank)
    stage2_ranking.append((doc_id, net_score))

# Sort candidates descending by Net_Score and prune to Top-30
stage2_ranking.sort(key=lambda x: x[1], reverse=True)
top_30_candidates = stage2_ranking[:30]

# STAGE 3: Proximity Boost (Executed strictly on Top-30 Survivors)
final_ranking = []

for doc_id, net_score in top_30_candidates:
    # Extract the position arrays for all query terms within this specific document
    term_positions_in_doc = [postings_lists[term][doc_id] for term in query_terms]

    # Compute the smallest proximity window using the O(L) algorithm
    smallest_window = compute_smallest_window(term_positions_in_doc)

    # Calculate Proximity Boost (Inverse relationship to window size)
    # Perfect proximity (window == len(query)) gives maximum boost
    q_len = len(query_terms)
    proximity_score = 1.0 / (smallest_window - q_len + 1) if smallest_window >= q_len else 0.0

    # Final blended score calculation
    final_score = net_score + (beta * proximity_score)
    final_ranking.append((doc_id, final_score))

# FINAL EXTREMA EXTRACTION: Top-10 Selection
final_ranking.sort(key=lambda x: x[1], reverse=True)
top_10_output = [doc_id for doc_id, score in final_ranking[:10]]

return top_10_output
```

بخش 3) تحلیل معماری پایپ‌لاین و اثبات مدیریت مصرف CPU

تحلیل کارایی هر یک از لایه‌های این پایپ‌لاین به شرح زیر تبیین می‌گردد تا دانشجویان با منطق توزیع بار آشنا شوند:

- 1. فاز اول (غربالگری بولی):** با فرض اینکه کل پایگاه داده شامل N سند باشد، لیست‌های قهرمان طول ثابتی برابر با r دارند ($r \ll N$). عملیات اشتراک‌گیری مجموعه در لایه اول، به جای کل اسناد دیسک، روی این ساختارهای کوچک انجام شده و فضای کاندیداها را در کسری از میلی‌ثانیه به عددی بسیار کوچک مهار می‌کند.

2. فاز دوم (محاسبه امتیاز خالص): محاسبات این فاز شامل عملیات‌های ریاضی ساده (ضرب و جمع خطی) است. سیستم امتیاز متنی را با امتیاز ایستای پیچ‌رنک (PR) که از قبل در فاز آفلاین محاسبه شده، ترکیب خطی می‌کند. خروجی این مرحله بر اساس یک مرتب‌سازی سریع، به دقیقاً 30 سند هرس می‌شود.

3. فاز سوم (مدیریت گلوگاه پروکسیمیتی): در این مرحله، الگوریتم سنگین نشانگری `compute_smallest_window` فراخوانی می‌شود. از آنجایی که این تابع منحصراً برای 30 سند برتر فاز قبل اجرا می‌شود، کل هزینه زمانی سیستم برابر با $30 \times O(L)$ خواهد بود (که L مجموع موقعیت کلمات پرسمان در این 30 سند است).

بخش 4) نتیجه‌گیری

اگر این ماژول فاقد فازبندی آبخاری بود و قرار بود امتیاز مجاورت را برای تمام اسناد حاوی کلمات پرسمان محاسبه کند، پیچیدگی زمانی سیستم تابعی از تعداد کل اسناد نامزد در وب می‌شد که به فلج شدن پردازنده می‌انجامید. این طراحی اثبات می‌کند که با مهار محاسبات سنگین در انتهای پایپ‌لاین، مصرف CPU به یک کران بالای ثابت و پایدار محدود شده و زمان پاسخ‌گویی به راحتی در مرز زیر 100 میلی‌ثانیه تثبیت می‌گردد.

پاسخنامه مسئله 7.31: معماری مرتب‌سازی تأثیر با امتیاز ایستا و توقف زودهنگام (Impact Ordering Architecture with Static Scoring and Early Termination)

بخش 1) مقدمه

در توسعه موتورهای جستجوی تجارت الکترونیک (E-commerce Search Engines)، چالش اصلی ایجاد تعادل میان محاسبه دقیق ارتباط معنایی و تضمین زمان پاسخ‌گویی (Latency) زیر 50 میلی‌ثانیه است. استفاده از تکنیک مرتب‌سازی تأثیر (Impact Ordering) یک راهکار معماری بسیار قدرتمند برای رسیدن به این هدف است. در ادامه، نحوه ادغام امتیازات و طراحی مکانیزم توقف زودهنگام را از منظر مهندسی سیستم بررسی می‌کنیم.

بخش 2) استراتژی مرتب‌سازی لیست‌های پستی (Index Construction)

در نمایه معکوس استاندارد، اسناد بر اساس شناسه سند (DocID) مرتب می‌شوند. اما در Impact Ordering، ما باید لیست را بر اساس «میزان تأثیرگذاری» مرتب کنیم. چالش اینجاست که ما 2 معیار داریم: یک معیار پویا وابسته به کلمه (tf-idf) و یک معیار ایستا مستقل از کلمه (Static Score) یا امتیاز فروش.

برای جلوگیری از سرشار محاسباتی در زمان اجرای پرسمان (Query-time)، این دو معیار باید در فاز پیش‌پردازش (Offline Phase) با یکدیگر ترکیب شوند.

مکانیزم ادغام ریاضی (Mathematical Baking):

اگر از یک مدل ترکیب ضربی استفاده کنیم، می‌توانیم امتیاز ایستا را مستقیماً در وزن‌های متنی حل کنیم. فرض کنید تابع امتیازدهی نهایی سیستم چنین ساختاری دارد:

$$\text{FinalScore}(q, d) = \sum_{t \in q} (w_{t,d} \times \text{StaticScore}(d))$$

بر این اساس، در زمان ساخت نمایه، سیستم برای هر جفت (کلمه-سند) یک «امتیاز تأثیر ترکیبی» محاسبه می‌کند:

$$\text{Impact}(t, d) = w_{t,d} \times \text{StaticScore}(d)$$

معماری لیست پستی:

لیست پستی برای هر کلمه t ، دیگر بر اساس **DocID** مرتب نمی‌شود؛ بلکه **اکیداً به صورت نزولی (Descending)** بر اساس مقدار $\text{Impact}(t, d)$ مرتب می‌گردد. در این ساختار، کالاهایی که هم به کلمه پرسمان مرتبط هستند و هم فروش بالایی دارند، در صدر لیست هر کلمه قرار می‌گیرند.

بخش 3) پردازش پرسمان و شرط توقف زودهنگام (Early Termination Condition)

وقتی کاربر پرسمان دو کلمه‌ای «لپ‌تاپ گیمینگ» را جستجو می‌کند، موتور جستجو به جای پیمایش کامل، از الگوریتمی مشابه الگوریتم آستانه فاگین (**Fagin's Threshold Algorithm - TA**) استفاده می‌کند.

روند اجرای پیمایش (Impact-at-a-Time):

1. **پیمایش موازی:** سیستم 2 نشانگر (**Pointer**) روی ابتدای لیست پستی «لپ‌تاپ» (ptr_1) و «گیمینگ» (ptr_2) قرار

می‌دهد و آن‌ها را به صورت موازی به سمت پایین حرکت می‌دهد.

2. **مدیریت نتایج:** یک صف اولویت (**Min-Heap**) با ظرفیت **Top-K** برای نگهداری بهترین کالاها ایجاد می‌شود. کمترین

امتیاز در میان این **K** سند برتر را با متغیر S_{min} می‌شناسیم.

طراحی دقیق شرط توقف (The Termination Trigger):

در هر مرحله از حرکت نشانگرها، سیستم مقادیری که نشانگرها در آن لحظه روی آن‌ها قرار دارند را می‌خواند. از آنجایی که لیست‌ها به صورت نزولی مرتب شده‌اند، عدد زیر نشانگر، **بیشترین امتیاز ممکن** است که اسناد پردازش‌نشده پایین لیست می‌توانند از آن کلمه کسب کنند.

سیستم در هر گام یک **کران بالا (Upper Bound)** برای اسناد ندیده محاسبه می‌کند:

$$UB = \text{Impact}(t_1, ptr_1) + \text{Impact}(t_2, ptr_2)$$

شرط توقف زودهنگام:

الگوریتم به محض برقرار شدن نامعادله زیر، پیمایش هر دو لیست را بلافاصله متوقف می‌کند:

$$UB \leq S_{min}$$

بخش 4) چرا هیچ سند مهمی از دست نمی‌رود؟ (Mathematical Guarantee)

این شرط یک توقف تقریبی نیست، بلکه از نظر ریاضی **100%** قطعی (**Exact**) است. وقتی مجموع بهترین حالت‌های ممکن برای اسناد باقی‌مانده (**UB**)، کوچکتر یا مساوی با ضعیف‌ترین سند در لیست **Top-K** فعلی ما (S_{min}) باشد، کاملاً غیرممکن است که با ادامه دادن لیست، کالایی پیدا کنیم که بتواند جایگاه کالاهای فعلی را در لیست نتایج نهایی تهدید کند. با این مکانیزم، موتور جستجو اغلب با خواندن تنها درصد بسیار کوچکی از ابتدای لیست‌های پستی، عملیات را متوقف کرده و نتایج را در کسری از میلی‌ثانیه به کاربر نمایش می‌دهد.

پاسخنامه مسئله 7.32: معماری پایپ‌لاین بازیابی مقیاس‌پذیر در محیط با حافظه محدود

(Scalable Retrieval Pipeline Architecture in Memory-Constrained Environments)

در مهندسی سیستم‌های بازیابی اطلاعات برای مقیاس‌های وب (مانند 1 میلیارد سند) با محدودیت شدید حافظه اصلی (**RAM**)، نمی‌توان از مدل‌های دقیق و کلاسیک فضای برداری استفاده کرد. برای یافتن 10 سند برتر ($K = 10$)، معماری باید بر پایه «تقریب هوشمند» (**Intelligent Approximation**) و «کوچک‌سازی پیش‌دستانه فضای جستجو» بنا شود.

در ادامه، جریان داده (**Data Flow**) این معماری یکپارچه را در 3 فاز متوالی طراحی و تحلیل می‌کنیم.

بخش 1) نمودار جریان داده و مراحل پردازش (Data Flow Pipeline)

فاز 1: مازول حذف نمایه (Index Elimination)

ورودی: پرسمان خام کاربر (مثلاً شامل 5 کلمه).

عملیات: پردازشگر پرسمان ابتدا فراوانی سند (**df**) را برای هر کلمه از روی یک واژه‌نامه بسیار فشرده در **RAM** می‌خواند. سپس بر اساس طول پرسمان، کلماتی که **df** بسیار بالایی دارند (کلمات عمومی و توقف) را به صورت پویا حذف می‌کند.

خروجی: یک پرسمان هرس‌شده شامل q' کلمه کلیدی با ارزش (**idf** بالا).

فاز 2: واکنشی لیست‌های قهرمان (Champion Lists Retrieval)

ورودی: q' کلمه کلیدی باقیمانده از فاز اول.

عملیات: به جای بارگذاری لیست‌های پستی کامل (که برای 1 میلیارد سند در **RAM** غیرممکن است)، سیستم مستقیماً به «لیست‌های قهرمان» مقیم در **RAM** مراجعه می‌کند. برای هر کلمه، تنها شناسه r سند برتر (مثلاً $r = 5000$) که بالاترین وزن متنی را دارند واکنشی می‌شود.

خروجی: مجموعه‌ای از لیست‌های پستی بسیار کوتاه به طول حداکثر r .

فاز 3: توابع امتیازدهی تقریب‌زننده (Approximated Scoring & Top-K)

ورودی: لیست‌های قهرمان واکنشی‌شده.

عملیات: در این مرحله، سیستم به جای محاسبه دقیق ضرب نقطه‌ای با اعداد اعشاری 32 بیتی (**Float32**)، از وزن‌های کوانتیزه‌شده (**Quantized Weights**) استفاده می‌کند. مقادیر **tf-idf** از پیش به اعداد صحیح 3 بیتی (بازه 1 تا 8) نگاشت شده‌اند. سیستم با استفاده از حساب صحیح سریع (**Fast Integer Arithmetic**) روی این اعداد کوچک، امتیازات تقریب‌خورده را برای اجتماع اسناد موجود در لیست‌های قهرمان محاسبه می‌کند.

خروجی: امتیازات در یک صف اولویت (**Min-Heap**) با ظرفیت $K = 10$ درج شده و 10 سند برتر به کاربر بازگردانده می‌شود.

بخش 2) اثبات معماری: چرا این طراحی حافظه RAM را بهینه مصرف می‌کند؟

برای اثبات بهینگی این پایپ‌لاین، مصرف حافظه را در مدل استاندارد با مدل طراحی‌شده مقایسه می‌کنیم. فرض کنید $N = 10^9$ (یک میلیارد وب‌سایت) و اندازه واژگان کلیدی $|V| = 10^7$ ترم باشد.

1. بهینه‌سازی فضایی با لیست‌های قهرمان ($O(r)$ به جای $O(N)$):

در نمایه استاندارد، طول میانگین لیست پستی تابعی از N است. نگهداری این لیست‌ها در **RAM** نیازمند ترابایت‌ها حافظه است.

در این معماری، با قفل کردن طول لیست به یک عدد ثابت r (مثلاً **5000**)، کل حافظه مورد نیاز برای نگهداری تمام لیست‌های قهرمان در **RAM** برابر است با: $|V| \times r \times \text{DocID Size}$. این ساختار، پیچیدگی فضایی را از $O(N)$ به کران بالای ثابت $O(r)$ تقلیل می‌دهد که به راحتی در چند گیگابایت **RAM** سرور جا می‌گیرد.

2. بهینه‌سازی ساختاری با کوانتیزه‌کردن وزن‌ها در امتیازدهی تقریبی:

اگر سیستم بخواهد برای همان لیست‌های قهرمان، وزن دقیق **tf-idf** را نگه دارد، به ازای هر سند **32** بیت (4 بایت) **RAM** مصرف می‌شود.

با استفاده از تابع امتیازدهی تقریب‌زننده (کوانتیزه‌کردن به **8** سطح تأثیر)، سیستم تنها به $\log_2(8) = 3$ بیت حافظه به ازای هر وزن نیاز دارد. این تکنیک، اندازه لیست‌های قهرمانِ مقیم در حافظه را بیش از **90 درصد** فشرده‌تر می‌کند (کاهش از **32** بیت به **3** بیت به ازای هر رکورد).

3. جلوگیری از سرریز حافظه محاسباتی (Heap Overflow) با حذف نمایه:

با اجرای «حذف نمایه» در خط مقدم پایپ‌لاین، سیستم از ایجاد مجموعه‌های نامزد (**Candidate Sets**) عظیم در زمان اجرا جلوگیری می‌کند. اگر کلمات متداول حذف نمی‌شدند، عملیات اجتماع (**Union**) میان لیست‌ها باعث می‌شد تخصیص حافظه پویا (**Runtime RAM Allocation**) برای نگهداری موقت اسناد و محاسبه امتیازات به شدت بالا برود و باعث سرریز حافظه یا کندی سیستم شود.

بخش 3) نتیجه‌گیری

این معماری اثبات می‌کند که با چشم‌پوشی کنترل‌شده از کسری از دقت (از طریق امتیازدهی تقریبی و برش قهرمانان)، می‌توان یک موتور جستجوی مقیاس‌پذیر برای **1** میلیارد سند طراحی کرد که کل ساختار داده عملیاتی آن (**Operational Data Structure**) درون **RAM** یک سرور استاندارد قرار گیرد و بدون نیاز به دسترسی‌های کندِ دیسک (**Disk I/O**)، نتایج برتر را در مرتبه میلی‌ثانیه بازگرداند.

پاسخنامه مسئله 7.33: طراحی و پیاده‌سازی نمایه‌های چندلایه و لیست‌های قهرمان

(Design and Implementation of Tiered Indexes and Champion Lists)

بخش 1) مقدمه و منطق امتیازدهی

این مسئله یکی از هسته‌های اصلی معماری موتورهای جستجوی مدرن را هدف قرار می‌دهد. برای مدیریت این ساختار، باید فرآیند را به **2** فاز متمایز پیش‌پردازش (آفلاین) و اجرای پرسمان (آنلاین) تقسیم کنیم.

در فاز پیش‌پردازش، تخصیص اسناد به لایه اول (**Champion List**) باید بر اساس ترکیبی از ارتباط متنی (**tf**) و کیفیت ایستا (**PageRank**) صورت گیرد. یک فرمول پایه برای این امتیازدهی به شکل زیر تعریف می‌شود:

$$\text{Combined_Score}(d) = \alpha \cdot tf(t, d) + \beta \cdot PR(d)$$

در ادامه، ساختار کدی پایتون برای اجرای این **2** فاز پیاده‌سازی شده است. مطابق با استانداردهای توسعه نرم‌افزار، تمامی مستندات و کامنت‌های داخل کد به زبان انگلیسی (بدون استفاده از کاراکترهای فارسی) نوشته شده‌اند تا یکپارچگی سورس‌کد حفظ شود.

بخش 2) پیاده‌سازی:

```
tiered_index.py

def build_tiered_index(inverted_index, pagerank_scores, tier1_capacity=50, alpha=0.7, beta=0.3):
    """
    Part A: Offline Preprocessing
    Constructs a two-tier index based on TF and PageRank scores.
    """
    tier1_index = {}
    tier2_index = {}

    for term, postings in inverted_index.items():
        doc_scores = []

        for doc_id, tf_value in postings.items():
            pr_score = pagerank_scores.get(doc_id, 0.0)

            # Calculate combined static and dynamic score
            combined_score = (alpha * tf_value) + (beta * pr_score)
            doc_scores.append((doc_id, combined_score))

        # Sort documents descending based on the combined score
        doc_scores.sort(key=lambda x: x[1], reverse=True)

        # Split into Tier 1 (Champion List) and Tier 2 (Fallback)
        tier1_docs = doc_scores[:tier1_capacity]
        tier2_docs = doc_scores[tier1_capacity:]

        # Store only doc_ids in the final tier structures for fast set operations
        tier1_index[term] = set([doc[0] for doc in tier1_docs])
        tier2_index[term] = set([doc[0] for doc in tier2_docs])

    return tier1_index, tier2_index

def execute_smart_query(query_terms, k_results, tier1_index, tier2_index):
    """
    Part B: Online Query Processing
    Executes intersection on Tier 1 first, conditionally cascading to Tier 2.
    """
    if not query_terms or k_results <= 0:
        return []

    # Step 1: Attempt intersection exclusively on Tier 1
    t1_postings = []
    for term in query_terms:
        t1_postings.append(tier1_index.get(term, set()))

    t1_intersection = set.intersection(*t1_postings) if t1_postings else set()

    # Early Termination: Tier 1 satisfies the requirement
    if len(t1_intersection) >= k_results:
        return list(t1_intersection)[:k_results]

    # Step 2: Smart Cascading to Tier 2
    # We only reach here if Tier 1 results are insufficient (< K)
    combined_postings = []

    for term in query_terms:
        t1_set = tier1_index.get(term, set())
        t2_set = tier2_index.get(term, set())

        # Logical union of both tiers to form the complete postings list
        full_term_set = t1_set | t2_set
        combined_postings.append(full_term_set)

    # Intersection on the complete sets
    full_intersection = set.intersection(*combined_postings) if combined_postings else set()

    # Step 3: Isolate new documents to prevent redundant processing
```

```
new_candidates = full_intersection - t1_intersection
```

```
# Step 4: Merge results, prioritizing Tier 1's high-quality documents
final_results = list(t1_intersection) + list(new_candidates)
```

```
return final_results[:k_results]
```

بخش 3) تحلیل معماری کد

مدیریت فاز آفلاین (build_tiered_index): این تابع به ازای هر کلمه، ابتدا تمامی اسناد را بر اساس یک امتیاز ترکیبی خطی ارزیابی و مرتب می‌کند. این کار تضمین می‌کند که اسناد لایه اول، نه تنها بیشترین تکرار کلمه (**tf**) را دارند، بلکه از نظر ساختار گراف وب (**PageRank**) نیز معتبرترین هستند. استفاده از ساختار داده **set** در انتهای این فاز، برای آماده‌سازی داده‌ها جهت فاز آنلاین حیاتی است.

هوشمندی فاز آنلاین (execute_smart_query): در این بخش، عملگر تقاطع (**set.intersection**) در پایتون که با زبان **C** بهینه‌سازی شده است، ابتدا منحصراً روی لایه اول اجرا می‌شود.

اگر شرط حدنصاب برقرار شود، تابع با بازدهی **100%** خاتمه می‌یابد.

در صورت نیاز به بازگشایی لایه دوم، الگوریتم به جای دور ریختن دستاوردهای قبلی، با استفاده از تفاضل مجموعه‌ها (**full_intersection - t1_intersection**) تنها اسناد جدید را شناسایی می‌کند. این اسناد جدید سپس به انتهای لیست نتایج لایه اول (که کیفیت بالاتری دارند) پیوست می‌شوند تا عدد **K** تکمیل گردد.

پاسخنامه مسئله 7.34: تحلیل پیچیدگی و گلوگاه‌ها در هرس خوشه‌ای سلسله‌مراتبی (Hierarchical Cluster Pruning: Complexity and Bottlenecks)

بخش 1) مقدمه

در هرس خوشه‌ای پایه (سطح)، با انتخاب $\sqrt{N}$ رهبر، پیچیدگی زمانی از مرتبه $O(N)$ به $O(\sqrt{N})$ کاهش می‌یابد. اما با حرکت به سمت یک ساختار درختی و سلسله‌مراتبی (**Hierarchical Clustering**)، می‌توانیم این فضای جستجو را به شکل نمایی و به مراتب قدرتمندتر هرس کنیم. در این ساختار، اسناد در پایین‌ترین سطح (برگ‌ها) قرار می‌گیرند و رهبران در سطوح بالاتر، نماینده خوشه‌های زیرین خود هستند.

بخش 2) اثبات تقلیل پیچیدگی زمانی (Time Complexity Proof)

برای اثبات ریاضی کاهش پیچیدگی زمانی، ساختار درخت را با پارامترهای زیر مدل‌سازی می‌کنیم:

N : تعداد کل اسناد مجموعه.

b : ضریب انشعاب (**Branching Factor**) یا تعداد فرزندان هر گره رهبر در درخت.

h : عمق درخت (**Depth**).

1. مدل‌سازی ساختار درخت:

در یک درخت متوازن، در سطح اول (ریشه) b رهبر اصلی داریم. هر یک از این رهبران، خود نماینده b رهبر در سطح پایین‌تر هستند. این روند تا رسیدن به برگ‌ها (تک‌تک اسناد) ادامه می‌یابد. بنابراین، رابطه بین تعداد کل اسناد و عمق درخت به صورت

$$N = b^h$$

با گرفتن لگاریتم از دو طرف معادله، عمق درخت برابر خواهد بود با:

$$h = \log_b(N)$$

2. محاسبه زمان پردازش (Online Processing Time):

هنگامی که پرسمان کاربر (q) وارد سیستم می‌شود، جستجو از ریشه درخت آغاز می‌گردد:

در سطح اول، پرسمان با b رهبر مقایسه می‌شود تا شبیه‌ترین رهبر انتخاب شود (تعداد محاسبات $= b$).

سیستم وارد شاخه رهبر برنده شده و در سطح دوم، مجدداً پرسمان را با b فرزند آن رهبر مقایسه می‌کند (تعداد محاسبات $= b$).

این فرآیند پایین رفتن درخت، دقیقاً به اندازه عمق درخت (h) تکرار می‌شود تا سیستم به برگ‌ها و اسناد نهایی برسد.

بنابراین، کل عملیات محاسبه شباهت کسینوسی (**Total Comparisons**) برابر است با:

$$\text{Total Comparisons} = h \cdot b$$

با جایگذاری h در معادله:

$$\text{Total Comparisons} = b \cdot \log_b(N)$$

3. نتیجه‌گیری اثبات:

از آنجا که b یک عدد ثابت و بسیار کوچک‌تر از N است، ضریب ثابت در تحلیل مجانبی نادیده گرفته می‌شود. در نتیجه، جستجوی سلسله‌مراتبی، پیچیدگی زمانی وحشتناک خطی در جستجوی جامع را به **مرتبه زمانی لگاریتمی** تقلیل می‌دهد:

$$\mathcal{O}(\log N)$$

بخش 3) تحلیل گلوگاه مهندسی (Bottleneck Analysis)

گلوگاه «خطای انحراف در ریشه» (Early Routing Error):

بزرگترین نقطه ضعف و گلوگاه این معماری، شکنندگی آن در برابر تصمیمات سطوح بالاست. اگر یک پرسمان از نظر معنایی در مرز بین دو خوشه قرار داشته باشد (**Ambiguous Query**)، ممکن است به دلیل اختلاف بسیار ناچیز در شباهت کسینوسی، سیستم رهبر **A** را به جای رهبر **B** در همان سطح اول (ریشه) انتخاب کند.

از آنجا که درخت، شاخه‌های متفاوتی دارد، یک انتخاب اشتباه در بالا باعث می‌شود جستجو به طور کامل وارد یک زیردرخت اشتباه شود. در این حالت، تمام اسناد به شدت مرتبطی که در شاخه رهبر **B** قرار داشتند تا انتها نادیده گرفته می‌شوند. این گلوگاه منجر به افت فاجعه‌بار شاخص بازخوانی (Recall Degradation) می‌گردد.

بخش 4) مکانیزم و راهکار دور زدن گلوگاه (Proposed Solution)

برای حل این مشکل، معماری جستجو باید از حالت «مسیر واحد» (**Single-path**) خارج شده و به «پیمایش چندمسیری» (**Multi-path Traversal**) ارتقا یابد. این راهکار در ادبیات هوش مصنوعی به **Beam Search** معروف است:

الگوریتم پیمایش با پرتو (Beam Search in Hierarchical Trees):

به جای آنکه در هر سطح منحصراً **1** رهبر برنده انتخاب شود، سیستم یک پارامتر انعطاف‌پذیری به نام «عرض پرتو» (k) تعریف می‌کند (مثلاً $k = 2$ یا $k = 3$).

نحوه عملکرد: در سطح اول، سیستم پرسمان را با رهبران مقایسه کرده و به جای یک شاخه، k شاخه برتر می‌شود.

در سطح دوم، سیستم پرسمان را با فرزندان هر k شاخه ارزیابی کرده و مجدداً از بین تمام آن‌ها، k رهبر محلی برتر را برای ادامه مسیر به پایین انتخاب می‌کند.

دستاوردها: این مکانیزم حاشیه امنیت (**Safety Margin**) سیستم را به شدت افزایش می‌دهد. اگر پرسمان در مرز دو خوشه کلان قرار داشته باشد، سیستم به طور همزمان هر دو مسیر را تا رسیدن به برگ‌ها کاوش می‌کند. این کار، خطای انحراف در ریشه را خنثی کرده و بازخوانی (**Recall**) را به سطح ایده‌آل بازمی‌گرداند، در حالی که پیچیدگی زمانی همچنان در مرتبه بسیار سریع $O(k \cdot b \log N)$ یا همان $O(\log N)$ باقی می‌ماند.